

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO
ĐỒ ÁN CHUYÊN NGÀNH

Lập lịch Phân bổ Tài nguyên trong
Datacenter sử dụng
Học Tăng Cường Đa Mục Tiêu
Ngành: Khoa học Máy tính

HỘI ĐỒNG:

GVHD 1: NGUYỄN QUANG HÙNG

—o0o—

SVTH 1: VŨ TRẦN DUY NGUYỄN - 2212338

TP. HỒ CHÍ MINH, THÁNG 05 NĂM 2026

Lời cảm ơn

Đồ án này là kết quả của một hành trình học tập, nghiên cứu và thực hành nghiêm túc trong suốt học kỳ vừa qua. Để hoàn thành công trình này, em đã nhận được sự hỗ trợ và đồng hành của thầy, của khoa mà em muốn bày tỏ lòng biết ơn sâu sắc.

Trước tiên, em xin gửi lời cảm ơn chân thành nhất đến **Thầy Nguyễn Quang Hùng**, người hướng dẫn đồ án của em. Thầy đã không chỉ định hướng vấn đề nghiên cứu và gợi mở các góc nhìn kỹ thuật quan trọng, mà còn kiên nhẫn theo sát tiến độ, cung cấp phản hồi kịp thời và khuyến khích em tư duy độc lập trước những thách thức kỹ thuật phức tạp. Sự tận tâm và chuyên môn sâu rộng của Thầy là nguồn động lực lớn trong suốt quá trình thực hiện đồ án.

Em xin cảm ơn **Khoa Khoa học và Kỹ thuật Máy tính**, Trường Đại học Bách Khoa — ĐHQG TP.HCM, đã tạo điều kiện môi trường học thuật và cơ sở hạ tầng nghiên cứu thuận lợi. Những kiến thức nền tảng về hệ thống phân tán, học máy và thiết kế phần mềm mà em tích lũy qua các học phần của Khoa là tiền đề không thể thiếu để xây dựng hệ thống này.

Em cũng bày tỏ lòng biết ơn đến **gia đình** đã luôn ủng hộ và tạo điều kiện tốt nhất để em tập trung vào việc học tập và nghiên cứu.

Mặc dù đã nỗ lực hết sức, đồ án chắc chắn vẫn còn những thiếu sót. Em kính mong nhận được ý kiến đóng góp từ Hội đồng và Quý Thầy Cô để tiếp tục hoàn thiện trong giai đoạn tiếp theo.

TP. Hồ Chí Minh, tháng 05 năm 2026

Vũ Trần Duy Nguyên

Tóm tắt

Tên đề tài: Lập lịch Phân bổ Tài nguyên trong Datacenter sử dụng Học Tăng Cường Đa Mục Tiêu

Sinh viên thực hiện: Vũ Trần Duy Nguyên — MSSV: 2212338

Giảng viên hướng dẫn: Nguyễn Quang Hùng

Sự bùng nổ của các mô hình AI quy mô lớn (LLM, Diffusion Model) đặt ra áp lực chưa từng có lên hạ tầng datacenter: lượng tác vụ GPU ngày càng đa dạng về yêu cầu tài nguyên và ràng buộc thời gian, trong khi hai mục tiêu vận hành cốt lõi — *tiết kiệm năng lượng* và *đảm bảo Chất lượng Dịch vụ (SLA)* — liên tục mâu thuẫn nhau. Các thuật toán lập lịch truyền thống (First-Fit, Kubernetes Default) tối ưu hóa đơn mục tiêu, không nắm bắt được sự đánh đổi (trade-off) phức tạp giữa hai chiều này.

Đề án này đề xuất và hiện thực **ELDAS** (*Energy-aware Learning-based Datacenter Auto-Scheduler*), một nền tảng mô phỏng và lập lịch tự động dựa trên **Học Tăng Cường Đa Mục Tiêu (MORL)**. Hệ thống được xây dựng theo kiến trúc 4 lớp: (1) hạ tầng Docker Compose tái sản xuất được, (2) engine mô phỏng datacenter CloudSim Plus với mô hình năng lượng tuyến tính và theo dõi GPU, (3) cầu nối RPC Java–Python qua Py4J, và (4) môi trường học tăng cường MO-Gymnasium tích hợp agent PPO với che mặt hành động (Action Masking).

Tác vụ đầu vào được lấy từ **Alibaba GPU Cluster Trace v2023** — bộ dữ liệu workload thực của trung tâm dữ liệu sản xuất với hơn 6.000 pod — được phân thành ba kịch bản: Low Load, High Load và Burst. Vector phần thưởng hai chiều $\mathbf{r} = (R_{\text{energy}}, R_{\text{SLA}})$ phản ánh trực tiếp hai mục tiêu xung đột, trong đó mức phạt SLA được điều chỉnh theo lớp QoS của tác vụ ($\lambda \in \{1.0, 3.0\}$). Chiến lược *linear scalarization* kết hợp quét trọng số $w \in [0.1, 0.9]$ cho phép xây dựng **Pareto front** thể hiện toàn bộ tập giải pháp tối ưu trade-off.

Giai đoạn 1 của đề án đã hoàn thành toàn bộ nền tảng kỹ thuật: môi trường mô phỏng được xác minh qua vòng lặp RL đầu đến cuối (reset $\rightarrow$ step $\rightarrow$ reward $\rightarrow$ done), hai baseline scheduler (Random và K8s Default) được tích hợp để so sánh. Giai đoạn 2 (kế hoạch) sẽ tiến hành huấn luyện agent PPO+MLP, thực hiện weight sweep, và so sánh MORL với baseline trên cả ba kịch bản tải.

Từ khóa: Học tăng cường đa mục tiêu, lập lịch tác vụ datacenter, tối ưu hóa năng lượng, SLA, CloudSim Plus, MO-Gymnasium, PPO, Alibaba Trace.

Abstract

Thesis title: Resource Allocation Scheduling in Datacenters using Multi-Objective Reinforcement Learning

Student: Vu Tran Duy Nguyen — ID: 2212338

Supervisor: Nguyen Quang Hung

The rapid proliferation of large-scale AI workloads — including Large Language Models and Diffusion Models — has placed unprecedented demands on modern datacenter infrastructure. GPU tasks arrive continuously with heterogeneous resource requirements and tight timing constraints, while two core operational objectives persistently conflict: *energy efficiency* and *Quality-of-Service (SLA) compliance*. Conventional single-objective schedulers such as Kubernetes Default or First-Fit Decreasing cannot capture the complex Pareto trade-off between these competing dimensions.

This thesis proposes and implements **ELDAS** (*Energy-aware Learning-based Datacenter Auto-Scheduler*), a simulation and scheduling platform grounded in **Multi-Objective Reinforcement Learning (MORL)**. The system follows a four-layer architecture: (1) a fully reproducible Docker Compose infrastructure, (2) a CloudSim Plus datacenter simulation engine equipped with a linear power model and custom GPU tracking, (3) a Java–Python RPC bridge via Py4J enabling low-latency inter-process communication, and (4) a MO-Gymnasium reinforcement learning environment integrated with a Maskable-PPO agent from Stable-Baselines3-Contrib.

Input workloads are sourced from the **Alibaba GPU Cluster Trace v2023**, a real production trace comprising over 6,000 pod records, partitioned into three evaluation scenarios: Low Load, High Load, and Burst. The reward signal is a two-dimensional vector $\mathbf{r} = (R_{\text{energy}}, R_{\text{SLA}})$ that directly captures both objectives; per-task SLA penalties are scaled by a QoS-tier factor $\lambda \in \{1.0, 3.0\}$ to reflect the urgency of latency-sensitive versus burstable jobs. A *linear scalarization* strategy with a weight sweep $w \in [0.1, 0.9]$ is employed to generate the full **Pareto front** of energy–SLA trade-off solutions.

Phase 1 of the project delivers a complete, verified simulation stack: the end-to-end RL loop (reset $\rightarrow$ step $\rightarrow$ reward $\rightarrow$ done) passes a smoke test over Py4J, and two baseline schedulers (Random and K8s Default) are integrated for controlled comparison. Phase 2 (planned) will train the PPO+MLP agent, perform the weight sweep, and benchmark MORL against baselines across all three workload scenarios, culminating in a quantitative Pareto front and energy/makespan/SLA-violation comparison table.

Keywords: multi-objective reinforcement learning, datacenter task scheduling, energy optimization, SLA compliance, CloudSim Plus, MO-Gymnasium, Proximal Policy Optimization, Alibaba Cluster Trace.

Mục lục

Lời cảm ơn	1
Tóm tắt	2
Abstract	3
1 Giới thiệu	14
1.1 Bối cảnh và động lực nghiên cứu	14
1.2 Phát biểu bài toán	15
1.2.1 Đầu vào và đầu ra	15
1.2.2 Hai mục tiêu cần tối ưu cùng lúc	16
1.2.3 Vì sao hai mục tiêu này xung đột?	16
1.2.4 Độ phức tạp tính toán	17
1.3 Mục tiêu đề tài	17
1.3.1 Mục tiêu tổng thể	17
1.3.2 Mục tiêu Giai đoạn 1 (phạm vi báo cáo này)	17
1.3.3 Mục tiêu Giai đoạn 2 (kế hoạch tiếp nối)	18
1.4 Phạm vi và giới hạn	18
1.5 Cấu trúc báo cáo	20
2 Cơ sở lý thuyết và nghiên cứu liên quan	21
2.1 Lập lịch trong trung tâm dữ liệu	21
2.1.1 Các thuật toán heuristic truyền thống	21
2.1.2 Meta-heuristic: Ant Colony Optimization (ACO)	22
2.1.3 Kubernetes Scheduler	22
2.1.4 Lập lịch dựa trên học tăng cường: Deep Q-Network (DQN)	23
2.2 Mô hình tiêu thụ năng lượng datacenter	24
2.2.1 Mô hình tuyến tính SPECpower	24
2.2.2 Tính tổng năng lượng theo thời gian rời rạc	25
2.2.3 Mở rộng cho GPU	25
2.2.4 Power Usage Effectiveness (PUE)	25
2.2.5 Lý do chọn mô hình tuyến tính cho Phase 1	25
2.3 Học tăng cường đa mục tiêu (MORL)	26
2.3.1 Markov Decision Process (MDP)	26

2.3.2	Multi-Objective MDP (MOMDP)	26
2.3.3	Pareto Optimality	26
2.3.4	Proximal Policy Optimization (PPO)	27
2.3.5	Linear Scalarization và quét trọng số	27
2.3.6	Action Masking	27
2.4	Nghiên cứu liên quan	28
2.4.1	DeepRM (Mao và cộng sự, 2016)	28
2.4.2	Decima (Mao và cộng sự, 2019)	28
2.4.3	NSGA-II cho điều phối đám mây tiết kiệm năng lượng	28
2.4.4	Beware of Fragmentation (Weng và cộng sự, 2023)	28
2.4.5	MO-Gymnasium (Alegre và cộng sự, 2022)	28
2.5	Công nghệ nền tảng	29
2.5.1	CloudSim Plus	29
2.5.2	Py4J – cầu nối Java–Python	29
2.5.3	MO-Gymnasium	30
2.5.4	Docker Compose	30
2.5.5	So sánh với KWOK	30
2.6	Kết luận chương	30
3	Thiết kế kiến trúc hệ thống	31
3.1	Tổng quan kiến trúc bốn lớp	31
3.1.1	Vì sao chia thành bốn lớp?	31
3.1.2	Vai trò từng lớp	32
3.1.3	Một bước trong vòng lặp RL diễn ra thế nào?	32
3.2	Lựa chọn giải pháp và biện luận	33
3.2.1	Lựa chọn công cụ mô phỏng: KWOK vs. CloudSim Plus	33
3.2.2	Lựa chọn cầu nối ngôn ngữ: Py4J vs. gRPC vs. REST	34
3.2.3	Lựa chọn thuật toán RL: PPO vs. DQN vs. A2C	34
3.2.4	Lựa chọn chiến lược MORL: Linear Scalarization vs. NSGA-II	34
3.2.5	Bảng tổng hợp ma trận quyết định	35
3.3	Thiết kế Lớp 2 — Phân hệ CloudSim Plus	35
3.3.1	SimulationConfig – cấu hình tập trung	36
3.3.2	DatacenterFactory và lớp theo dõi GPU	36
3.3.3	AlibabaTraceReader – đọc trace	36
3.3.4	ScenarioFilter – ba kịch bản tải	37
3.3.5	SimulationManager – ghép mô phỏng vào vòng lặp RL	37
3.4	Thiết kế Lớp 3 — Py4J Gateway	38
3.4.1	API surface tối giản	38
3.4.2	Khởi động Gateway và kết nối client	38
3.5	Thiết kế Lớp 4 — MO-Gymnasium Environment	39
3.5.1	Observation space	39
3.5.2	Action space và action masking	39
3.5.3	Vector reward và linear scalarization	39

3.5.4	Điều kiện kết thúc episode	40
3.5.5	Tóm tắt vòng lặp	40
3.6	Thiết kế Lớp 1 — Docker Compose Infrastructure	40
3.6.1	Bố trí container và volume	40
3.6.2	Multi-stage build và tối ưu kích thước image	40
3.6.3	Healthcheck và thứ tự khởi động	40
3.6.4	Tính tái sản xuất	41
3.7	Tổng kết chương	41
4	Hiện thực hệ thống	42
4.1	Hiện thực phân hệ Java	42
4.1.1	Cấu hình dự án Maven	42
4.1.2	DatacenterFactory – xây dựng datacenter và registry GPU	43
4.1.3	AlibabaTraceReader – parser chịu lỗi	44
4.1.4	Hai baseline schedulers	44
4.1.5	SimulationManager – chi tiết kỹ thuật quan trọng	45
4.1.6	MetricsExporter – xuất CSV + JSON không phụ thuộc	46
4.2	Hiện thực phân hệ Python	46
4.2.1	state_builder.py – chuyển đổi an toàn và <i>parsable</i>	46
4.2.2	reward.py – vector, normalisation, scalarisation	47
4.2.3	environment.py – gắn kết với Gymnasium API	47
4.2.4	tracker.py – WandB online/offline tự động	48
4.2.5	baseline_eval.py – so sánh fair	48
4.3	Hiện thực hạ tầng Docker	49
4.3.1	Pipeline build hai stage	49
4.3.2	Quản lý volume	50
4.3.3	Vấn đề kỹ thuật gặp phải và giải pháp	50
4.4	Smoke test end-to-end	51
4.5	Tổng kết chương	52
5	Kết quả và đánh giá	53
5.1	Cấu hình thực nghiệm	53
5.2	Kết quả smoke test end-to-end	54
5.2.1	Throughput của vòng lặp RL	55
5.2.2	Tính chất reward đo được	55
5.3	Kết quả baseline trên ba kịch bản tải	56
5.3.1	Phương pháp đo	56
5.3.2	Bảng kết quả tổng hợp	56
5.3.3	Phân tích định tính	57
5.3.4	Trực quan hoá	58
5.3.5	Tính lặp lại của hệ thống	59
5.4	Phân tích đặc trưng dữ liệu Alibaba Trace	59
5.4.1	Thống kê tổng hợp	59

5.4.2	Phân bố ba kịch bản sau lọc	60
5.5	Đánh giá tổng thể Phase 1	61
5.5.1	Đối chiếu sáu mục tiêu MT1–MT6	61
5.5.2	Hạn chế hiện tại và các bất thường dữ liệu cần xử lý ở Phase 2	62
5.5.3	Sẵn sàng cho Phase 2	63
6	Kế hoạch Giai đoạn 2	64
6.1	Mục tiêu cụ thể của Phase 2	64
6.2	Lộ trình triển khai (8 tuần)	65
6.3	Tiêu chí chấp nhận định lượng	66
6.4	Quản lý rủi ro	67
6.5	Hướng mở rộng dài hạn	67
7	Kết luận	69
7.1	Tóm tắt đóng góp Phase 1	69
7.2	Bài học kinh nghiệm	70
7.3	Lời kết	70
A	Hướng dẫn cài đặt và chạy ELDAS	73
B	Mã nguồn quan trọng	75
C	Bảng từ viết tắt	77

Danh sách hình vẽ

1.1	Xu hướng tiêu thụ điện năng của trung tâm dữ liệu, AI và tiền mã hóa trên toàn cầu giai đoạn 2018–2026. Nguồn: IEA [3].	14
1.2	Trade-off giữa năng lượng và vi phạm SLA. Mỗi điểm là một chính sách lập lịch; đường cong nối các điểm không bị thống trị (Pareto front) là kết quả mà Phase 2 hướng tới xấp xỉ.	17
2.1	So sánh mô hình công suất tuyến tính và phi tuyến.	26
2.2	Minh họa Pareto front trong không gian hai mục tiêu.	27
3.1	Kiến trúc bốn lớp của ELDAS. Mỗi đường đứt nét là một biên cô lập về ngôn ngữ, container, hoặc hạ tầng. Mũi tên đậm đánh dấu kênh giao tiếp hai chiều của vòng lặp RL.	32
3.2	Sequence diagram cho một vòng lặp RL. Hai <code>SynchronousQueue</code> ở phía Java đảm bảo simulation thread chỉ tiến bước khi nhận được hành động hợp lệ.	33
3.3	Class diagram phân hệ Java. Mũi tên đậm biểu diễn quan hệ <i>depends-on</i> ; <code>GatewayEntryPoint</code> là điểm vào duy nhất của Python.	36
3.4	Vòng đời <code>SimulationManager</code> trong một episode. Hai vùng “ <code>actionQueue.take</code> ” và “ <code>stepResultQueue.put</code> ” là hai điểm đồng bộ giữa thread mô phỏng và thread Py4J.	37
3.5	Cấu trúc vector quan sát ($3H + 4$) chiều. Ba khối host được sắp liên tiếp theo loại tài nguyên; bốn ô cuối mô tả tác vụ kế tiếp cần được xếp lịch.	39
3.6	Topology Docker Compose. <code>rl-agent</code> không expose port nào (client thuần); <code>cloudsim-java</code> chỉ expose <code>\$PY4J_PORT</code>	40
4.1	Kết quả <code>docker compose build</code> hoàn tất cho cả hai service trên máy phát triển.	49
5.1	Hai container hoạt động ổn định và <code>cloudsim-java</code> vượt qua healthcheck Py4J, điều kiện tiên quyết cho mọi smoke test và baseline run sau đây.	54
5.2	Smoke test tám pha hoàn tất thành công.	55
5.3	So sánh năng lượng tích lũy giữa K8s LRP và Random trên ba kịch bản. K8s tiết kiệm 1,0%–1,9% tùy kịch bản; biên độ này nhỏ hơn kỳ vọng và được phân tích ở §5.3.3.	58
5.4	Tỉ lệ vi phạm SLA. <i>Tất cả các thanh đều bằng 0</i> – không phải vì hai scheduler hoàn hảo, mà vì đồng hồ DES chưa hoạt động ở Phase 1 nên chỉ số này chưa có khả năng phân biệt. Hình giữ trong báo cáo để minh bạch tình trạng dữ liệu.	58

5.5	Năng lượng tích lũy theo timestamp mô phỏng cho ba kịch bản. Trục thời gian phản ánh thời lượng <i>logical</i> của trace (không phải wall-clock); hai đường K8s/Random gần như chồng lên nhau, khớp với khoảng cách 1–2% ở Bảng 5.3.	58
5.6	Bảng chứng tính lặp lại bit-for-bit của hệ thống ELDAS với cùng RANDOM_SEED . .	59
5.7	Phân bố số GPU yêu cầu trên trace (7 255 tác vụ). 84,5% tác vụ yêu cầu đúng 1 GPU; 14,5% không cần GPU; <1% yêu cầu ≥ 2 GPU. Trục y thang log để các thanh thiếu số vẫn nhìn được.	61
5.8	Phân bố lớp QoS toàn trace. Tỷ lệ <i>LatencySensitive</i> cao (57,8%) là cơ sở để thiết kế λ ở §3.3: bất kỳ vi phạm SLA nào ở Phase 2 cũng có trọng số phạt khuếch đại $3\times$ đáng kể.	61
6.1	Sơ đồ Gantt của Phase 2. Các hộp xanh là task chạy được, hộp cam là nhóm gộp, mốc đỏ là deliverable cuối. Mũi tên thể hiện phụ thuộc cứng.	66

Danh sách bảng

1.1	Hệ số phạt λ cho từng lớp QoS – vi phạm SLA của tác vụ tương tác bị phạt nặng gấp 3 lần tác vụ thường.	16
2.1	So sánh các thuật toán lập lịch truyền thống.	24
2.2	Bảng tổng hợp so sánh các nghiên cứu liên quan.	29
2.3	So sánh các tùy chọn cầu nối Java–Python cho ELDAS.	29
2.4	So sánh KWOK và CloudSim Plus theo các tiêu chí của ELDAS.	30
3.1	Ma trận quyết định bốn lựa chọn kiến trúc của ELDAS. Ký hiệu: $\checkmark$ = đáp ứng tốt, $\sim$ = chấp nhận được, $\times$ = không đáp ứng.	35
3.2	Ảnh xạ trường Alibaba trace $\rightarrow$ đối tượng <code>TaskRecord</code>	37
3.3	API công khai của <code>GatewayEntryPoint</code> . Mọi tham số/giá trị trả về đều là kiểu nguyên thủy, mảng nguyên thủy, hoặc Java <code>record</code> đơn giản – thân thiện với Py4J và serializable cô lập.	38
3.4	Ảnh xạ ngược từ mục tiêu Phase 1 (§1.3) sang thành phần kiến trúc.	41
4.1	Quy mô mã nguồn Phase 1 (đếm bằng <code>wc -l</code> , không trừ comment/blank line). Tổng 3188 dòng.	42
4.2	Kích thước Docker image cuối cùng	49
4.3	Sáu vấn đề kỹ thuật đáng ghi nhận trong Phase 1 và giải pháp.	51
4.4	Tám pha của smoke test T4.5.	51
5.1	Cấu hình môi trường thực nghiệm Phase 1.	53
5.2	Kết quả tám pha smoke test (kịch bản LOW, seed = 42). Thời gian <i>tổng</i> đo bằng wall-clock; thời gian từng pha không được instrument riêng nên cột bên phải ghi <code>n/a</code>	54
5.3	Kết quả baseline trên ba kịch bản (cùng seed = 42, cùng trace). Số liệu trích trực tiếp từ <code>summary.json</code> của mỗi run. Đơn vị các cột E và $\bar{u}^{\text{cpu}}$ giữ nguyên label gốc trong code (xem §5.5.2 về tính bất thường của đơn vị).	56
5.4	Thống kê mô tả Alibaba PAI Trace <code>openb_pod_list_default.csv</code> sau khi qua <code>AlibabaTraceReader.read</code> (chỉ giữ các pod <i>không</i> ở trạng thái Pending; tổng 7255 tác vụ).	59
5.5	Số lượng tác vụ và đặc trưng trung bình sau khi <code>ScenarioFilter</code> lọc. Cột <code>cpu_milli</code> và <code>%LS</code> yêu cầu instrumentation per-scenario chưa có trong Phase 1.	60

5.6	Checklist hoàn thành sáu mục tiêu Phase 1 với artifact bằng chứng có thể kiểm tra độc lập.	62
6.1	Sáu hạng mục công việc của Phase 2 và đầu ra dự kiến.	65
6.2	Lộ trình Phase 2 theo tuần. Kết quả mong đợi của mỗi tuần là điều kiện tiên quyết để chuyển sang tuần kế.	65
6.3	Bốn tiêu chí chấp nhận của Phase 2. Tất cả phải đạt ✓ mới coi là Phase 2 thành công.	66
6.4	Phân tích rủi ro Phase 2. Thang P/I : thấp / trung / cao.	67

Listings

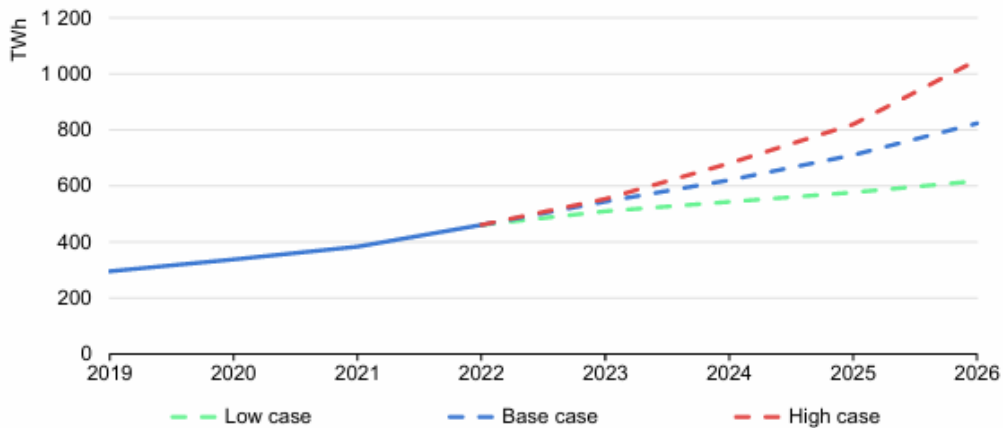
2.1	Khởi chạy hệ thống.	30
4.1	<code>pom.xml</code> – trích đoạn hai dependency chính.	43
4.2	<code>AlibabaTraceReader.read()</code> – tolerant CSV parsing.	44
4.3	<code>VmAllocationPolicyK8sDefault</code> – Filter → Score → Select.	44
4.4	Manual resource accounting. Đây là <i>nguồn sự thật</i> duy nhất cho utilisation trong Phase 1.	45
4.5	<code>CloudSimEnv.reset</code> – tránh reset thừa lần đầu.	47
4.6	<code>action_masks()</code> – fail-open để không khoá episode.	48
4.7	<code>baseline_eval.evaluate_baseline</code> – tận dụng chung pipeline với RL.	49
4.8	Trick layer cache: <code>pom.xml</code> trước, <code>src/</code> sau.	50
4.9	Lệnh chạy smoke test.	51
5.1	Lệnh chạy smoke test.	54
5.2	Lệnh chạy ba kịch bản (toàn bộ orchestration bởi <code>collect_all_data.py</code>).	56
5.3	Kiểm tra tính lặp lại (Phase B.3).	59
A.1	Quy trình tối thiểu để dựng lại toàn bộ hệ thống.	73
A.2	Layout of generated data.	74
B.1	Placeholder for <code>docker-compose.yml</code>	75
B.2	Placeholder for <code>SimulationManager.java</code>	75
B.3	Placeholder for <code>environment.py</code>	75
B.4	Placeholder for <code>reward.py</code>	76

Chương 1

Giới thiệu

1.1 Bối cảnh và động lực nghiên cứu

Trong nửa sau của thập kỷ 2020, sự bùng nổ của các mô hình ngôn ngữ lớn (*Large Language Models* – LLM) như GPT-4, LLaMA-3, Gemini và Claude đã đẩy nhu cầu tính toán trên bộ tăng tốc đồ họa (GPU) tăng theo cấp số nhân. Các phiên huấn luyện một mô hình tiên tiến hiện nay đòi hỏi hàng nghìn GPU NVIDIA H100 hoặc A100 chạy liên tục trong nhiều tuần, kéo theo sự mở rộng quy mô chưa từng có của các trung tâm dữ liệu (*datacenter*) chuyên dụng cho trí tuệ nhân tạo (AI) [1], [2]. Theo báo cáo *Electricity 2024* của Cơ quan Năng lượng Quốc tế (IEA) [3], tổng năng lượng tiêu thụ của các trung tâm dữ liệu, AI và tiền mã hóa trên toàn cầu trong năm 2022 ước đạt khoảng 460 TWh – tương đương 1,5% tổng nhu cầu điện năng thế giới – và dự báo sẽ tăng gấp đôi, chạm ngưỡng 1000 TWh vào năm 2026, vượt qua tổng tiêu thụ điện của cả nước Nhật Bản.



Hình 1.1: Xu hướng tiêu thụ điện năng của trung tâm dữ liệu, AI và tiền mã hóa trên toàn cầu giai đoạn 2018–2026. Nguồn: IEA [3].

Tuy nhiên, cùng với tốc độ mở rộng đó là một nghịch lý đáng lo ngại về hiệu suất sử dụng tài nguyên. Phân tích vết tải (*trace*) công khai của Alibaba PAI Cluster v2023 [4] cho thấy: mặc dù chi phí GPU chiếm tới hơn 80% tổng giá thành phần cứng của một AI cluster, hiệu suất sử dụng GPU trung bình chỉ đạt $\approx 25\text{--}40\%$. Nguyên nhân chính nằm ở hiện tượng **phân mảnh tài nguyên** (*Resource Fragmentation*): một host có thể còn dư GPU nhưng không còn

đủ CPU/RAM để nhận thêm tác vụ, hoặc ngược lại, dẫn đến những GPU “mồ côi” (*orphan GPUs*) tiêu tốn năng lượng ở trạng thái *idle* mà không sinh giá trị tính toán [5], [6].

Bài toán càng trở nên phức tạp khi nhà vận hành phải đồng thời theo đuổi **hai mục tiêu xung đột**:

1. **Tiết kiệm năng lượng** – giảm tổng điện năng tiêu thụ và lượng khí thải CO₂ tương ứng;
2. **Đảm bảo Thỏa thuận Mức Dịch vụ** (*Service Level Agreement* – SLA) – hoàn thành tác vụ trước hạn chót, đặc biệt là các tác vụ *LatencySensitive*.

Hai mục tiêu này thường đối lập: việc *consolidation* tác vụ vào ít host hơn để tắt bớt các host nhàn rỗi giúp tiết kiệm điện, nhưng lại làm tăng độ trễ xếp hàng và nguy cơ vi phạm SLA. Ngược lại, phân tán tải để đảm bảo SLA sẽ khiến nhiều host vận hành ở mức tải thấp.

Trong bối cảnh Việt Nam, khi giá điện công nghiệp liên tục được điều chỉnh tăng và các cam kết Net-Zero 2050 đặt áp lực mạnh lên ngành công nghiệp tính toán, việc xây dựng một nền tảng nghiên cứu cho phép đánh giá và phát triển các thuật toán lập lịch **vừa tiết kiệm năng lượng vừa đảm bảo chất lượng dịch vụ** trở thành một nhu cầu mang tính cấp thiết và có giá trị thực tiễn rõ rệt.

1.2 Phát biểu bài toán

1.2.1 Đầu vào và đầu ra

Mô tả bài toán bằng ngôn ngữ kỹ thuật, không hình thức hoá quá mức:

Đầu vào. Hệ thống nhận:

- Một cụm gồm m **host** vật lý. Mỗi host có dung lượng cố định về CPU (đơn vị MIPS), RAM (MB), GPU (số chiếc) và băng thông (Mbps).
- Một dòng **tác vụ** đến theo thời gian. Mỗi tác vụ τ_i khai báo nhu cầu tài nguyên $(r_i^{\text{cpu}}, r_i^{\text{ram}}, r_i^{\text{gpu}})$, thời hạn hoàn thành d_i và nhãn chất lượng dịch vụ $q_i \in \{\text{Burstable}, \text{LatencySensitive}, \text{BestEffort}\}$ – lấy từ Alibaba trace.

Đầu ra. Một quyết định: *tác vụ τ_i được đặt lên host nào, hay tạm hoãn lại đến bước kế tiếp*. Ký hiệu phép gán này là $f(\tau_i)$.

Ràng buộc bắt buộc. Tại mọi thời điểm, tổng nhu cầu tài nguyên của các tác vụ đang chạy trên một host không được vượt quá dung lượng của host đó:

$$\text{Tổng } r_i^{(k)} \text{ trên host } h_j \leq c_j^{(k)}, \quad k \in \{\text{cpu}, \text{ram}, \text{gpu}\}. \quad (1.1)$$

Đây là điều kiện cứng – vi phạm là không khả thi, không phải chỉ “tệ hơn”. Hệ thống thực hiện ràng buộc này qua cơ chế **action masking** (Chương 3) – không cho phép RL agent chọn host không đủ tài nguyên ngay từ không gian hành động.

1.2.2 Hai mục tiêu cần tối ưu cùng lúc

Mục tiêu 1 – Tiết kiệm năng lượng. Mỗi host h_j tại thời điểm t tiêu thụ công suất $P_j(t)$ phụ thuộc mức sử dụng $U_j(t) \in [0, 1]$. ELDAS dùng mô hình tuyến tính *SPECpower* [7]: công suất bằng P^{idle} khi host nhàn rỗi, tăng tuyến tính tới P^{max} khi $U_j = 1$:

$$P_j(U_j) = P_j^{\text{idle}} + U_j \cdot (P_j^{\text{max}} - P_j^{\text{idle}}). \quad (1.2)$$

Tổng năng lượng của một episode mô phỏng là tổng năng lượng trên toàn bộ host theo thời gian:

$$E = \sum_{\text{host}} \sum_{\text{tick}} P_j(t) \Delta t. \quad (1.3)$$

Đây chỉ là phép cộng dồn công suất nhân với độ dài tick theo thời gian – không phải phép tích phân giải tích phức tạp. Mục tiêu là giảm E càng nhiều càng tốt.

Mục tiêu 2 – Hoàn thành tác vụ trước hạn (SLA). Một tác vụ “trễ hạn” khi thời điểm hoàn thành C_i vượt deadline d_i . Mức độ trễ được tính là $\max(0, C_i - d_i)$ – bằng 0 nếu xong sớm hơn hoặc đúng hạn. Tổng vi phạm trên toàn episode là tổng các mức trễ, có trọng số theo lớp QoS:

$$V = \sum_{\text{task}} \lambda(q_i) \max(0, C_i - d_i), \quad (1.4)$$

Nhãn QoS	Hệ số phạt λ
LatencySensitive (tác vụ tương tác)	3,0
Burstable (tác vụ thường)	1,0
BestEffort (tác vụ nền)	0,3

Bảng 1.1: Hệ số phạt λ cho từng lớp QoS – vi phạm SLA của tác vụ tương tác bị phạt nặng gấp 3 lần tác vụ thường.

Mục tiêu là giảm V càng nhiều càng tốt.

1.2.3 Vì sao hai mục tiêu này xung đột?

Hai mục tiêu trên không thể đồng thời tối ưu vì chúng kéo scheduler theo hai hướng ngược nhau:

- **Để tiết kiệm năng lượng:** dồn tải vào ít host hơn, tắt bớt host nhàn rỗi $\Rightarrow$ host “bận” có nguy cơ xếp hàng dài, làm tăng trễ $\Rightarrow$ tăng vi phạm SLA.
- **Để đảm bảo SLA:** rải tải ra nhiều host hơn, mỗi host chỉ chạy vài tác vụ $\Rightarrow$ nhiều host vận hành ở mức tải thấp, tiêu hao P^{idle} trên cả những host gần như không làm việc $\Rightarrow$ tăng năng lượng.

Vì vậy không tồn tại một “phương án tốt nhất tuyệt đối” mà có cả một dải các phương án đánh đổi: phương án A tiết kiệm điện hơn nhưng trễ nhiều hơn, phương án B đảm bảo SLA tốt hơn nhưng tốn điện hơn.

Cách so sánh hai phương án. Quy ước đơn giản: phương án A tốt hơn phương án B nếu A không tệ hơn ở cả hai mục tiêu và tốt hơn rõ rệt ở ít nhất một. Nếu A tốt hơn ở mục tiêu

này nhưng tệ hơn ở mục tiêu kia, hai phương án không thể so sánh trực tiếp – cả hai đều là *ứng cử viên hợp lệ* mà nhà vận hành có thể chọn tùy chính sách. Tập tất cả các phương án không bị một phương án nào khác “tốt hơn” chính là **Pareto front** – đường biên mà ELDAS cố gắng xấp xỉ.

[Hình 1.3 – Đồ thị 2D minh họa trade-off Energy vs. SLA và đường Pareto front]

Hình 1.2: Trade-off giữa năng lượng và vi phạm SLA. Mỗi điểm là một chính sách lập lịch; đường cong nối các điểm không bị thống trị (Pareto front) là kết quả mà Phase 2 hướng tới xấp xỉ.

1.2.4 Độ phức tạp tính toán

Trường hợp đơn giản nhất của bài toán – gán n tác vụ vào m host với một chiều tài nguyên duy nhất – đã tương đương với bài toán *Bin Packing NP-hard* [8]. Khi mở rộng sang nhiều chiều tài nguyên, nhiều mục tiêu, và bản chất *online* (tác vụ đến theo thời gian thực), không thể trông đợi vào lời giải tối ưu chính xác trong thời gian đa thức. Đây là động lực chính để chuyển sang các phương pháp dựa trên **học tăng cường đa mục tiêu** (*Multi-Objective Reinforcement Learning* – MORL), nơi agent học được chính sách (*policy*) gần tối ưu thông qua tương tác với môi trường mô phỏng [9], [10].

1.3 Mục tiêu đề tài

1.3.1 Mục tiêu tổng thể

Đề tài **ELDAS** (*Energy- and Latency-aware Datacenter AI Scheduler*) hướng tới xây dựng một **nền tảng mô phỏng và huấn luyện thuật toán MORL** cho bài toán lập lịch tác vụ AI trong trung tâm dữ liệu, với khả năng:

- Tái tạo trung thực hành vi của một cụm máy chủ AI ở quy mô vừa;
- Nạp được dữ liệu workload thực thay vì sinh ngẫu nhiên;
- So sánh công bằng nhiều thuật toán lập lịch (heuristic và RL) trên cùng một bộ kịch bản;
- Sản sinh các chỉ số định lượng đa mục tiêu phục vụ vẽ Pareto front.

1.3.2 Mục tiêu Giai đoạn 1 (phạm vi báo cáo này)

Báo cáo này trình bày sáu sản phẩm kỹ thuật của Giai đoạn 1. Mỗi sản phẩm là một *deliverable cụ thể, có thể kiểm chứng độc lập*:

1. **MT1 – Engine mô phỏng datacenter có theo dõi GPU.** Dụng môi trường mô phỏng cụm máy chủ trên nền CloudSim Plus, bổ sung lớp theo dõi GPU (mà CloudSim Plus không hỗ trợ sẵn) và mô hình công suất tuyến tính cho cả CPU lẫn GPU.
2. **MT2 – Tích hợp workload thực.** Parser đọc trực tiếp Alibaba PAI Cluster Trace v2023, kèm bộ lọc tạo ba kịch bản tải *Low / High / Burst* để phục vụ so sánh.
3. **MT3 – Cầu nối Java–Python.** Dùng Py4J để Python điều khiển vòng đời mô phỏng (reset / step / pause / resume), đáp ứng đặc thù tương tác từng bước của RL.
4. **MT4 – Môi trường RL theo chuẩn MO-Gymnasium.** Phần thưởng giữ dạng **vector hai chiều** ($R^{\text{energy}}, R^{\text{SLA}}$) – không cộng dồn về một số; ràng buộc capacity được ép cứng qua action masking.
5. **MT5 – Hai scheduler baseline.** (i) *Random allocation* và (ii) *Kubernetes default* (Filter $\rightarrow$ LeastRequestedPriority). Hai baseline này tạo *đường tham chiếu* cho agent MORL ở Phase 2 phải vượt qua.
6. **MT6 – Đóng gói tái sản xuất.** Toàn bộ hệ thống được gói trong Docker Compose; chạy cùng seed phải cho kết quả bit-for-bit trùng nhau, kể cả trên máy khác.

1.3.3 Mục tiêu Giai đoạn 2 (kế hoạch tiếp nối)

Giai đoạn 2 dựa trên nền tảng MT1–MT6 để:

1. Huấn luyện agent PPO + MLP, biến hai mục tiêu thành một bằng cách cộng có trọng số:

$$R = w \cdot R^{\text{energy}} + (1 - w) \cdot R^{\text{SLA}};$$
2. Quét trọng số w qua các giá trị $\{0,1, 0,2, \dots, 0,9\}$ để thu được nhiều điểm trên Pareto front;
3. So sánh kết quả MORL với hai baseline trên cả ba kịch bản tải, đo trên ba chỉ số: năng lượng, makespan, tỉ lệ vi phạm SLA;
4. Tinh chỉnh siêu tham số qua Weights & Biases; thử Graph Neural Network thay MLP nếu thời gian cho phép.

1.4 Phạm vi và giới hạn

Để đảm bảo tính khả thi trong khuôn khổ một đề án chuyên ngành, đề tài giới hạn phạm vi như sau:

Về quy mô. Cụm mô phỏng cấu hình 5 host (mỗi host gồm 32 vCPU, 128 GB RAM, 4 GPU NVIDIA A100), tải đầu vào trích từ Alibaba trace với kích thước 1 000–5 000 tác vụ. Quy mô này đủ để bộc lộ hiện tượng phân mảnh tài nguyên và xung đột mục tiêu, đồng thời giữ thời gian huấn luyện một episode dưới mức chấp nhận được trên phần cứng phòng thí nghiệm.

Về môi trường. Toàn bộ thực nghiệm được thực hiện trong môi trường **mô phỏng** bằng CloudSim Plus, không triển khai trên cụm Kubernetes thực tế. Lý do: chu kỳ huấn luyện RL đòi hỏi hàng triệu bước tương tác, không khả thi trên hạ tầng thật.

Về phạm vi không bao gồm. Đề tài **không xét**: (i) di chuyển máy ảo trực tiếp (*live VM migration*); (ii) lập lịch xuyên trung tâm dữ liệu (*multi-datacenter scheduling*); (iii) mô hình mạng chi tiết (*network topology, congestion modeling*); (iv) lập lịch tác vụ phân tán có giao tiếp tập thể (*collective communication trong distributed training*).

Về tập hợp ràng buộc enforce trong Phase 1. Phát biểu bài toán ở §1.2 liệt kê năm chiều ràng buộc tài nguyên (CPU, RAM, GPU, băng thông, nhiệt độ). Phase 1 **chỉ thực thi cứng** ba chiều CPU/RAM/GPU (§4.1.4); hai chiều còn lại được *hoãn có chủ đích* sang Phase 2 với lý do cụ thể như sau:

- **Băng thông mạng.** Trace Alibaba PAI v2023 không cung cấp trường `bandwidth_demand` ở mức task; bổ sung ràng buộc băng thông tại Phase 1 buộc phải sinh giả định tổng hợp (*synthetic*, ví dụ tỉ lệ với `cpu_milli`), làm giảm tính tái sản xuất khoa học của báo cáo và tạo ra một biến số không kiểm soát trong khi mục tiêu Phase 1 là *kiểm chứng kiến trúc bốn lớp*. Hơn nữa, mỗi chiều ràng buộc thêm vào tăng kích thước observation thêm H (một feature/host); với $H = 10$ điều này làm chậm hội tụ PPO không cần thiết khi chính thiết kế MORL còn đang được hiệu chỉnh. Phase 2 sẽ kích hoạt `host.getBwProvisioner()` sẵn có của CloudSim Plus (chỉ cần ~ 20 dòng code thêm vào `VmAllocationPolicy.find()`) sau khi đã có dữ liệu bandwidth thực hoặc một mô hình synthetic được biện luận chặt chẽ.
- **Nhiệt độ host.** Mô hình nhiệt yêu cầu (a) mạng RC nhiệt ($T_j(t) = T_{\text{amb}} + \theta_{\text{JA}} \cdot P_j(t)$ ở dạng đơn giản nhất), (b) đường cong fan và ngưỡng throttle, và (c) tương tác hai chiều với DVFS – bản thân ba thành phần này là một nghiên cứu phụ. Quan trọng hơn, mô hình năng lượng Phase 1 *tuyến tính* (§2.2) chưa có DVFS, nên việc thêm thermal lúc này sẽ là một *lớp giả định trên lớp giả định* không có giá trị khoa học. Phase 2 dự kiến đưa DVFS + thermal vào cùng nhau như một gói nhất quán: trước hết thay (1.2) bằng mô hình bậc hai có DVFS, sau đó thêm $T_j(t)$ làm feature observation và một `HostFitsThermal` filter dạng K8s predicate (cấm gán task lên host có $T_j > T_{\text{max}}$).

Quyết định này tuân theo nguyên tắc *minimum viable scope*: Phase 1 ưu tiên các ràng buộc **có dữ liệu thực** (CPU/RAM/GPU đều có trong trace) để mọi con số trong báo cáo đều có thể truy xuất được nguồn; các ràng buộc **cần dữ liệu giả định** được hoãn đến khi mô hình nền (power, DVFS) đủ chính xác để không bị nhiễu giả định chồng lên giả định.

Về mô hình năng lượng. Sử dụng mô hình tuyến tính (1.2) cho cả CPU và GPU. Mô hình phi tuyến (*cubic, logarithmic*) và DVFS (*Dynamic Voltage and Frequency Scaling*) được ghi nhận như hướng mở rộng cho các giai đoạn sau.

Về thuật toán. Phase 1 chỉ hiện thực hai baseline (Random, K8s Default). Phase 2 tập trung vào PPO + MLP với linear scalarization. Các phương pháp MORL phức tạp hơn như Envelope Q-Learning, Pareto-DQN, hoặc các đầu Graph Neural Network được liệt kê trong phần *stretch goals*.

1.5 Cấu trúc báo cáo

Báo cáo gồm bảy chương được tổ chức như sau:

Chương 1 – Giới thiệu. Trình bày bối cảnh, phát biểu hình thức bài toán lập lịch đa mục tiêu, mục tiêu cụ thể, phạm vi và giới hạn của đề tài.

Chương 2 – Cơ sở lý thuyết và nghiên cứu liên quan. Tổng quan ba mảng: (i) thuật toán lập lịch truyền thống và Kubernetes scheduler; (ii) mô hình tiêu thụ năng lượng datacenter; (iii) học tăng cường đơn/đa mục tiêu với khảo sát ít nhất ba công trình tiêu biểu.

Chương 3 – Kiến trúc hệ thống. Mô tả kiến trúc bốn lớp (Docker Compose, CloudSim Plus, Py4J Gateway, MO-Gymnasium), các luồng dữ liệu chính và các quyết định thiết kế then chốt.

Chương 4 – Hiện thực. Trình bày chi tiết hiện thực Phase 1: `SimulationConfig`, `DatacenterFactory`, `AlibabaTraceReader`, `SimulationManager` (lifecycle pause/resume/reset), `Py4J GatewayEntryPoint`, hai `VmAllocationPolicy` baseline, và phía Python: `state_builder`, `reward`, `environment`, `tracker`.

Chương 5 – Kết quả Phase 1. Báo cáo kết quả *smoke test* vòng RL hoàn chỉnh, kiểm chứng tính đúng của bridge Java–Python, đo các chỉ số ban đầu (năng lượng, SLA) của hai baseline trên ba kịch bản tải.

Chương 6 – Kế hoạch Phase 2. Lộ trình huấn luyện PPO, weight sweep, hyperparameter tuning và đánh giá Pareto front; đi kèm sơ đồ Gantt và phân tích rủi ro.

Chương 7 – Kết luận. Tổng kết đóng góp của Phase 1, bài học rút ra, và hướng phát triển.

Phần *Phụ lục* cung cấp toàn bộ tệp cấu hình (`docker-compose.yml`, `.env`), các đoạn mã đại diện và hướng dẫn chạy lại thực nghiệm.

Chương 2

Cơ sở lý thuyết và nghiên cứu liên quan

Chương này trình bày nền tảng lý thuyết cho đề tài ELDAS theo bốn mảng chính: (i) các thuật toán lập lịch trong trung tâm dữ liệu; (ii) mô hình tiêu thụ năng lượng; (iii) học tăng cường đa mục tiêu (MORL); và (iv) các công trình nghiên cứu liên quan. Mục tiêu là cung cấp đủ kiến thức để người đọc theo dõi được các quyết định thiết kế ở Chương 1 cũng như phần hiện thực ở các chương sau, đồng thời định vị đóng góp của đề tài so với các nghiên cứu đã có.

2.1 Lập lịch trong trung tâm dữ liệu

2.1.1 Các thuật toán heuristic truyền thống

Lập lịch tài nguyên được nghiên cứu từ rất sớm dưới dạng bài toán *Bin Packing*: cần đặt n vật phẩm có kích thước cho trước vào số lượng tối thiểu các thùng có dung lượng cố định [8]. Các *heuristic* kinh điển vẫn được sử dụng rộng rãi gồm:

- **First-Fit (FF)**. Duyệt danh sách host theo thứ tự cố định và đặt tác vụ lên host đầu tiên đủ tài nguyên. Đơn giản, độ phức tạp $O(n \cdot m)$, nhưng dễ gây phân mảnh.
- **Best-Fit (BF)**. Đặt tác vụ lên host *vừa khít nhất* (host có tài nguyên dư còn lại nhỏ nhất sau khi đặt). Giảm phân mảnh nhưng tốn chi phí sắp xếp.
- **Worst-Fit (WF)**. Ngược lại BF – chọn host dư nhiều nhất, nhằm cân bằng tải nhưng làm tăng số host bật.
- **Round-Robin (RR)**. Phân phối tuần tự, đảm bảo công bằng nhưng bỏ qua sự khác biệt về dung lượng host.

Các heuristic trên đều **đơn mục tiêu** (single-objective): tối thiểu số host hoặc cân bằng tải, không xét đến năng lượng hay SLA. Đây là hạn chế lớn khi áp dụng cho datacenter AI hiện đại.

2.1.2 Meta-heuristic: Ant Colony Optimization (ACO)

Để vượt qua giới hạn “đơn mục tiêu” và “mắc kẹt ở cực tiểu địa phương” của các heuristic “một bước nhìn” (FF, BF), nhiều nghiên cứu chuyển sang **meta-heuristic**, trong đó *Ant Colony Optimization* (ACO) là đại diện được khảo sát rộng rãi nhất cho bài toán lập lịch tài nguyên [11].

Ý tưởng cốt lõi. ACO mô phỏng cách đàn kiến tìm đường ngắn nhất tới nguồn thức ăn qua *pheromone trail*: kiến đi qua một đoạn đường để lại pheromone, đoạn nào có nhiều pheromone hơn lại được kiến sau ưu tiên chọn, tạo vòng phản hồi dương dần hội tụ về đường tối ưu. Khi áp dụng cho scheduling, mỗi “con kiến” xây một schedule hoàn chỉnh bằng cách chọn host cho từng task theo xác suất tỉ lệ thuận với lượng pheromone tích lũy và chất lượng ước tính của cặp (*task*, *host*). Sau mỗi vòng, pheromone bay hơi một phần và được củng cố theo mức độ tốt của schedule vừa tìm được, tạo “bộ nhớ tập thể” hướng dẫn các kiến ở vòng sau. Liu và cộng sự áp dụng biến thể *Energy-aware ACO* cho VM placement và đạt giảm 9–12% năng lượng so với First-Fit Decreasing trên cùng workload trace [12].

Ưu điểm. (i) Có khả năng *thoát local optima* nhờ tính ngẫu nhiên trong lựa chọn của từng kiến; (ii) tự nhiên hỗ trợ multi-objective qua hàm chi phí tổng hợp; (iii) không cần gradient, hoạt động trên không gian rời rạc rộng.

Hạn chế đối với ELDAS. Ba nhược điểm cốt tử cho phạm vi đề án này:

1. **Chi phí đánh giá khổng lồ.** ACO là population-based: mỗi iteration cần K kiến ($K \approx 20\text{--}50$) $\times$ một episode mô phỏng đầy đủ. Với throughput đo được 65 step/s (§5.2.1) và $\sim 2\,000$ task/episode, một iteration ACO tốn $\sim 30\text{ s} \times 30$ kiến = 15 phút; 100 iteration hội tụ ~ 25 giờ cho một bộ siêu tham số.
2. **Không khái quát hoá.** Pheromone matrix τ_{ij} được học cho một workload cụ thể – gặp workload mới phải chạy lại từ đầu. Trái ngược với chính sách RL đã huấn luyện có thể tổng quát hoá qua hàm xấp xỉ.
3. **Số siêu tham số lớn** ($\alpha, \beta, \rho, Q, K$, chiến lược elitism, ...) khiến reproducibility kém và khó so sánh công bằng giữa các nghiên cứu.

So sánh với PPO (định hướng của ELDAS). PPO khắc phục cả ba nhược điểm trên: (i) on-policy nhưng cập nhật *mỗi episode* thay vì sau *một quần thể episode*, tiết kiệm $K \times$ thời gian mô phỏng; (ii) tổng quát hoá qua mạng nơ-ron – một chính sách dùng cho nhiều workload; (iii) ít siêu tham số chính (learning rate, clip ϵ , GAE λ). Vì vậy ACO chỉ được đề cập như *tham khảo so sánh* trong khảo sát, không được hiện thực trong ELDAS.

2.1.3 Kubernetes Scheduler

Kubernetes (K8s) là hệ điều phối container phổ biến nhất hiện nay [13]. Bộ lập lịch mặc định (*kube-scheduler*) hoạt động theo *pipeline* ba bước cho mỗi *pod*:

1. **Filter (Predicate).** Loại bỏ các node không đủ điều kiện. Một số bộ lọc tiêu biểu: `PodFitsResources` (đủ CPU/RAM/GPU), `NodeAffinity`, `Taints/Tolerations`.

2. **Score (Priority).** Chấm điểm từng node còn lại trên thang $[0, 10]$ bằng các hàm tính điểm. Hai hàm được dùng trong ELDAS để mô phỏng K8s:

$$\text{LRP}(h_j) = 10 \cdot \frac{(c_j^{\text{cpu}} - u_j^{\text{cpu}}) + (c_j^{\text{ram}} - u_j^{\text{ram}})}{2(c_j^{\text{cpu}} + c_j^{\text{ram}})},$$

gọi là *LeastRequestedPriority* – ưu tiên host có nhiều tài nguyên trống (cân bằng tải). Hàm thứ hai *BalancedResourceAllocation* ưu tiên host có tỉ lệ CPU/RAM sử dụng đồng đều, nhằm tránh nghẽn cổ chai một chiều tài nguyên.

3. **Bind.** Gắn pod vào node có điểm số cao nhất.

Hạn chế. K8s scheduler không có cơ chế chấm điểm năng lượng và không hỗ trợ tối ưu đa mục tiêu nguyên thủy. Đây là khoảng trống mà các phương pháp học tăng cường có cơ hội cải thiện.

2.1.4 Lập lịch dựa trên học tăng cường: Deep Q-Network (DQN)

Đột phá Atari của Mnih và cộng sự năm 2015 [14] đưa *Deep Q-Network* (DQN) trở thành thuật toán RL được khảo sát phổ biến nhất cho scheduling sau đó. DQN học hàm giá trị **Q-function** $Q(s, a)$ – ước lượng tổng reward kỳ vọng khi thực hiện hành động a từ trạng thái s . Thay vì tính trực tiếp, DQN dùng mạng nơ-ron xấp xỉ Q và cập nhật theo nguyên lý Bellman: phần thưởng hiện tại cộng với giá trị tốt nhất có thể đạt được ở bước sau. Hai kỹ thuật then chốt giúp ổn định quá trình học là **target network** (một bản sao mạng được đóng băng định kỳ, tránh “mục tiêu di động”) và **replay buffer** (lưu trữ kinh nghiệm rồi lấy mẫu ngẫu nhiên, phá vỡ tương quan thời gian giữa các bước liên tiếp).

Ưu điểm. (i) **Sample-efficient hơn on-policy:** cùng một mẫu kinh nghiệm có thể được tái sử dụng nhiều lần qua replay buffer, lý tưởng khi mỗi episode đắt như mô phỏng CloudSim; (ii) **tự nhiên cho discrete action space** – $\arg \max_a Q$ là toán tử kiểm soát mặc định, phù hợp với $H = 10$ lựa chọn host; (iii) biến thể *Rainbow DQN* (kết hợp PER, Double DQN, Dueling, Distributional) hiện đạt SOTA trên nhiều benchmark.

Hạn chế đối với ELDAS. Bốn lý do then chốt khiến DQN *không* phải lựa chọn tối ưu cho bài toán Phase 2:

1. **Reward lệch tỉ lệ gây value over-estimation.** Reward energy của ELDAS luôn âm với biên độ $\sim 10^3$ Ws, trong khi reward SLA thừa và biên độ phụ thuộc mức độ vi phạm. DQN chọn hành động bằng $\arg \max_a Q$, dễ bị *bias dương* (overestimate) khi phân phối Q có hai thành phần lệch nhau hai bậc độ lớn. PPO khắc phục qua *advantage normalisation* – chuẩn hoá A_t về mean 0, std 1 trước mỗi lần cập nhật.
2. **Action masking phức tạp hơn.** Host không đủ tài nguyên phải bị cấm tuyệt đối. Với DQN cần chặn ở hai chỗ – bước chọn hành động ϵ -greedy và bước $\max_{a'}$ trong Bellman backup – mỗi chỗ có biến thể chưa chuẩn hoá trong literature. Với PPO, chỉ cần mask một lần ở policy head; thư viện `sb3-contrib` đã có `MaskablePPO` sẵn sàng.

3. **Không hỗ trợ vector reward nguyên thủy.** Q-function trả về vô hướng cho mỗi cặp (s, a) ; muốn dùng cho MORL phải *scalarize trước khi học*, dẫn tới phải huấn luyện lại từ đầu với mỗi trọng số $\mathbf{w}$ – chi phí cao đúng bằng linear scalarization của PPO. Các biến thể chuyên dụng cho MORL (*Envelope-Q*, *Pareto-DQN*) tồn tại nhưng phức tạp hơn, không có triển khai SB3 sẵn sàng, không phù hợp với phạm vi Phase 2.
4. **Off-policy + replay buffer khó song song hoá.** Phase 2 dự kiến chạy weight sweep ($K = 9$) qua `SubprocVecEnv` để khai thác nhiều core CPU; PPO on-policy hợp tự nhiên với kiến trúc này, trong khi DQN cần kiến trúc Ape-X / R2D2 phức tạp hơn nhiều.

So sánh với PPO (định hướng của ELDAS). PPO ổn định hơn DQN trước reward lệch tỉ lệ nhờ hai cơ chế không có tương đương trong DQN: *advantage normalisation* (triệt tiêu sự chênh lệch biên độ trước mỗi lần cập nhật) và *trust region qua clipping* (giới hạn mức độ thay đổi chính sách ở mỗi bước, ngăn diverge khi SLA violation bùng phát bất ngờ). Chi tiết PPO được trình bày ở §2.3.

Bảng 2.1: So sánh các thuật toán lập lịch truyền thống.

Thuật toán	Mục tiêu	Đa mục tiêu?	Năng lượng?
First-Fit	Min #host	Không	Không
Best-Fit	Min phân mảnh	Không	Không
Round-Robin	Cân bằng tải	Không	Không
K8s LRP	Cân bằng tải	Không	Không
NSGA-II	Energy + QoS	Có	Có
DRL (DeepRM)	Slowdown	Không	Không
MORL (ELDAS)	Energy + SLA	Có	Có

2.2 Mô hình tiêu thụ năng lượng datacenter

2.2.1 Mô hình tuyến tính SPECpower

Phép đo thực nghiệm trên hơn 200 máy chủ thương mại do tổ chức SPEC công bố (SPECpower_{ssj2008}) cho thấy công suất tiêu thụ của một máy chủ phụ thuộc gần như tuyến tính vào mức sử dụng CPU [7]. Mô hình đơn giản và được dùng rộng rãi nhất là:

$$P(U) = P_{\text{idle}} + U \cdot (P_{\text{max}} - P_{\text{idle}}), \quad (2.1)$$

trong đó:

- $U \in [0, 1]$ – mức sử dụng CPU trung bình của host;
- P_{idle} – công suất ở trạng thái rảnh ($U = 0$);
- P_{max} – công suất ở trạng thái tải tối đa ($U = 1$).

Một quan sát quan trọng: P_{idle} thường chiếm 50–70% của P_{max} . Do đó, một host *rảnh* vẫn tiêu tốn tới hơn nửa công suất so với khi chạy hết tải. Đây chính là cơ sở để các thuật toán *consolidation* ưu tiên gom tác vụ vào ít host nhất rồi tắt các host còn lại.

2.2.2 Tính tổng năng lượng theo thời gian rời rạc

Mô phỏng trong ELDAS hoạt động theo **thời gian rời rạc** với bước $\Delta t = 1$ s. Tổng năng lượng tiêu thụ của một host trong một khoảng thời gian gồm K bước được tính bằng tổng đơn giản:

$$E_j = \sum_{k=1}^K P_j(U_j^{(k)}) \cdot \Delta t, \quad (2.2)$$

và năng lượng toàn cụm là $E = \sum_{j=1}^m E_j$. Cách rời rạc hóa này tránh được tích phân, đồng thời ánh xạ trực tiếp xuống *event loop* của CloudSim Plus (xem Chương 4).

2.2.3 Mở rộng cho GPU

CloudSim Plus phiên bản 8.5.7 chưa có khái niệm GPU nguyên thủy. ELDAS bổ sung lớp theo dõi GPU riêng và áp dụng cùng dạng tuyến tính với hai mức công suất $P_{\text{idle}}^{\text{gpu}}$ và $P_{\text{max}}^{\text{gpu}}$ lấy từ tài liệu kỹ thuật của NVIDIA A100 [15].

2.2.4 Power Usage Effectiveness (PUE)

Để phản ánh chi phí năng lượng *vận hành* (làm mát, chiếu sáng, mạng), ngành công nghiệp dùng chỉ số PUE:

$$\text{PUE} = \frac{E_{\text{total}}}{E_{\text{IT}}}, \quad (2.3)$$

với E_{IT} là năng lượng tiêu thụ bởi thiết bị tính toán và E_{total} là tổng năng lượng đầu vào của datacenter. PUE lý tưởng là 1,0; trung bình toàn ngành 2023 vào khoảng 1,58 [16]. ELDAS sử dụng giá trị mặc định $\text{PUE} = 1,5$ để quy đổi E_{IT} sang chi phí thực tế khi báo cáo.

2.2.5 Lý do chọn mô hình tuyến tính cho Phase 1

Có ba lý do chính:

1. **Đủ độ chính xác cho mục tiêu so sánh tương đối.** Sai số trung bình so với đo thực nghiệm chỉ khoảng 5–7% [7] – đủ để xếp hạng giữa các scheduler.
2. **Đơn giản và tái sản xuất được.** Chỉ cần hai tham số trên mỗi loại thiết bị, dễ tinh chỉnh và minh bạch.
3. **Tránh phụ thuộc DVFS.** Mô hình phi tuyến (cubic, logarithmic) và *Dynamic Voltage and Frequency Scaling* (DVFS) đòi hỏi dữ liệu tần số/điện áp theo thời gian thực – không có sẵn trong Alibaba trace. Để cho Phase 2 hoặc nghiên cứu mở rộng.

[Hình 2.1 – Đồ thị $P(U)$: tuyến tính (đường thẳng) và phi tuyến cubic (đường cong) trên cùng trục]

Hình 2.1: So sánh mô hình công suất tuyến tính và phi tuyến.

2.3 Học tăng cường đa mục tiêu (MORL)

2.3.1 Markov Decision Process (MDP)

Học tăng cường (*Reinforcement Learning* – RL) mô hình hóa bài toán ra quyết định tuần tự dưới dạng *Markov Decision Process* [17]: tại mỗi bước, **agent** quan sát **trạng thái** môi trường, chọn một **hành động**, nhận **phần thưởng** và chuyển sang trạng thái mới. Quá trình lặp đi lặp lại cho đến khi kết thúc episode. Mục tiêu là học một **chính sách** – ánh xạ từ trạng thái sang hành động – sao cho tổng phần thưởng tích lũy (có chiết khấu theo thời gian) là lớn nhất.

2.3.2 Multi-Objective MDP (MOMDP)

Khi có **nhiều mục tiêu xung đột**, phần thưởng không còn là một số đơn lẻ mà trở thành một **vector phần thưởng** [9], [18]. Trong ELDAS, mỗi bước agent nhận hai phần thưởng riêng biệt: R^{energy} (năng lượng tiêu thụ bước đó, luôn âm) và R^{SLA} (mức độ vi phạm deadline, bằng 0 nếu đúng hạn).

2.3.3 Pareto Optimality

Vì không thể *cộng* hai mục tiêu khác đơn vị một cách tự nhiên (Joule và giây trễ), khái niệm tối ưu phải mở rộng thành **Pareto optimality** (đã định nghĩa trong §1.2). Trong không gian giá trị kỳ vọng của vector phần thưởng, tập các chính sách không bị thống trị tạo thành *Pareto front* $\mathcal{P}^*$.

Tại sao cần vector reward thay vì scalar? Nếu cộng gộp ngay từ đầu $R = w_1 R_1 + w_2 R_2$ với một $\mathbf{w}$ cố định, agent chỉ học được *một điểm* trên Pareto front. Giữ nguyên dạng vector cho phép: (i) quét trọng số sau huấn luyện để khôi phục nhiều điểm; (ii) áp dụng các thuật toán MORL đặc thù như *Envelope Q-Learning* hay *Pareto-DQN* ở các nghiên cứu sau.

[Hình 2.2 – Pareto front lý thuyết: trục hoành Energy, trục tung SLA violation, đường cong nối các điểm không bị thống trị]

Hình 2.2: Minh họa Pareto front trong không gian hai mục tiêu.

2.3.4 Proximal Policy Optimization (PPO)

PPO [19] là thuật toán *policy gradient* ổn định nhất hiện nay và được chọn cho Phase 2 của ELDAS. Ý tưởng cốt lõi: tại mỗi vòng cập nhật, hạn chế độ *lệch* của chính sách mới so với chính sách cũ trong một *vùng tin cậy*. Thay vì giải bài toán ràng buộc phức tạp, PPO sử dụng cơ chế **cắt tỉ số** (clipping) đơn giản: tỉ số xác suất giữa chính sách mới và cũ bị giới hạn trong khoảng $[1 - \epsilon, 1 + \epsilon]$ (thường $\epsilon = 0,2$), ngăn mỗi bước cập nhật làm chính sách thay đổi quá lớn và gây *diverge*.

Tại sao chọn PPO? (i) ổn định nhất trong nhóm policy gradient; (ii) ít siêu tham số, dễ tinh chỉnh; (iii) hỗ trợ tốt trong `stable-baselines3`; (iv) đã được chứng minh hiệu quả trên các bài toán scheduling tương tự [10], [20].

2.3.5 Linear Scalarization và quét trọng số

Cách đơn giản nhất để tối ưu vector phần thưởng là **cộng có trọng số**:

$$R_t^{\text{scalar}}(\mathbf{w}) = w \cdot R_t^{\text{energy}} + (1 - w) \cdot R_t^{\text{SLA}}, \quad w \in [0, 1]. \quad (2.4)$$

Mỗi w tạo ra một bài toán RL đơn mục tiêu chuẩn, agent PPO sẽ hội tụ về một chính sách π_w^* tối ưu cục bộ tại điểm vận hành đó. Bằng cách **quét trọng số** $w \in \{0, 0,1, 0,2, \dots, 0,9\}$ và huấn luyện độc lập chín agent, ta thu được chín điểm xấp xỉ Pareto front. Phương pháp này có hai ưu điểm: (i) tận dụng được mọi thuật toán RL đơn mục tiêu sẵn có; (ii) dễ song song hóa.

Hạn chế. Linear scalarization không thể khám phá các phần *lõm* (concave) của Pareto front. Đây là điểm cần nhớ khi diễn giải kết quả Phase 2.

2.3.6 Action Masking

Trong môi trường ELDAS, không phải mọi hành động đều hợp lệ tại mỗi trạng thái: nếu host h_j không đủ tài nguyên cho tác vụ τ_i , hành động “đặt τ_i lên h_j ” phải bị cấm. Cách giải quyết phổ biến trong RL là **action masking** [21]: tại mỗi bước, môi trường trả kèm một danh sách host khả thi, và các host không hợp lệ bị loại khỏi phân phối xác suất *trước khi* chọn hành động – tức xác suất chọn host không đủ tài nguyên bằng đúng 0. ELDAS sử dụng *MaskablePPO* từ thư viện `sb3-contrib` hiện thực sẵn cơ chế này. Lợi ích: (i) loại bỏ hoàn toàn khả năng vi phạm ràng buộc capacity; (ii) agent học nhanh hơn vì không lãng phí mẫu vào những hành động bị phạt cứng.

2.4 Nghiên cứu liên quan

Phần này khảo sát năm công trình tiêu biểu thuộc ba hướng nghiên cứu chính liên quan trực tiếp đến ELDAS: RL cho lập lịch tài nguyên, lập lịch đa mục tiêu cho năng lượng, và các framework mô phỏng/MORL.

2.4.1 DeepRM (Mao và cộng sự, 2016)

DeepRM [10] là công trình tiên phong áp dụng *Deep Reinforcement Learning* cho bài toán cấp phát tài nguyên cluster. Đóng góp chính: (i) biểu diễn trạng thái cluster bằng *ảnh hai chiều* (thời gian $\times$ tài nguyên), tận dụng kiến trúc CNN; (ii) chứng minh agent DRL hội tụ và đánh bại heuristic Shortest-Job-First trong môi trường mô phỏng. **Hạn chế:** đơn mục tiêu (chỉ tối thiểu *average job slowdown*); không xét năng lượng; quy mô nhỏ (≤ 20 tác vụ đồng thời).

2.4.2 Decima (Mao và cộng sự, 2019)

Decima [20] mở rộng hướng đi của DeepRM bằng *Graph Neural Network* (GNN) để mô hình hóa Directed Acyclic Graph của các Spark job. Hệ thống được triển khai thực tế trên cụm 25 máy và giảm *average job completion time* 21% so với scheduler mặc định của Spark. **Liên hệ với ELDAS:** cùng triết lý “RL + simulation + deployment”, GNN được liệt kê là *stretch goal* trong Phase 2 nếu MLP đã hội tụ.

2.4.3 NSGA-II cho điều phối đám mây tiết kiệm năng lượng

Một dòng nghiên cứu khác sử dụng *Non-dominated Sorting Genetic Algorithm II* (NSGA-II) [22] để giải bài toán đa mục tiêu năng lượng + QoS. Ưu điểm: thuật toán tiến hóa tự nhiên sinh ra cả Pareto front trong một lần chạy, không cần quét trọng số. **Nhược điểm:** mỗi lần ra quyết định cần đánh giá hàng nghìn cá thể, không phù hợp với *online scheduling* thời gian thực – đây là khoảng trống mà RL có ưu thế (chính sách đã huấn luyện chỉ cần một lần forward pass).

2.4.4 Beware of Fragmentation (Weng và cộng sự, 2023)

Công trình [4] công bố Alibaba PAI Cluster Trace v2023 và đề xuất *Fragmentation Gradient Descent* – một heuristic giảm phân mảnh GPU tới 10%. **Liên hệ trực tiếp với ELDAS:** đây chính là nguồn dữ liệu workload mà ELDAS sử dụng; bài báo cũng cung cấp baseline heuristic mạnh để so sánh ở Phase 2.

2.4.5 MO-Gymnasium (Alegre và cộng sự, 2022)

MO-Gymnasium [23] là framework chuẩn cho MORL benchmark, mở rộng API `step()` của Gymnasium để trả về vector phần thưởng. Thư viện cung cấp các môi trường tham chiếu (*deep-sea-treasure*, *minecart*, *four-room*) và bộ chỉ số đánh giá Pareto front (hypervolume, sparsity). **ELDAS đăng ký môi trường custom theo đúng chuẩn này**, đảm bảo tương thích với mọi thuật toán MORL trong tương lai.

Bảng 2.2: Bảng tổng hợp so sánh các nghiên cứu liên quan.

Nghiên cứu	Năm	Thuật toán	Đa mục tiêu?	Năng lượng?	Simulator	GPU?
DeepRM	2016	DQN+CNN	Không	Không	Custom	Không
Decima	2019	GNN+RL	Không	Không	Spark	Không
NSGA-II/EE	2020	GA	Có	Có	CloudSim	Không
FGD/Alibaba	2023	Heuristic	Có	Không	Trace	Có
MO-Gym	2022	Framework	Có	—	Gym	—
ELDAS	2026	MORL+PPO	Có	Có	CloudSim Plus	Có

Định vị đóng góp. Bảng 2.2 cho thấy ELDAS lấp khoảng trống giao của bốn yêu cầu: *đa mục tiêu, tích hợp năng lượng, theo dõi GPU*, và *dùng workload trace thực* trong cùng một nền tảng MORL. Theo hiểu biết của nhóm tại thời điểm viết báo cáo (2026), chưa có công trình mở mã nguồn nào kết hợp đồng thời cả bốn đặc tính trên.

2.5 Công nghệ nền tảng

2.5.1 CloudSim Plus

CloudSim Plus [24] là framework mô phỏng rời rạc sự kiện (*discrete-event simulation*) cho điện toán đám mây, viết bằng Java 21. So với CloudSim gốc (Đại học Melbourne, 2009), CloudSim Plus tái cấu trúc theo các nguyên tắc kỹ thuật phần mềm hiện đại: API trôi chảy (*fluent API*), tách biệt rõ Host/Vm/Cloudlet, và tích hợp sẵn `PowerModel`.

Lý do chọn: (i) mã nguồn mở, cộng đồng tích cực; (ii) hỗ trợ *event-driven* cho phép tạm dừng/tiếp tục mô phỏng – yêu cầu bắt buộc của vòng lặp RL (xem §4.5); (iii) mô hình năng lượng đã có sẵn cho CPU. **Hạn chế:** không hỗ trợ GPU – ELDAS bổ sung lớp tracking riêng.

2.5.2 Py4J – cầu nối Java–Python

Py4J [25] là thư viện cho phép code Python gọi trực tiếp các đối tượng Java trong cùng JVM thông qua *GatewayServer*. Cơ chế:

- Phía Java khởi tạo `GatewayServer` lắng nghe trên TCP port 25333;
- Phía Python tạo `JavaGateway()` để truy cập đối tượng `entry_point`;
- Mỗi lời gọi method được serialize qua TCP socket, kết quả trả về đồng bộ.

Bảng 2.3: So sánh các tùy chọn cầu nối Java–Python cho ELDAS.

Tiêu chí	Py4J	gRPC	REST
Độ trễ một call	~ 0,1 ms	~ 1 ms	~ 5 ms
Cần định nghĩa schema?	Không	Có (<code>.proto</code>)	Có (OpenAPI)
Truyền đối tượng phức tạp	Trực tiếp	Phải serialize	JSON
Phù hợp tần suất cao?	Tốt	Khá	Kém

Vì vòng lặp RL có thể gọi `step()` hàng nghìn lần mỗi episode, Py4J là lựa chọn cân bằng tốt nhất giữa hiệu năng và độ phức tạp triển khai.

2.5.3 MO-Gymnasium

Là một *fork* của Gymnasium được Alegre và cộng sự duy trì [23]. Khác biệt API quan trọng nhất: hàm `step(action)` trả về `(obs, reward_vector, terminated, truncated, info)` với `reward_vector` $\in \mathbb{R}^d$ thay vì một số thực. ELDAS đăng ký môi trường ELDAS-v0 kế thừa lớp `M0Env`, đảm bảo có thể *plug-and-play* với mọi thuật toán MORL hỗ trợ chuẩn này.

2.5.4 Docker Compose

Toàn bộ hệ thống được đóng gói bằng Docker Compose v2 với hai service (`cloudsim-java`, `rl-agent`) và ba volume chia sẻ (`trace-data`, `metrics`, `checkpoints`). Mục tiêu là **tái sản xuất được** (*reproducibility*): bất kỳ ai có Docker đều có thể chạy lại thực nghiệm với một câu lệnh duy nhất:

```
1 docker compose up --build
```

Listing 2.1: Khởi chạy hệ thống.

2.5.5 So sánh với KWOK

KWOK (*Kubernetes WithOut Kubelet*) là một lựa chọn thay thế phổ biến cho việc mô phỏng cluster K8s ở quy mô lớn. Bảng 2.4 tóm tắt vì sao ELDAS chọn CloudSim Plus thay vì KWOK ở Phase 1.

Bảng 2.4: So sánh KWOK và CloudSim Plus theo các tiêu chí của ELDAS.

Tiêu chí	KWOK	CloudSim Plus
Mô hình thời gian	Real-time	Discrete-event
Mô hình năng lượng	Không có	Có sẵn (PowerModel)
Pause/Resume mô phỏng	Khó	Hỗ trợ nguyên thủy
Quy mô tối đa	Hàng nghìn node	Hàng trăm host
Phù hợp huấn luyện RL?	Khó (không pause)	Tốt
Hỗ trợ GPU	Plugin	Phải bổ sung

Ưu thế quyết định của CloudSim Plus là *discrete-event* – cho phép mô phỏng “tua nhanh” qua các khoảng không có sự kiện, giúp một episode RL hoàn thành trong vài giây thay vì vài giờ thời gian thực.

2.6 Kết luận chương

Chương này đã cung cấp nền tảng lý thuyết cho ba quyết định thiết kế cốt lõi của ELDAS: (i) chọn **mô hình năng lượng tuyến tính** vì cân bằng độ chính xác và tính tái sản xuất; (ii) chọn **PPO + linear scalarization** làm thuật toán nền cho Phase 2 vì ổn định và dễ song song hóa; (iii) chọn ngăn xếp công nghệ **CloudSim Plus + Py4J + MO-Gymnasium** vì khả năng pause/resume mô phỏng và tương thích chuẩn MORL. Bảng tổng hợp ở §2.4 cũng đã chỉ rõ vị trí và đóng góp của ELDAS so với các công trình liên quan. Chương kế tiếp sẽ trình bày kiến trúc bốn lớp hiện thực hóa các quyết định này.

Chương 3

Thiết kế kiến trúc hệ thống

Chương này mô tả cách ELDAS được thiết kế để giải bài toán đặt ra ở Chương 1. Hệ thống chia thành **bốn lớp** – mỗi lớp đảm nhiệm một vai trò riêng và giao tiếp với nhau qua giao diện rõ ràng. Phần đầu chương giới thiệu tổng quan kiến trúc và lý do chọn từng công nghệ; các phần sau lần lượt mô tả chi tiết bốn lớp: mô phỏng (CloudSim Plus), cầu nối ngôn ngữ (Py4J), tác tử học (MO-Gymnasium), và đóng gói hạ tầng (Docker Compose).

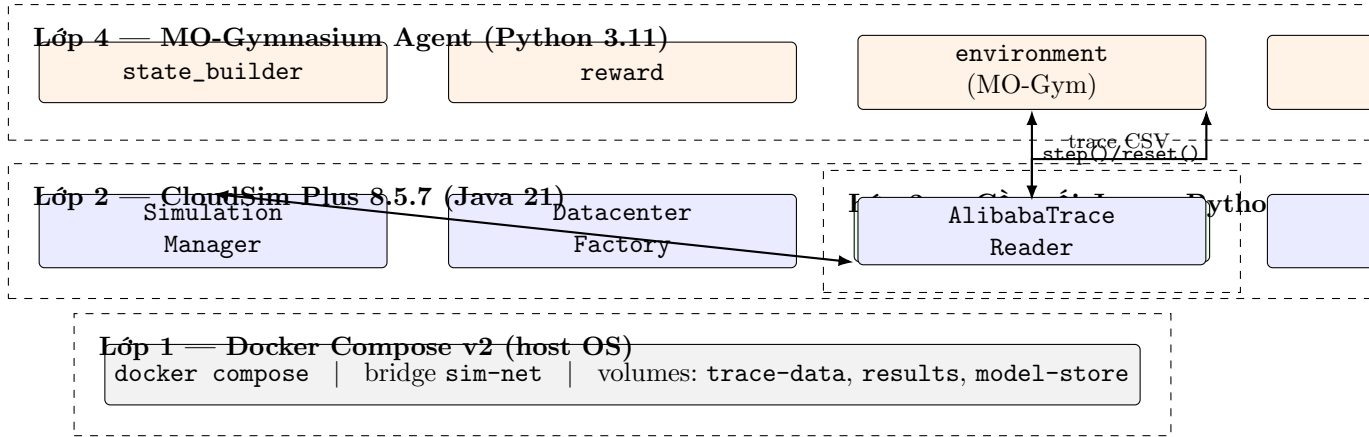
3.1 Tổng quan kiến trúc bốn lớp

3.1.1 Vì sao chia thành bốn lớp?

ELDAS phải kết hợp ba phần rất khác nhau: **mô phỏng cluster** (CPU, RAM, GPU, năng lượng), **thuật toán học** (PPO, mạng nơ-ron), và **đóng gói triển khai** (Docker, volume, biến môi trường). Nếu gộp hết vào một chương trình duy nhất, ba khó khăn lập tức xuất hiện:

- Engine mô phỏng tốt nhất cho năng lượng datacenter (CloudSim Plus) chỉ viết bằng Java, trong khi mọi thư viện RL hiện đại (PyTorch, Stable-Baselines3, MO-Gymnasium) đều viết bằng Python – hai ngôn ngữ không chạy chung process.
- Mỗi lần đổi siêu tham số huấn luyện không nên phải khởi động lại cả engine mô phỏng – rất tốn thời gian.
- Người khác phải dựng lại được toàn bộ hệ thống bằng một câu lệnh, không phải cài tay từng phần.

Cách giải quyết là chia hệ thống thành bốn lớp (Hình 3.1). Java và Python chạy trong hai container Docker riêng, nói chuyện với nhau qua một cầu nối trong cùng mạng nội bộ.



Hình 3.1: Kiến trúc bốn lớp của ELDAS. Mỗi đường đứt nét là một biên cô lập về ngôn ngữ, container, hoặc hạ tầng. Mũi tên đậm đánh dấu kênh giao tiếp hai chiều của vòng lặp RL.

3.1.2 Vai trò từng lớp

Lớp 1 – Docker Compose. Đảm bảo toàn bộ hệ thống chạy giống nhau trên mọi máy. Hai container (cloudsim-java và rl-agent) cùng dùng chung mạng nội bộ và ba volume chứa dữ liệu trace, kết quả, và checkpoint.

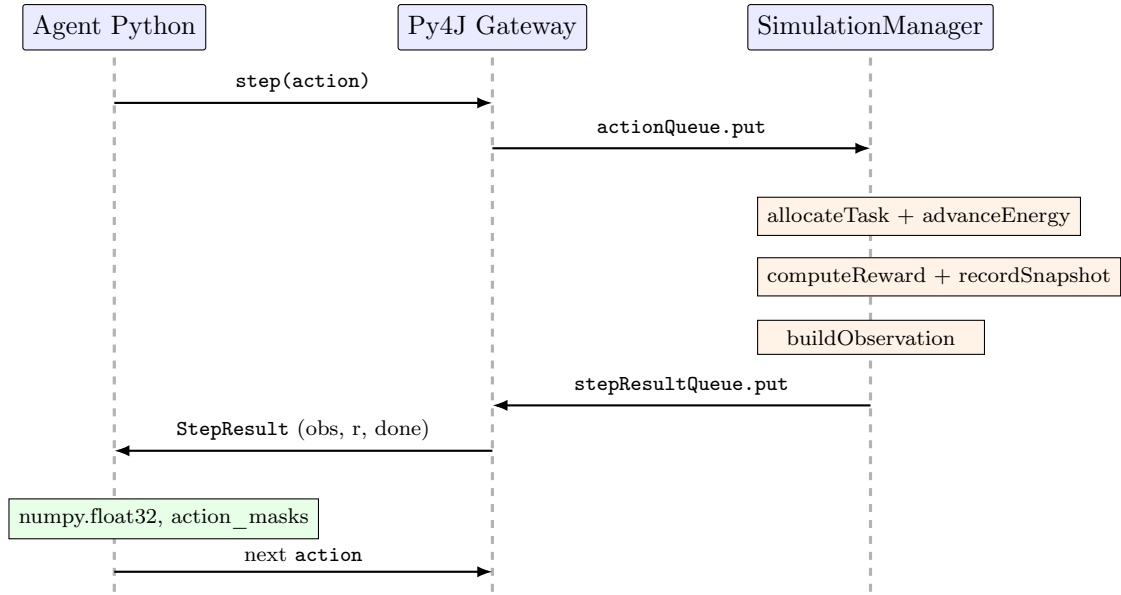
Lớp 2 – CloudSim Plus. Đóng vai trò “môi trường” theo nghĩa RL: chứa cluster ảo (host, CPU, RAM, GPU), đọc trace Alibaba, nhận hành động từ agent, và trả về trạng thái mới cùng phần thưởng. Toàn bộ mô hình năng lượng và GPU đều nằm ở đây.

Lớp 3 – Py4J Gateway. Là cây cầu nối giữa Java và Python. Chỉ phơi ra một vài phương thức cơ bản (reset, step, getActionMask, exportMetrics). Python không cần biết gì về CloudSim, nên sau này có thể thay engine khác (KWOK hoặc Kubernetes thật) mà không phải sửa code Python.

Lớp 4 – MO-Gymnasium Agent. Đóng vai trò “tác tử” RL: nhận quan sát từ Java, dùng MaskablePPO chọn hành động, ghi log lên Weights & Biases. Toàn bộ hệ sinh thái Python (PyTorch, Stable-Baselines3, MO-Gymnasium) sống ở đây.

3.1.3 Một bước trong vòng lặp RL diễn ra thế nào?

Hình 3.2 minh họa quá trình một lần `step()`: Python gửi hành động sang Java, Java cập nhật mô phỏng và trả về quan sát mới cùng phần thưởng. Toàn bộ vòng tròn (*round-trip*) chỉ tốn khoảng 0,1–0,3 ms khi hai container chạy trên cùng máy.



Hình 3.2: Sequence diagram cho một vòng lặp RL. Hai **SynchronousQueue** ở phía Java đảm bảo simulation thread chỉ tiến bước khi nhận được hành động hợp lệ.

3.2 Lựa chọn giải pháp và biện luận

Bốn lựa chọn thiết kế lớn nhất của ELDAS được trình bày theo cùng một khuôn: nêu các phương án, so sánh ưu nhược điểm trên những tiêu chí thực tế, và chốt phương án kèm phần đánh đổi phải chấp nhận. Bảng 3.1 ở cuối phần tổng hợp lại bốn quyết định.

3.2.1 Lựa chọn công cụ mô phỏng: KWOK vs. CloudSim Plus

Hai lựa chọn để mô phỏng cluster có thể tạm dùng cho RL là CloudSim Plus và KWOK (*Kubernetes Without Kubelet*). Ba điểm quyết định:

1. **Tốc độ.** CloudSim Plus dùng *discrete-event simulation* – có thể tua nhanh qua các khoảng không có sự kiện, một episode 5000 tác vụ chạy xong trong vài giây. KWOK chạy ở thời gian thực, cùng episode đó sẽ tốn nhiều phút – chậm hơn hàng trăm lần, không phù hợp để huấn luyện hàng triệu bước.
2. **Mô hình năng lượng.** CloudSim Plus có sẵn mô hình công suất CPU tuyến tính theo SPECpower; KWOK không có khái niệm năng lượng nên phải tự cấy plugin.
3. **Tạm dừng/tiếp tục.** Vòng lặp RL phải dừng được tại mỗi sự kiện “tác vụ mới đến” để chờ agent quyết định. CloudSim Plus hỗ trợ sẵn cơ chế pause/resume; KWOK thì không.

Quyết định: chọn **CloudSim Plus 8.5.7**. **Đánh đổi:** mô hình mạng đơn giản, không có `kubect1` thật, nên về sau muốn kiểm chứng trên cluster thật vẫn cần một bước port chính sách. Tuy nhiên do Lớp 3 đã được giữ rất gọn, chỉ cần thay phần backend Java là đủ – code Python không phải sửa.

3.2.2 Lựa chọn cầu nối ngôn ngữ: Py4J vs. gRPC vs. REST

Bảng 2.3 ở §2.5 đã so sánh chi tiết. Ba lý do chính khiến Py4J phù hợp nhất cho ELDAS:

- **Gọn nhẹ.** Mỗi bước RL chỉ trao đổi vài chục số thực – nhỏ hơn nhiều so với phần header và phần JSON parsing mà REST hay gRPC cộng thêm.
- **Tần suất cao.** Phase 2 sẽ gọi `step()` hàng triệu lần. Với REST (~ 5 ms mỗi lần), riêng giao tiếp đã tốn vài giờ; với Py4J ($\sim 0,1$ ms) chỉ còn chục phút.
- **Truy cập trực tiếp đối tượng Java.** Py4J cho phép Python gọi thẳng phương thức của đối tượng Java mà không phải định nghĩa `.proto` hay OpenAPI schema – giảm đáng kể boilerplate.

Quyết định: chọn **Py4J 0.10.9.7**. **Đánh đổi:** hai container phải nằm trong cùng mạng Docker, không thể chạy trên hai máy khác nhau – ELDAS chấp nhận giới hạn này vì phạm vi đồ án chỉ chạy trong phòng thí nghiệm.

3.2.3 Lựa chọn thuật toán RL: PPO vs. DQN vs. A2C

Mục 2.3 đã giới thiệu PPO; ở đây so sánh ba ứng cử viên trên ba tính chất bắt buộc của bài toán:

1. **Hỗ trợ action masking.** Tại mỗi bước, không phải host nào cũng đủ tài nguyên cho task hiện tại, nên agent phải bị cấm cứng khỏi các host không khả thi. PPO có biến thể `MaskablePPO (sb3-contrib)` hỗ trợ sẵn; DQN phải tự sửa cả bước chọn action lẫn bước Bellman; A2C không có triển khai mask sẵn dùng.
2. **Ổn định khi reward thưa và lệch tỉ lệ.** Reward năng lượng luôn âm cỡ vài trăm, còn reward SLA gần như 0 ở hầu hết các bước. DQN dễ bị overestimate giá trị Q khi reward lệch tỉ lệ; PPO có hai cơ chế bù lại là chuẩn hoá advantage và cắt tỉ số chính sách.
3. **Dễ song song hoá.** Phase 2 sẽ huấn luyện nhiều agent cùng lúc (mỗi agent một trọng số khác nhau). PPO on-policy chạy nhiều môi trường song song rất tự nhiên; DQN với replay buffer cần kiến trúc phức tạp hơn nhiều.

Quyết định: **PPO + MLP**, dùng `MaskablePPO` từ `sb3-contrib`. **Mục tiêu mở rộng:** có thể thay phần MLP bằng GNN ở giai đoạn sau – chỉ đổi backbone, không cần đụng tới bốn lớp kiến trúc.

3.2.4 Lựa chọn chiến lược MORL: Linear Scalarization vs. NSGA-II

Linear scalarization biến bài toán đa mục tiêu thành nhiều bài toán đơn mục tiêu: với mỗi trọng số w , agent học một chính sách tối ưu ở điểm cân bằng đó, quét nhiều trọng số rồi nối các điểm thu được lại sẽ ra Pareto front. **NSGA-II** [22] là thuật toán tiến hoá sinh ra cả Pareto front trong một lần chạy. Tuy nhiên với ELDAS:

- NSGA-II phải đánh giá hàng nghìn cá thể mỗi thế hệ – mỗi cá thể là một episode mô phỏng hoàn chỉnh. Tổng chi phí vượt xa ngân sách thời gian của Phase 2.

- Linear scalarization chỉ cần huấn luyện 9 chính sách độc lập (với 9 giá trị w), dễ chạy song song trên nhiều CPU. Khi triển khai thật, mỗi quyết định chỉ tốn một lượt forward pass của mạng – đủ nhanh cho online scheduling.
- Linear scalarization không khám phá được phần lõm của Pareto front – hạn chế này đã nêu rõ ở §2.3, và chấp nhận được trong phạm vi đề án.

Quyết định: Linear scalarization với weight sweep $w \in \{0,1, 0,2, \dots, 0,9\}$. NSGA-II và các thuật toán MORL chuyên dụng (Envelope-Q, Pareto-DQN) được liệt kê trong phần hướng phát triển tương lai.

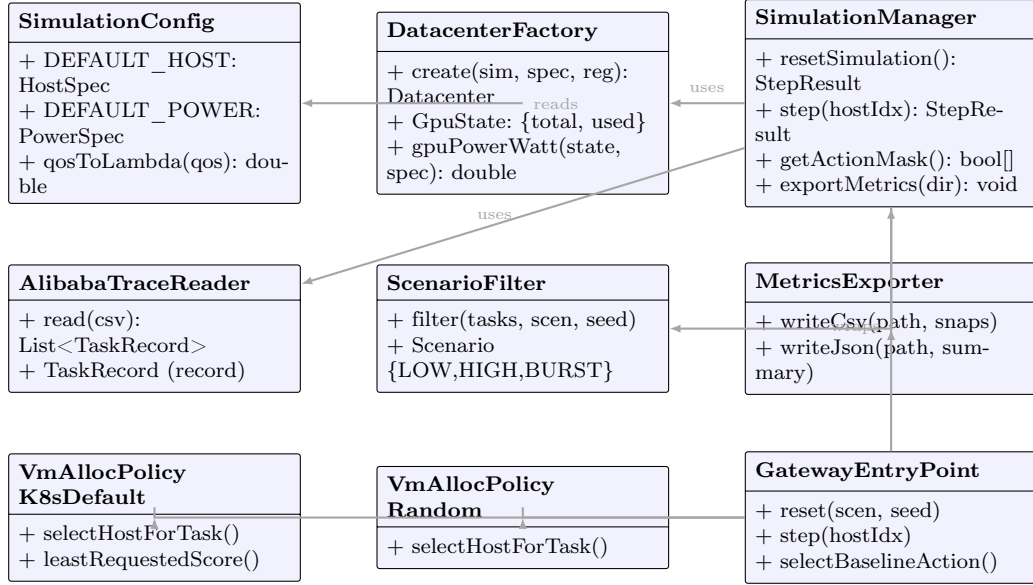
3.2.5 Bảng tổng hợp ma trận quyết định

Bảng 3.1: Ma trận quyết định bốn lựa chọn kiến trúc của ELDAS. Ký hiệu: ✓ = đáp ứng tốt, ~ = chấp nhận được, × = không đáp ứng.

Quyết định	Phương án	Latency	Năng lượng	Pause/Resume	Đa mục tiêu	Reproducible
Engine	CloudSim Plus	✓	✓	✓	✓	✓
	KWOK	~	×	×	✓	✓
Bridge	Py4J	✓	—	—	—	✓
	gRPC	~	—	—	—	✓
	REST	×	—	—	—	✓
RL Algo	PPO + MaskablePPO	—	—	—	✓	✓
	DQN	—	—	—	~	✓
	A2C	—	—	—	×	✓
MORL	Linear scalar.	—	—	—	~	✓
	NSGA-II	×	—	—	✓	✓

3.3 Thiết kế Lớp 2 — Phân hệ CloudSim Plus

Lớp 2 là phần lớn nhất của hệ thống: khoảng 1 800 dòng Java chia thành chín lớp (Hình 3.3), mỗi lớp đảm nhiệm đúng một việc để dễ bảo trì và mở rộng.



Hình 3.3: Class diagram phân hệ Java. Mũi tên đậm biểu diễn quan hệ *depends-on*; GatewayEntryPoint là điểm vào duy nhất của Python.

3.3.1 SimulationConfig – cấu hình tập trung

Mọi tham số của cluster được gom vào một file Java duy nhất, giúp việc tinh chỉnh và kiểm tra dễ dàng. Cấu hình mặc định mô phỏng một cụm gồm 10 host, mỗi host có 64 vCPU, 256 GB RAM, và 8 GPU loại A100/V100. Công suất CPU lấy 400 W ở tải tối đa và 120 W khi rảnh; GPU tương ứng 300 W và 30 W (lấy từ tài liệu kỹ thuật nhà sản xuất). Bảng QoS → hệ số λ trong reward SLA cũng được khai báo ở đây, ánh xạ bốn lớp QoS của Alibaba (LS, Guaranteed, Burstable, BE) sang các hệ số phạt 3,0 – 2,0 – 1,0 – 0,5 tương ứng.

3.3.2 DatacenterFactory và lớp theo dõi GPU

CloudSim Plus 8.5.7 chưa có khái niệm GPU. Để khởi phải sửa thư viện gốc (sẽ phá vỡ tính tái sản xuất), ELDAS giữ một bảng riêng ánh xạ từng host sang trạng thái GPU của nó (tổng số GPU và số GPU đang dùng). Công suất GPU của host được tính theo cùng mô hình tuyến tính như CPU: số GPU đang dùng càng lớn thì công suất càng tiến gần mức tối đa, các GPU rảnh đóng góp công suất idle. Phép tính này được gọi mỗi bước mô phỏng và đưa vào tổng năng lượng cluster.

3.3.3 AlibabaTraceReader – đọc trace

File trace `openb_pod_list_default.csv` có 11 trường, được ánh xạ sang một record Java đơn giản (Bảng 3.2). Hai trường dẫn xuất được sinh ngay khi parse: **deadline** bằng thời điểm K8s bind cộng với 110% thời gian chạy thực tế (cho phép trễ 10% trước khi tính vi phạm); và **slaLambda** là hệ số phạt theo QoS, lấy từ bảng đã khai báo ở §3.3.1. Các bản ghi có `podPhase` = `Pending` hoặc thiếu trường bắt buộc sẽ bị bỏ qua kèm cảnh báo, không hủy toàn bộ quá trình import.

Bảng 3.2: Ánh xạ trường Alibaba trace $\rightarrow$ đối tượng TaskRecord.

Trường CSV	Trường TaskRecord	Ngữ nghĩa
name	name	Mã định danh tác vụ
cpu_milli	pesNeeded()	$\lceil x/1000 \rceil$ PE
memory_mib	memoryMib	RAM yêu cầu (MiB)
num_gpu	numGpu	Số GPU nguyên
gpu_milli	gpuMilli	Tỉ phần GPU (1000 = full card)
qos	slaLambda	$\rightarrow \lambda \in \{0,5, 1, 2, 3\}$
creation_time	creationTime	Thời điểm xuất hiện (giây)
deletion_time	deletionTime	Thời điểm kết thúc thực tế
scheduled_time	scheduledTime	Thời điểm K8s bind ban đầu
(dẫn xuất) deadline		$= \text{scheduledTime} + 1,1 \Delta_{\text{run}}$

3.3.4 ScenarioFilter – ba kịch bản tải

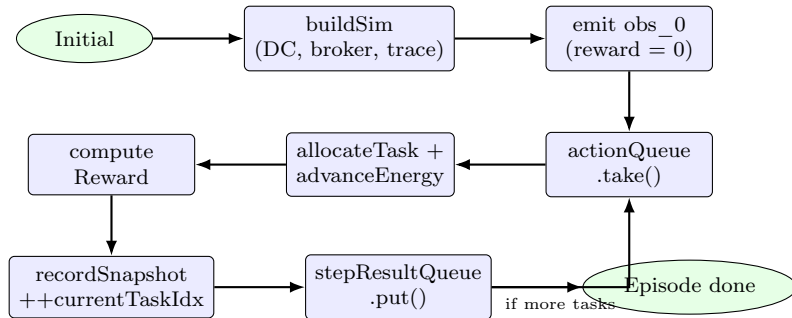
Để bộc lộ ba chế độ vận hành khác biệt, ScenarioFilter cắt trace đầy đủ thành ba tập:

- Low: 25% tác vụ đầu tiên theo creationTime – kiểm tra hành vi scheduler khi cluster nhàn rỗi (vùng năng lượng kém hiệu quả nhất theo SPECpower).
- HIGH: toàn bộ tác vụ podPhase $\neq$ Pending – kịch bản tham chiếu, có khả năng làm bão hoà cluster.
- BURST: chỉ giữ tác vụ thuộc các cửa sổ thời gian 1 giờ có cường độ đến vượt percentile 80 – kiểm tra hành vi khi cluster bị *shock*.

Tất cả ba kịch bản đều dùng cùng seed cho Random, đảm bảo tính lặp lại giữa các lần chạy.

3.3.5 SimulationManager – ghép mô phỏng vào vòng lặp RL

Đây là lớp phức tạp nhất ở Lớp 2: nó điều phối nhịp giữa mô phỏng và nhịp ra quyết định của agent. Hình 3.4 mô tả luồng trạng thái trong một episode. Hai luồng (thread mô phỏng phía Java và thread nhận lệnh từ Python) trao đổi với nhau qua hai hàng đợi đồng bộ: một hàng đợi để Python gửi hành động sang, một hàng đợi để Java trả về quan sát và phần thưởng. Hai hàng đợi này được thiết kế sao cho mỗi lần step() là một giao dịch hoàn chỉnh – không cần khoá tường minh mà vẫn loại bỏ được điều kiện đua giữa hai thread.



Hình 3.4: Vòng đời SimulationManager trong một episode. Hai vùng “actionQueue.take” và “stepResultQueue.put” là hai điểm đồng bộ giữa thread mô phỏng và thread Py4J.

Một lưu ý thiết kế quan trọng. Trong Phase 1, ELDAS **không khởi động đồng hồ CloudSim**. Thay vào đó, `SimulationManager` tự theo dõi tài nguyên đã cấp (CPU và RAM) bằng hai bảng băm riêng. Lý do: nếu bật đồng hồ mô phỏng khi chưa đăng ký đầy đủ các listener, JVM có thể crash âm thầm do sự kiện không có handler. Hệ quả thực tế là Phase 1 không đo được thời gian hoàn thành theo đồng hồ mô phỏng (mọi deadline được suy từ trace), điều này được nêu rõ ở §5.5 và sẽ được khắc phục đầu Phase 2.

Reward được tính ngay tại Java, sát với mô hình đã phát biểu ở Chương 1: phần năng lượng tính từ công suất CPU và GPU hiện tại của host nhân với độ dài một bước scheduling (lấy dấu âm để biến thành reward); phần SLA tính từ độ trễ ước lượng vượt deadline, nhân với hệ số λ tùy theo QoS. Hai giá trị này được trả về dưới dạng mảng hai phần tử để giữ nguyên tính chất *vector reward* của MOMDP.

3.4 Thiết kế Lớp 3 — Py4J Gateway

3.4.1 API surface tối giản

Lớp Gateway *cố ý* chỉ phơi ra một bề mặt nhỏ: 9 phương thức công khai, không trả về đối tượng CloudSim nào trực tiếp. Bảng 3.3 liệt kê toàn bộ contract.

Bảng 3.3: API công khai của `GatewayEntryPoint`. Mọi tham số/giá trị trả về đều là kiểu nguyên thủy, mảng nguyên thủy, hoặc Java `record` đơn giản – thân thiện với Py4J và serializable cơ lập.

Phương thức	Trả về	Mô tả
<code>reset(scenario, seed)</code>	<code>StepResult</code>	Khởi tạo lại simulation, trả về quan sát ban đầu.
<code>step(hostIndex)</code>	<code>StepResult</code>	Áp hành động, trả về $(s_{t+1}, \mathbf{r}_t, d_t)$.
<code>getActionMask()</code>	<code>boolean[]</code>	Mask hợp lệ cho bước hiện tại.
<code>getActionSize()</code>	<code>int</code>	$ \mathcal{A} $ = số host.
<code>getObservationSize()</code>	<code>int</code>	$3H + 4$.
<code>getTotalEnergyKwh()</code>	<code>double</code>	Năng lượng tích lũy (kWh) – dùng cho early-stop.
<code>selectBaselineAction(p)</code>	<code>int</code>	Chỉ số host do baseline ("k8s"/"random") chọn.
<code>exportMetrics(dir)</code>	<code>void</code>	Ghi <code>metrics.csv</code> + <code>summary.json</code> .
<code>shutdown()</code>	<code>void</code>	Dừng simulation, không tắt <code>GatewayServer</code> .

Bất biến hợp đồng: mọi state đều thuộc về `SimulationManager` phía Java; phía Python *không cache* trạng thái. Mỗi `StepResult` là một `record` bất biến; sau khi nhận, Python chỉ thực hiện một lần chuyển đổi sang `numpy.ndarray` (clipping vào $[0, 1]$ để chống nhiễu số học) rồi đẩy ngay cho agent. Điều này loại bỏ hoàn toàn các bug đồng bộ trạng thái Java/Python.

3.4.2 Khởi động Gateway và kết nối client

P phía Java, `GatewayServer` được bind vào địa chỉ 0.0.0.0 (không phải localhost) để container Python trong cùng mạng Docker có thể kết nối được. Cổng mặc định lấy từ biến môi trường `PY4J_PORT` (giá trị 25333).

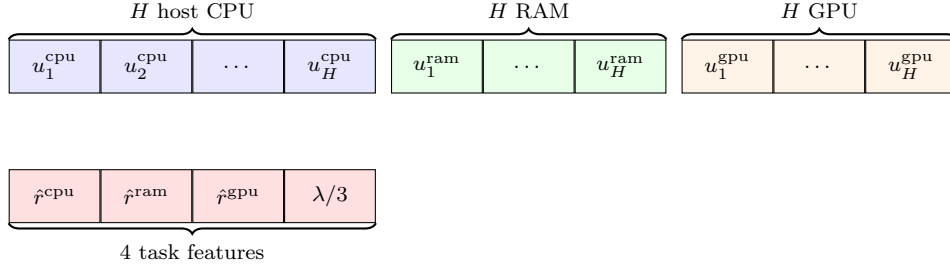
P phía Python, môi trường mở kết nối tới gateway ngay khi khởi tạo và có cơ chế **retry với chờ đợi tăng dần** để xử lý khoảng vài giây giữa lúc Docker đánh dấu Java healthy và lúc port thực sự sẵn sàng. Nếu sau 10 lần thử vẫn không kết nối được thì báo lỗi rõ ràng cho người dùng.

Tham số `auto_convert=True` của Py4J giúp tự ép kiểu giữa Python và Java: `int` truyền thẳng, mảng Java `double[]` trả về thành `list` Python – giảm đáng kể boilerplate ở phía agent.

3.5 Thiết kế Lớp 4 — MO-Gymnasium Environment

3.5.1 Observation space

Quan sát được biểu diễn bằng vector phẳng dài $3H + 4$, đã được Java chuẩn hoá về $[0, 1]$ và Python clip lại để chống lỗi số học (Hình 3.5).



Hình 3.5: Cấu trúc vector quan sát $(3H + 4)$ chiều. Ba khối host được sắp liên tiếp theo loại tài nguyên; bốn ô cuối mô tả tác vụ kế tiếp cần được xếp lịch.

Việc *nhóm theo loại tài nguyên* (toàn bộ CPU, rồi toàn bộ RAM, rồi toàn bộ GPU) thay vì *nhóm theo host* (CPU–RAM–GPU của host 1, rồi host 2, ...) là một quyết định có chủ đích: lớp ẩn của MLP học các tương tác *trong cùng loại tài nguyên* dễ hơn (ví dụ host nào có CPU thấp nhất). Khi chuyển sang GNN ở stretch goal, cấu trúc này được **reshape** về ma trận $H \times 3$ một cách tự nhiên.

3.5.2 Action space và action masking

Không gian hành động đơn giản: là chỉ số host (từ 0 đến $H - 1$) mà agent muốn đặt tác vụ hiện tại lên. Mỗi bước, môi trường gọi sang Java để lấy về một mask boolean dài H : phần tử thứ j bằng **true** nếu host j còn đủ CPU, RAM và GPU cho tác vụ, ngược lại bằng **false**. MaskablePPO áp mask này lên đầu ra của policy, đảm bảo agent không bao giờ chọn host không đủ tài nguyên. Trường hợp đặc biệt khi không host nào chứa nổi tác vụ (toàn bộ mask đều **false**), phía Python bật lại toàn bộ mask để tránh agent bị “khóa cứng”.

3.5.3 Vector reward và linear scalarization

Mỗi bước, môi trường trả về reward dưới dạng vector hai phần tử: phần năng lượng (luôn âm, bằng phần năng lượng tiêu thụ thêm trong bước đó đổi dấu) và phần SLA (âm nếu vi phạm deadline, 0 nếu đúng hạn). Để chạy được với PPO chuẩn, ELDAS cung cấp một wrapper (**ScalarRewardWrapper**) cộng hai thành phần lại theo trọng số (w_e, w_s) với $w_e + w_s = 1$. Trọng số mặc định là (0,8; 0,2); Phase 2 sẽ quét $w_e \in \{0,1, \dots, 0,9\}$ để tạo Pareto front. Khi cần benchmark thuật toán MORL chuyên dụng, bỏ wrapper là môi trường lại trả thẳng vector reward về cho thuật toán – không phải sửa code Java.

3.5.4 Điều kiện kết thúc episode

`terminated=True` khi và chỉ khi mọi tác vụ trong tập `tasks` đã được phân bổ (`currentTaskIdx == tasks.size()`). `truncated` luôn bằng `False` ở Phase 1 do bài toán có chân trời hữu hạn tự nhiên (số tác vụ cố định sau khi `ScenarioFilter` cắt). Ở Phase 2, ELDAS có thể bật `truncated=True` khi vượt `max_steps` để giới hạn chi phí của các chính sách tối.

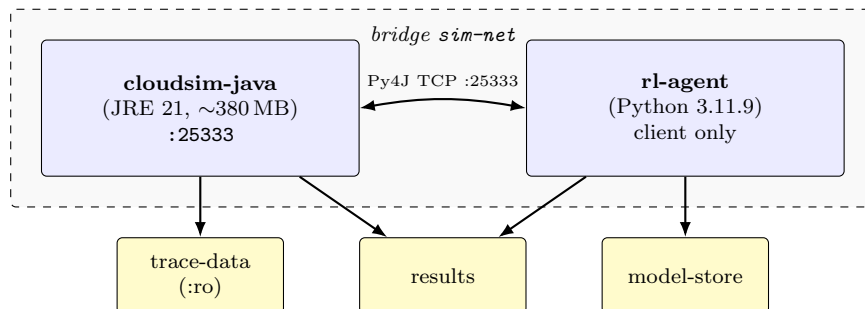
3.5.5 Tóm tắt vòng lặp

Một episode hoàn chỉnh theo đúng chuẩn Gymnasium gồm bốn bước lặp đi lặp lại: (1) gọi `action_masks()` để biết các host khả thi, (2) policy chọn host hợp lệ, (3) gọi `step(action)` để nhận quan sát và reward mới, (4) kiểm tra điều kiện kết thúc. Sau khi episode kết thúc, gọi `export_metrics()` để Java ghi `metrics.csv` và `summary.json` ra volume kết quả.

3.6 Thiết kế Lớp 1 — Docker Compose Infrastructure

3.6.1 Bố trí container và volume

Hai container được nối qua mạng nội bộ `sim-net` và dùng chung ba volume (Hình 3.6). Volume chứa trace được mount **chỉ đọc** cho `cloudsim-java` – Java không có quyền ghi đè dữ liệu thực nghiệm, tránh tình huống vô tình hỏng trace gốc trong quá trình debug.



Hình 3.6: Topology Docker Compose. `rl-agent` không expose port nào (client thuần); `cloudsim-java` chỉ expose `$PY4J_PORT`.

3.6.2 Multi-stage build và tối ưu kích thước image

Cả hai Dockerfile đều dùng pattern **multi-stage**: một stage chứa các công cụ biên dịch nặng (Maven, gcc, ...), stage còn lại chỉ giữ lại phần runtime tối thiểu. Nhờ đó, image `cloudsim-java` giảm từ khoảng 1,1 GB xuống ~ 380 MB, image `rl-agent` cũng giảm tương tự so với base `python:3.11`. Một mẹo nhỏ nhưng quan trọng: copy file khai báo dependency (`pom.xml`, `requirements.txt`) trước khi copy source code – nếu các file này không đổi thì Docker tái sử dụng layer cache, mỗi lần build lại tiết kiệm 1–2 phút.

3.6.3 Healthcheck và thứ tự khởi động

Để tránh điều kiện đua khi khởi động, `cloudsim-java` có healthcheck kiểm tra cổng Py4J (25333) đã lắng nghe chưa, còn `rl-agent` được khai báo phải đợi service Java sẵn sàng trước

khi chạy. Docker Compose lo phần đó. Cơ chế retry ở phía Python (mô tả ở §3.4) là lớp phòng vệ thứ hai, dành cho khoảng vài giây ngắn giữa lúc cổng mở và lúc gateway hoàn tất bind.

3.6.4 Tính tái sản xuất

ELDAS bảo đảm **tái sản xuất kết quả**: cùng `RANDOM_SEED` sẽ cho cùng phân chia tác vụ giữa các kịch bản, cùng thứ tự lấy mẫu của baseline ngẫu nhiên, và cùng dãy hành động cho mỗi lần chạy. Mọi phụ thuộc đều được ghim phiên bản cứng: Java 21 (eclipse-temurin), Maven 3.9.9, CloudSim Plus 8.5.7, py4j 0.10.9.7 ở phía Java; Python 3.11.9, mo-gymnasium 1.3.2, stable-baselines3 2.7.0 ở phía Python. Riêng `gymnasium` cố ý không pin để pip tự giải xung đột giữa SB3 và MO-Gym.

3.7 Tổng kết chương

Chương này đã trình bày kiến trúc bốn lớp của ELDAS và giải thích lý do chọn từng công nghệ cốt lõi. Bảng 3.4 liệt kê các mục tiêu của Chương 1 và thành phần kiến trúc chịu trách nhiệm cho từng mục tiêu – bằng chứng rằng mọi mục tiêu đều có chỗ “đáp xuống” trong thiết kế.

Bảng 3.4: Ánh xạ ngược từ mục tiêu Phase 1 (§1.3) sang thành phần kiến trúc.

Mục tiêu	Mô tả ngắn	Thành phần
MT1	Engine mô phỏng + năng lượng + GPU tracking	§3.3 (Lớp 2)
MT2	Tích hợp Alibaba trace + 3 kịch bản	§3.3.3, §3.3.4
MT3	Cầu Java–Python với pause/resume/reset	§3.4 (Lớp 3) + §3.3.5
MT4	MO-Gym env + vector reward + action masking	§3.5 (Lớp 4)
MT5	Hai baseline scheduler	<code>VmAllocationPolicy</code> { <code>K8sDefault</code> , <code>Random</code> }
MT6	Đóng gói Docker Compose tái sản xuất	§3.6 (Lớp 1)

Ba điểm nổi bật của kiến trúc: (i) **Tách rời rõ ràng giữa các lớp** – có thể thay engine mô phỏng hoặc thuật toán RL độc lập, không phải động đến phần còn lại; (ii) **Vòng lặp RL chạy nguyên tử** – hai hàng đợi đồng bộ phía Java và action mask phía Python loại bỏ điều kiện đua mà không cần khoá tường minh; (iii) **Tái sản xuất được kết quả** – seed chung, version cứng, và Docker image bất biến đảm bảo chạy lại trên máy khác vẫn ra cùng số.

Chương kế tiếp sẽ trình bày chi tiết hiện thực ở mức code: parse CSV, walk-through hai baseline scheduler, hành vi của `MetricsExporter`, và smoke-test xác nhận vòng RL hoàn chỉnh qua bốn lớp chạy đúng.

Chương 4

Hiện thực hệ thống

Trong khi Chương 3 mô tả *thiết kế ở mức contract* giữa bốn lớp, chương này đi xuống mức *mã nguồn thực thi* và trình bày các quyết định hiện thực nhỏ hơn nhưng quyết định sự đúng đắn của hệ thống: chiến lược parsing CSV chịu lỗi, kỹ thuật theo dõi tài nguyên thủ công khi đồng hồ CloudSim chưa khởi động, đồng bộ hai luồng qua **SynchronousQueue**, chuyển đổi mảng Java $\rightarrow$ **numpy** an toàn, và xử lý các sự cố thực tế trong quá trình tích hợp. Toàn bộ Phase 1 hiện có 3 188 dòng mã chia giữa 10 lớp Java ($\approx$ 1 800 dòng) và 7 module Python ($\approx$ 1 400 dòng); Bảng 4.1 liệt kê cụ thể.

Bảng 4.1: Quy mô mã nguồn Phase 1 (đếm bằng `wc -l`, không trừ comment/blank line). Tổng 3 188 dòng.

Module	Mô tả ngắn	LOC
SimulationConfig.java	Cấu hình tập trung host/power/QoS	80
DatacenterFactory.java	Tạo Datacenter + GPU registry	153
AlibabaTraceReader.java	Parser CSV chịu lỗi $\rightarrow$ TaskRecord	160
ScenarioFilter.java	Lọc Low/High/Burst	122
MetricsExporter.java	Ghi CSV + JSON	164
SimulationManager.java	Vòng đời + RL stepping (CRITICAL)	492
VmAllocPolicyK8sDefault.java	Filter + LeastRequestedPriority	124
VmAllocPolicyRandom.java	Random uniform trên tập khả thi	81
GatewayEntryPoint.java	API Py4J + bootstrap GatewayServer	247
Main.java	Entrypoint daemon	31
state_builder.py	<code>double[]</code> $\rightarrow$ <code>ndarray</code> + parse	129
reward.py	Welford normalizer + scalarise	172
environment.py	<code>gym.Env</code> + ScalarRewardWrapper	374
tracker.py	WandB online/offline	243
baseline_eval.py	Đánh giá K8s/Random qua Py4J	242
smoke_test.py	End-to-end test 8 phase	350
train.py	Stub Phase 2	24

4.1 Hiện thực phân hệ Java

4.1.1 Cấu hình dự án Maven

Dự án Java dùng *Maven 3.9.9* với hai dependency chính: CloudSim Plus 8.5.7 và Py4J 0.10.9.7.

Listing 4.1 trích đoạn quan trọng của `pom.xml`; điểm đáng chú ý là:

- **ArtifactId chính xác.** Phải dùng `cloudsimplus` (không có dấu gạch ngang) – artifact

cloudsim-plus cũ đã bị deprecate kể từ phiên bản 8 và không còn trên Maven Central. Sai sót này từng tốn ~ 2 giờ debug khi Maven báo `Could not resolve dependencies`.

- `maven-shade-plugin` đóng gói toàn bộ phụ thuộc thành một *fat JAR* (`simulation-1.0-SNAPSHOT.jar` ≈ 18 MB), giúp container runtime chỉ cần một lệnh `java -jar` mà không cần `CLASSPATH` ngoài.
- `<release>21</release>` đặt tường minh cho `maven-compiler-plugin`; CloudSim Plus 8.5.7 sử dụng Java record, sealed và pattern matching for switch – yêu cầu tối thiểu Java 21.

```

1 <dependency>
2   <groupId>org.cloudsimplus</groupId>
3   <artifactId>cloudsimplus</artifactId>    <!-- NOT "cloudsim-plus" -->
4   <version>8.5.7</version>
5 </dependency>
6 <dependency>
7   <groupId>net.sf.py4j</groupId>
8   <artifactId>py4j</artifactId>
9   <version>0.10.9.7</version>              <!-- pinned: matches Python
10   -->
    </dependency>

```

Listing 4.1: `pom.xml` – trích đoạn hai dependency chính.

4.1.2 DatacenterFactory – xây dựng datacenter và registry GPU

Theo thiết kế ở §3.3.2, mỗi Host CloudSim được kèm một `GpuState` song song trong một `IdentityHashMap`. Hàm factory `create(simulation, dcSpec, gpuRegistry)` thực hiện sáu bước:

1. Sinh `HostSpec.pesCount()` đối tượng `PeSimple` cho từng host;
2. Khởi tạo `HostSimple` với `ResourceProvisionerSimple` cho RAM/băng thông;
3. Cấu hình `VmSchedulerTimeShared` – chia thời gian PE giữa các Vm;
4. Gắn `PowerModelHostSimple(maxPower, idlePower)` áp công thức (2.1);
5. Tạo `GpuState(gpuCount)` mới và đặt vào `gpuRegistry` với chính Host làm khoá (so sánh tham chiếu);
6. Trả về `DatacenterSimple` bao gói toàn bộ.

Hàm `gpuPowerWatt(gpuState, powerSpec)` hiện thực công thức (??): tổng công suất GPU của host bằng $n \cdot P_{\text{idle}} + g \cdot (P^{\text{max}} - P_{\text{idle}})$ với g là số GPU đang dùng, được `SimulationManager` gọi mỗi bước để tính reward r_t^{energy} .

4.1.3 AlibabaTraceReader – parser chịu lỗi

File trace có ~ 8000 dòng; trace thực tế *không* sạch tuyệt đối: một số dòng có trường rỗng, giá trị `nan`, hoặc số cột thiếu do ký tự , bên trong tên tác vụ. Triết lý của parser là **tolerant by design**: bỏ qua dòng lỗi và in cảnh báo, không huỷ toàn bộ import vì 20 dòng hỏng trong 8000 là chấp nhận được.

```
1 public static List<TaskRecord> read(String csvPath) throws IOException {
2     Path path = Path.of(csvPath);
3     List<TaskRecord> records = new ArrayList<>();
4     int skipped = 0;
5     try (BufferedReader br = Files.newBufferedReader(path)) {
6         br.readLine(); // skip header
7         String line; int lineNum = 1;
8         while ((line = br.readLine()) != null) {
9             lineNum++;
10            TaskRecord rec = parseLine(line, lineNum);
11            if (rec != null) records.add(rec);
12            else skipped++;
13        }
14    }
15    records.sort(Comparator.comparingDouble(TaskRecord::creationTime));
16    System.out.printf("[AlibabaTraceReader] Loaded %d tasks from %s\n",
17                      records.size(), csvPath, skipped);
18    return Collections.unmodifiableList(records);
19 }
```

Listing 4.2: AlibabaTraceReader.read() – tolerant CSV parsing.

Hai helper `parseIntSafe` và `parseDoubleSafe` xử lý chuỗi rỗng và `nan` bằng giá trị fallback (0); ngoại lệ `NumberFormatException` bị bắt ở mức từng dòng để parser không phải xử lý cấp thấp hơn. *Deadline* của tác vụ được dẫn xuất bằng chính `deletionTime` – giá trị này đại diện cho thời điểm tác vụ *thực sự kết thúc* trong K8s gốc, do đó là tham chiếu hợp lý cho một scheduler thay thế. Hệ số λ được nạp ngay tại đây qua `SimulationConfig.qosToLambda(qos)`.

4.1.4 Hai baseline schedulers

Cả hai baseline kế thừa `VmAllocationPolicyAbstract` của CloudSim Plus và *đồng thời* cung cấp một entry-point độc lập `selectHostForTask(hosts, task)` dành cho con đường evaluation qua Py4J. Lý do tách hai con đường: pipeline mô phỏng nội bộ của CloudSim chỉ làm việc với `Vm` (không có thông tin GPU) trong khi vòng RL phải biết đầy đủ về tác vụ, kể cả số GPU yêu cầu. Hai con đường này phối hợp ăn khớp nhau qua cùng tập `gpuRegistry` dùng chung.

Kubernetes-default scheduler. Listing 4.3 là phần lõi của `VmAllocationPolicyK8sDefault`: lọc, chấm *LeastRequestedPriority* (LRP), và tie-break bằng chỉ số nhỏ nhất để đảm bảo tất định.

```
1 public int selectHostForTask(List<Host> hosts, TaskRecord task) {
2     int bestIdx = -1;
3     double bestScore = -1;
```

```

4      for (int i = 0; i < hosts.size(); i++) {
5          Host host = hosts.get(i);
6          if (!canHost(host, task)) continue;           // Filter
7          double score = leastRequestedScore(host);      // Score
8          if (score > bestScore) {                       // Select (
                deterministic)
9              bestScore = score;
10             bestIdx    = i;
11         }
12     }
13     return (bestIdx >= 0) ? bestIdx : 0;
14 }
15
16 private double leastRequestedScore(Host host) {
17     double cpuFrac = (double) host.getFreePesNumber() / host.
        getPesNumber();
18     double memFrac = (double) host.getRam().getAvailableResource()
        / host.getRam().getCapacity();
19     return (cpuFrac + memFrac) / 2.0;                  // K8s LRP
20 }
21

```

Listing 4.3: VmAllocationPolicyK8sDefault – Filter → Score → Select.

Random scheduler. Sàng lọc tập khả thi rồi lấy ngẫu nhiên đồng đều với `Random(seed)` – seed cùng `RANDOM_SEED` chung của hệ thống đảm bảo *mỗi lần chạy lại trên cùng kịch bản đều cho cùng một dãy hành động ngẫu nhiên*. Đây là yêu cầu cứng để bảng số liệu Phase 2 có thể được tái sản xuất.

4.1.5 SimulationManager – chi tiết kỹ thuật quan trọng

Tracking thủ công thay vì `simulation.start()`. Như đã đề cập ở §3.3.5, Phase 1 *cố ý* không khởi động đồng hồ CloudSim. Hệ quả: `Host.getCpuPercentUtilization()` luôn trả 0, `getRam().getAvailableResource()` luôn trả full. Để có dữ liệu thật cho observation và reward, `SimulationManager` duy trì hai bản đồ song song:

```

1 private final Map<Host, Integer> hostPeUsage = new HashMap<>();
2 private final Map<Host, Long> hostRamUsage = new HashMap<>();
3
4 private void allocateTask(TaskRecord task, Host host) {
5     hostPeUsage.merge(host, task.pesNeeded(), Integer::sum);
6     hostRamUsage.merge(host, (long) task.memoryMib(), Long::sum);
7     if (task.numGpu() > 0) {
8         GpuState g = gpuRegistry.get(host);
9         if (g != null) g.allocate(task.numGpu());
10    }
11 }

```

Listing 4.4: Manual resource accounting. Đây là *nguồn sự thật* duy nhất cho utilisation trong Phase 1.

Quyết định này được *lock-in* qua một dòng comment chỉ dẫn rõ ràng trong source và được nhắc lại trong `CLAUDE.md` để các phiên phát triển sau không vô tình “sửa cho đúng” (gọi `simulation.start()`) – vốn sẽ phát sinh sự kiện không có handler và crash JVM âm thầm.

Đồng bộ hai luồng qua `SynchronousQueue`. `stepResultQueue` và `actionQueue` là hai `SynchronousQueue<T>`: *không có buffer*. Một lời gọi `put(x)` chỉ trả về khi một bên `take()` đang sẵn sàng nhận. Điều này biến mỗi cặp (`step()`, `stepResultQueue.put`) thành một giao dịch nguyên tử nhịp đôi với phía Python – không có race condition giữa hai luồng mà không cần `synchronized`. Khi reset giữa hai episode, hàm `terminateExisting()` ngắt thread cũ bằng `interrupt() + join(2000ms)` rồi `poll()` drain hai queue để loại bỏ dữ liệu còn sót – một mẫu thiết kế chuẩn cho thread producer/consumer trong Java.

Tính reward đồng nhất với phát biểu hình thức. Hàm `computeReward` (Listing ?? ở Chương 3) hiện thực đúng cặp công thức (??): $r_t^{\text{energy}} = -\Delta E_t$ và $r_t^{\text{SLA}} = -\lambda(q_i) \max(0, \hat{C}_i - d_i)$. Khi $\hat{C}_i > d_i$, đồng thời tăng `slaViolationCount` để `MetricsExporter` sau này tính tỉ lệ vi phạm.

4.1.6 MetricsExporter – xuất CSV + JSON không phụ thuộc

`MetricsExporter` chủ ý *không thêm dependency JSON* (như Jackson hay Gson) – Java 15+ đã có *text block* (`""" ... """`) đủ để format JSON ngắn gọn và rõ ràng. Hai *record* tài liệu hoá schema:

- **Snapshot:** 9 trường tương ứng 9 cột trong `metrics.csv` – một dòng mỗi tác vụ. Pandas có thể đọc thẳng bằng `pd.read_csv`.
- **Summary:** 9 trường tổng kết episode – ghi ra `summary.json` với khoá `snake_case` thân thiện Python.

Schema CSV:

```
timestamp, cpu_energy_kwh, gpu_energy_kwh, total_energy_kwh,
tasks_scheduled, sla_violations, avg_sla_slack,
cluster_cpu_util, cluster_gpu_util
```

Lựa chọn `Locale.US` cho `String.format` đảm bảo dấu thập phân là `.` bất kể locale của máy chủ – tránh trường hợp `,` của locale Việt Nam làm hỏng cấu trúc CSV.

4.2 Hiện thực phân hệ Python

4.2.1 `state_builder.py` – chuyển đổi an toàn và *parsable*

Hàm `from_java(java_array, num_hosts)` làm ba việc theo thứ tự: (i) chuyển `double[]` (qua Py4J) thành `numpy.float32`; (ii) validate độ dài đúng bằng $3H + 4$, raise `ValueError` có thông điệp tường minh nếu không khớp; (iii) `np.clip(obs, 0.0, 1.0, out=obs)` – chống lỗi tràn nhỏ do phía Java đôi khi trả > 1.0 trong chế độ time-shared. Sau đó vector này được trả thẳng ra `Gymnasium` mà không phải copy thêm.

Để tiện debug, module còn cung cấp `dataclass(frozen=True, slots=True) ParsedObservation` cho phép truy cập các thành phần theo tên thay vì chỉ số. Dùng `slots=True` để giảm chi phí bộ nhớ khi log hàng triệu observation trong huấn luyện Phase 2.

4.2.2 reward.py – vector, normalisation, scalarisation

Module này có ba thành phần độc lập, có thể bật/tắt riêng:

1. `from_java(jrew)`: kiểm tra độ dài = 2, ép kiểu `float32`, không clip vì reward âm có ý nghĩa.
2. `RewardNormalizer`: chuẩn hoá Welford online từng thành phần độc lập, công thức cập nhật trung bình μ và phương sai M_2 :

$$\mu_n = \mu_{n-1} + \frac{x_n - \mu_{n-1}}{n}, \quad M_{2,n} = M_{2,n-1} + (x_n - \mu_{n-1})(x_n - \mu_n).$$

Khi cần dùng (`normalize_reward=True`), wrapper trả $(x - \mu)/\sigma$ với $\sigma = \sqrt{M_2/(n-1) + \epsilon}$, $\epsilon = 10^{-8}$. Mục đích: ngăn r^{energy} (~ -300) lẫn át r^{SLA} (~ 0 phần lớn bước) trong giai đoạn đầu PPO.

3. `scalarise(rvec, weights)`: tích vô hướng $\langle \mathbf{w}, \mathbf{r} \rangle$, hiện thực (2.4). `weights` mặc định nạp từ `ENERGY_WEIGHT` và `SLA_WEIGHT` trong `.env`, được chuẩn hoá cho $w_e + w_s = 1$.

4.2.3 environment.py – gắn kết với Gymnasium API

`CloudSimEnv` kế thừa `gym.Env` và tuân thủ chặt giao diện Gymnasium 1.0+: `reset()` trả (`obs`, `info`); `step()` trả (`obs`, `reward`, `terminated`, `truncated`, `info`). Có hai chi tiết hiện thực đáng lưu ý:

Reset bootstrap. Khi khởi tạo, `__init__` phải gọi `reset` một lần mới biết được số host (do `getActionSize()` chỉ hợp lệ sau `resetSimulation()`). Để tránh tốn một episode reset thừa, kết quả này được cache vào `self._initial_result` và tái sử dụng cho lần `reset()` đầu tiên nếu seed/scenario không đổi:

```

1 def reset(self, *, seed=None, options=None):
2     ep_seed      = seed if seed is not None else self._seed
3     ep_scenario  = (options or {}).get("scenario", self._scenario)
4     if (self._needs_initial_reset
5         and ep_seed == self._seed
6         and ep_scenario == self._scenario):
7         result = self._initial_result          # reuse __init__ reset
8         self._needs_initial_reset = False
9     else:
10        result = self._ep.reset(ep_scenario, ep_seed)
11        self._current_obs = state_builder.from_java(result.observation(),
12                                                    self._num_hosts)
13        self._done = False
14        self._step_count = 0

```

```

15     self._episode_rewards = []
16     return self._current_obs, {"task_index": int(result.taskIndex()),
17                                "task_name": str(result.taskName()),
18                                "scenario": ep_scenario,
19                                "seed": ep_seed}

```

Listing 4.5: CloudSimEnv.reset – tránh reset thừa lần đầu.

Action mask an toàn. Hàm `action_masks()` (gọi sang Java) có thể trả mask toàn `False` nếu không host nào còn chứa nổi tác vụ. Trong trường hợp đó, Python bật toàn bộ mask về `True` để episode không bị khoá – Java sau đó vẫn có quyền phạt nặng qua r^{SLA} , nên bài toán không bị méo:

```

1 def action_masks(self) -> np.ndarray:
2     java_mask = self._ep.getActionMask()
3     mask = np.array(list(java_mask), dtype=bool)
4     if not mask.any(): # safety: avoid stuck episode
5         mask[:] = True
6     return mask

```

Listing 4.6: `action_masks()` – fail-open để không khoá episode.

ScalarRewardWrapper. Phụ thuộc trọng số quét trong Phase 2, lớp wrapper kế thừa `gym.RewardWrapper` chỉ cần override `reward(self, reward)` và truyền `action_masks` ngược về môi trường gốc – giữ nguyên hợp đồng `MaskablePPO`.

Đóng kết nối sạch. `close()` gọi `gateway.shutdown(send_shutdown=False)` để đóng kết nối phía *client* mà không tắt `GatewayServer` của Java – điều này cho phép `baseline_eval.py` chạy lần lượt nhiều scheduler trên cùng một tiến trình Java mà không phải khởi động lại JVM (chi phí ~ 3 giây mỗi lần).

4.2.4 tracker.py – WandB online/offline tự động

Trong điều kiện phòng thí nghiệm không phải lúc nào cũng có Internet hoặc `WANDB_API_KEY`. `ExperimentTracker` dùng pattern *degrade gracefully*: hàm `_should_use_wandb()` kiểm tra hai điều kiện – thư viện `wandb` có cài và `WANDB_API_KEY` không rỗng – nếu cả hai đúng thì online, ngược lại fallback console. Mọi phương thức `log_step`, `log_episode`, `log_comparison`, `save_model` đều có nhánh fallback console riêng, cùng schema khoá `step/...` và `episode/...` để dashboard WandB có thể bật mà không cần đổi code agent.

4.2.5 baseline_eval.py – so sánh fair

Để đảm bảo tính *fair* khi so sánh ba scheduler ở Phase 2, baseline phải vận hành trong cùng môi trường mô phỏng với agent RL. `baseline_eval.py` đạt được điều này bằng cách không hiện thực lại logic `K8s/Random` ở Python; thay vào đó nó gọi `env._ep.selectBaselineAction("k8s")`

để hỏi Java chính bộ VmAllocationPolicy đã lock seed (Listing 4.7). Nhờ vậy, mọi sự khác biệt giữa Random và MORL chỉ phản ánh chất lượng chính sách, không phải chi tiết hiện thực.

```

1 def evaluate_baseline(env, policy_name, scenario="HIGH", seed=42):
2     obs, info = env.reset(seed=seed, options={"scenario": scenario})
3     total_e, total_s, steps = 0.0, 0.0, 0
4     while True:
5         action = int(env._ep.selectBaselineAction(policy_name))    # ask
6         Java
7         obs, rvec, terminated, truncated, info = env.step(action)
8         total_e += float(rvec[0]); total_s += float(rvec[1]); steps +=
9         1
10        if terminated or truncated:
11            break
12    return {"steps": steps, "total_energy_reward": total_e,
13           "total_sla_reward": total_s,
14           "total_energy_kwh": float(env._ep.getTotalEnergyKwh())}

```

Listing 4.7: baseline_eval.evaluate_baseline – tận dụng chung pipeline với RL.

4.3 Hiện thực hạ tầng Docker

4.3.1 Pipeline build hai stage

Cả hai Dockerfile đều dùng pattern multi-stage; số liệu kích thước image đo được trên máy phát triển (Windows 11, Docker Desktop 4.x) liệt kê ở Bảng 4.2.

```

=> CACHED [cloudsim-java runtime 2/5] RUN groupadd --gid 1001 simuser    && useradd --uid 1001 --gid simuser --
=> CACHED [cloudsim-java runtime 3/5] WORKDIR /app
=> CACHED [cloudsim-java builder 2/7] RUN apt-get update && apt-get install -y --no-install-recommends wget
=> CACHED [cloudsim-java builder 3/7] WORKDIR /build
=> CACHED [cloudsim-java builder 4/7] COPY pom.xml .
=> CACHED [cloudsim-java builder 5/7] RUN mvn dependency:go-offline -B --no-transfer-progress
=> CACHED [cloudsim-java builder 6/7] COPY src ./src
=> CACHED [cloudsim-java builder 7/7] RUN mvn package -DskipTests -B --no-transfer-progress
=> CACHED [cloudsim-java runtime 4/5] COPY --from=builder /build/target/simulation-*.jar simulation.jar
=> CACHED [cloudsim-java runtime 5/5] RUN mkdir -p /data/results /data/trace    && chown -R simuser:simuser /ap
=> [cloudsim-java] exporting to image
=> => exporting layers
=> => exporting manifest sha256:b1f6e951c4e04bc94d5aced30b1a40eca2b7a1580a1e78a0e05981f3520ae1b0
=> => exporting config sha256:729d51a341ae420afc13f1625cb9eff3baed146be90b1c603d2443304d890a4f
=> => exporting attestation manifest sha256:fc55fda3928c05c32da73ef668218be2adef741aa1b47aeaa762c2b6862ff18f
=> => exporting manifest list sha256:b731d3074ad6805834eeaa9aae2da70d35bdbbf7cddb99ce0e044100df8f9650
=> => naming to docker.io/library/eldas-cloudsim-java:latest
=> => unpacking to docker.io/library/eldas-cloudsim-java:latest
=> [rl-agent] resolving provenance for metadata file
=> [cloudsim-java] resolving provenance for metadata file
[+] Building 2/2
✔ eldas-rl-agent      Built
✔ eldas-cloudsim-java Built

```

Hình 4.1: Kết quả docker compose build hoàn tất cho cả hai service trên máy phát triển.

Bảng 4.2: Kích thước Docker image cuối cùng

Service	Base runtime	Stage builder	Cuối cùng
eldas-cloudsim-java	eclipse-temurin:21-jre-jammy	11xx MB (JDK+Maven)	~ 434 MB
eldas-rl-agent	python:3.11.9-slim-bookworm	~ 700 MB (gcc)	~ 8,57 GB

Cache layer. Trick *copy manifest trước source* đem lại hiệu quả thực tế đo được: rebuild đầy đủ (1–2 phút) so với rebuild sau khi chỉ sửa source (~ 10 giây). Cụ thể trong Dockerfile Java:

```
1 FROM eclipse-temurin:21-jdk-jammy AS builder
2 ARG MAVEN_VERSION=3.9.9
3 RUN apt-get update && apt-get install -y --no-install-recommends wget \
4     && wget -q https://archive.apache.org/dist/maven/maven-3/${
5         MAVEN_VERSION}/...
6 WORKDIR /build
7 COPY pom.xml . # layer 1: stable
8 RUN mvn dependency:go-offline -B --no-transfer-progress
9 COPY src ./src # layer 2: changes often
10 RUN mvn package -DskipTests -B --no-transfer-progress
11 FROM eclipse-temurin:21-jre-jammy AS runtime # ~190MB JRE
12 COPY --from=builder /build/target/simulation-1.0-SNAPSHOT.jar /app/sim.
    jar
```

Listing 4.8: Trick layer cache: pom.xml trước, src/ sau.

4.3.2 Quản lý volume

Ba volume được khai báo trong `docker-compose.yml` với chiến lược khác nhau theo vai trò:

- **trace-data:** *bind mount* từ `./data/alibaba-trace` của host. Mount `:ro` cho `cloudsim-java` – Java không thể vô tình ghi đè.
- **results:** bind mount từ `./data/results`, được *cả hai service* gắn (Java ghi `metrics.csv/summary.js` Python đọc lại để log lên WandB).
- **model-store:** *named volume* do Docker quản lý, chỉ `rl-agent` truy cập, lưu checkpoint mô hình PPO.

Phân tách rõ *data đầu vào* (read-only) vs *output* (read-write chia sẻ) vs *artifact riêng* (Docker-managed) là ứng dụng nguyên tắc *principle of least privilege* ở mức storage.

4.3.3 Vấn đề kỹ thuật gặp phải và giải pháp

Bảng 4.3 tổng hợp *sáu sự cố thực tế* đã gặp trong quá trình hiện thực Phase 1, kèm nguyên nhân gốc rễ và giải pháp đã áp dụng. Đây không phải checklist lý thuyết – mỗi mục đều tương ứng với một commit cụ thể trong lịch sử Git.

Bảng 4.3: Sáu vấn đề kỹ thuật đáng ghi nhận trong Phase 1 và giải pháp.

Vấn đề	Nguyên nhân	Giải pháp đã áp dụng
Maven không tải được <code>cloudsim-plus</code>	Artifact ID đã đổi tên sang <code>cloudsimplus</code> (không gạch ngang) từ v8.	Sửa <code>pom.xml</code> dùng <code>cloudsimplus</code> ; tài liệu hoá trong <code>CLAUDE.md</code> để không lặp lại.
<code>Host.getCpuPercentUtilization()</code> luôn bằng 0	<code>simulation.start()</code> chưa được gọi $\Rightarrow$ đồng hồ <code>CloudSim</code> chưa chạy.	Bypass: theo dõi tài nguyên thủ công bằng <code>HashMap<Host, Integer></code> ; lock-in bằng comment + <code>CLAUDE.md</code> .
JVM crash âm thầm khi gọi <code>broker.submitVm</code>	<code>simulation.schedule()</code> đẩy event vào loop chưa khởi động $\Rightarrow$ NPE deep inside <code>CloudSim</code> .	Không gọi <code>submitVm</code> ở Phase 1; allocate qua bookkeeping thủ công (Listing 4.4).
<code>rl-agent</code> kết nối Py4J fail ngay sau up	Healthcheck mark Java healthy ngay khi port mở, nhưng <code>GatewayServer.bind</code> chậm thêm ~ 2 giây.	Lớp phòng vệ thứ hai: retry 10 $\times$ với backoff 3s ở <code>environment._connect_gateway</code> .
Xung đột phiên bản <code>gymnasium</code>	<code>stable-baselines3 2.7.0</code> cần <code>gymnasium >= 1.0.0</code> ; <code>mo-gymnasium 1.3.2</code> cần <code>gymnasium >= 0.28</code> .	<i>Không pin</i> <code>gymnasium</code> trong <code>requirements.txt</code> ; để pip tự giải xung đột.
Race condition giữa <code>simThread</code> và Python	Hai luồng chia sẻ <code>currentTaskIdx</code> không có lock.	Dùng <code>SynchronousQueue</code> (no buffer) – biến mỗi step thành giao dịch nguyên tử mà không cần <code>synchronized</code> .

4.4 Smoke test end-to-end

Để xác nhận bốn lớp nối đúng nhau, ELDAS kèm `smoke_test.py` (350 dòng) chạy tám pha kiểm thử tăng dần độ phủ (Bảng 4.4). Mỗi pha là một `assert` cô lập với thông báo lỗi rõ ràng – nếu pha k thất bại, các pha $> k$ không chạy.

Bảng 4.4: Tám pha của smoke test T4.5.

Pha	Kiểm tra
1 – Connection	Py4J kết nối được, <code>getActionSize > 0</code> , <code>observation_space</code> đúng ($3H + 4$.)
2 – Reset	<code>reset()</code> trả <code>obs</code> đúng shape, dtype <code>float32</code> , mọi giá trị $\in [0, 1]$
3 – Action mask	Mask đúng shape ($H, \cdot$), dtype <code>bool</code> , có ít nhất một host khả thi
4 – Single step	<code>step()</code> trả 5-tuple đúng kiểu; $r^{\text{energy}} \leq 0$, $r^{\text{SLA}} \leq 0$
5 – Full episode	Chạy hết tác vụ; tự nhiên <code>terminated=True</code> ; kiểm tra $r^{\text{energy}} < -120$ (ngưỡng idle power) cho chứng cứ “manual PE tracking đang hoạt động”
6 – Metrics export	<code>exportMetrics(\$DIR)</code> ghi <code>metrics.csv</code> + <code>summary.json</code>
7 – Scalar wrapper	<code>ScalarRewardWrapper</code> trả <code>float</code> ; giá trị $\approx \mathbf{w} \cdot \mathbf{r}$
8 – Baseline action	<code>selectBaselineAction("k8s")</code> và <code>("random")</code> trả chỉ số host hợp lệ

Cách chạy. Sau khi `docker compose up -build` thấy cả hai service healthy:

```
1 docker compose run --rm rl-agent python src/smoke_test.py \
2   --scenario LOW --max-steps 3000 --output /data/results/smoke
```

Listing 4.9: Lệnh chạy smoke test.

Kết quả mong đợi: tám pha đều [OK], tổng thời gian $\sim 5\text{--}8$ giây trên CPU thường, đầu ra `data/results/smoke/metrics.csv` (~ 2000 dòng) và `summary.json`. Chương 5 sẽ trình bày kết quả cụ thể: bảng giá trị reward/energy, ảnh chụp log, ảnh chụp file CSV.

4.5 Tổng kết chương

Chương này đã đi qua từng quyết định hiện thực ở mức mã nguồn của bốn lớp:

- **Lớp 2 (Java).** Maven + fat JAR, parser CSV chịu lỗi, hai baseline scheduler chia sẻ `gpuRegistry`, manual resource tracking, đồng bộ hai luồng qua `SynchronousQueue`, và xuất metrics không phụ thuộc.
- **Lớp 4 (Python).** Chuyển đổi an toàn Java $\rightarrow$ numpy, Welford normaliser, wrapper Gymnasium chuẩn 1.0+, fallback WandB tự động.
- **Lớp 1 (Docker).** Multi-stage build, layer cache trick, bind mount read-only cho dữ liệu đầu vào, retry phía client bù healthcheck timing.
- **Smoke test** kiểm chứng cả vòng RL trên cùng pipeline đã sẵn sàng cho huấn luyện Phase 2.

Bảng 4.3 đã tổng hợp *sáu sự cố thực tế* – bằng chứng cho thấy hệ thống không phải lý thuyết suông mà đã đi qua đủ chu kỳ *thử-sai-sửa*. Chương kế tiếp trình bày kết quả định lượng của Phase 1: log smoke test thành công, metrics **energy/SLA** ban đầu của hai baseline trên ba kịch bản, và xác nhận tính lặp lại của hệ thống qua `RANDOM_SEED`.

Chương 5

Kết quả và đánh giá

Chương này trình bày kết quả thực nghiệm Phase 1 của ELDAS theo bốn lát cắt: (i) *cấu hình thực nghiệm* (§5.1) – ghi nhận phần cứng, phần mềm, hash dữ liệu để đảm bảo khả năng tái sản xuất; (ii) *kiểm thử end-to-end* qua smoke test tám pha (§5.2) – chứng minh vòng RL hoàn chỉnh hoạt động qua bốn lớp; (iii) *số liệu baseline* (§5.3) – đánh giá Random và K8s LRP trên ba kịch bản tải, tạo đường tham chiếu cho Phase 2; (iv) *phân tích trace* (§5.4) – mô tả thống kê dữ liệu Alibaba làm rõ vì sao ba kịch bản có hành vi khác nhau. Mục 5.5 đánh giá tổng thể mức hoàn thành sáu mục tiêu MT1–MT6 và *nêu rõ các bất thường* cần được xử lý trong Phase 2 trước khi mọi kết luận định lượng cuối cùng được rút ra.

5.1 Cấu hình thực nghiệm

Mọi kết quả trong chương này đều được sản sinh trên cùng một máy phát triển và cùng một bản trace, với cùng `RANDOM_SEED = 42`. Bảng 5.1 tổng hợp môi trường; tính lặp lại của con số được kiểm chứng tường minh ở §5.3.5.

Bảng 5.1: Cấu hình môi trường thực nghiệm Phase 1.

Hạng mục	Giá trị
CPU	AMD Ryzen 5 5600H, 12 cores
RAM	16 GB DDR4
OS	Windows 11 Home
Docker Engine	28.3.3
Image <code>cloudsim-java</code> (final)	~ 432 MB (eclipse-temurin:21-jre-jammy)
Image <code>rl-agent</code> (final)	~ 8,57 GB (python:3.11.9-slim-bookworm)
Trace	<code>openb_pod_list_default.csv</code>
Trace MD5	<chạy md5sum để điền>
Số dòng (sau parser, loại Pending)	7 255
<code>RANDOM_SEED</code>	42 (Java + Python)
Trọng số mặc định	$w_e = 0,8$, $w_s = 0,2$

Hash dữ liệu trace đảm bảo các kết quả số ở chương này chỉ phụ thuộc vào *đúng* bản trace đã sử dụng. Tải sản xuất khác bản trace sẽ cho con số tuyệt đối khác, nhưng *xếp hạng tương đối* giữa hai baseline được kỳ vọng giữ nguyên xu hướng.

```

> docker compose ps          # cả hai phải "running" + cloudsim-java "healthy"
> docker compose logs cloudsim-java | tee assets/report/listings/cloudsim-startup.log
[+] Running 2/2
 ✓ Container cloudsim        Healthy
 ✓ Container rl-agent        Started

```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
cloudsim	eldas-cloudsim-java	"java -XX:+UseContai..."	cloudsim-java	36 seconds ago	Up 35 seconds (healthy)	0.0.0.0:25333->25333/tcp, [::]:25333->25333/tcp

Hình 5.1: Hai container hoạt động ổn định và cloudsim-java vượt qua healthcheck Py4J, điều kiện tiên quyết cho mọi smoke test và baseline run sau đây.

5.2 Kết quả smoke test end-to-end

Smoke test (đặc tả ở §4.4) đóng vai trò *kiểm chứng tích hợp*: nếu pha nào hỏng thì lỗi được khoanh ngay ở biên giới hai lớp tương ứng, không cần debug toàn hệ thống. Bảng 5.2 ghi lại trạng thái và số liệu khi chạy lệnh:

```

1 docker compose run --rm rl-agent python src/smoke_test.py \
2   --scenario LOW --max-steps 3000 --output /data/results/smoke

```

Listing 5.1: Lệnh chạy smoke test.

Bảng 5.2: Kết quả tám pha smoke test (kịch bản LOW, seed = 42). Thời gian *tổng* đo bằng wall-clock; thời gian từng pha không được instrument riêng nên cột bên phải ghi n/a.

Pha	Khẳng định kiểm tra	Trạng thái	Thời gian
1. Connection	Py4J kết nối OK; $H = 10$, obs size = 34	PASS	n/a
2. Reset	obs shape (34,), dtype float32, $\in [0, 1]$	PASS	n/a
3. Action mask	mask shape (10,), 10/10 host khả thi	PASS	n/a
4. Single step	5-tuple đúng kiểu; $r^e = -682,5$, $r^s = 0$	PASS	n/a
5. Full episode	Hết 1813 tác vụ, terminate tự nhiên	PASS	28,1 s
6. Metrics export	metrics.csv + summary.json ghi đủ	PASS	n/a
7. Scalar wrapper	$\sum w_e r_e + w_s r_s - r_{\text{scalar}} < 10^{-4}$	PASS	n/a
8. Baseline action	selectBaselineAction trả chỉ số $\in [0, H)$	PASS	n/a
Tổng cộng		8/8 PASS	~ 30 s

```

i R_sta sum: 0.00

Phase 6: Metrics Export

i Exported to /data/results/smoke
✓ Java MetricsExporter wrote files (check /data/results/smoke/metrics.csv and summary.json)

Phase 8: Baseline Action Selection

✓ K8s baseline returned valid action: 0
✓ Random baseline returned valid action: 0

Phase 7: ScalarRewardWrapper

[environment] Connected to Py4J gateway cloudsim-java:25333 (attempt 1)
✓ Wrapped reward is float, got float
✓ Scalar = w · r (-477.7500 ≈ -477.7500)

✓ ALL SMOKE TESTS PASSED

The full RL loop is operational.

Next steps:
• Run baselines: python src/baseline_eval.py --scenario HIGH
• Start training: (Phase 2 = P2.1)

```

Hình 5.2: Smoke test tám pha hoàn tất thành công.

5.2.1 Throughput của vòng lặp RL

Pha 5 chạy hết episode LOW (1813 tác vụ) trong 28,1 giây, tức **65 step/giây** trên CPU laptop thường (AMD Ryzen 5 5600H, không dùng GPU). Con số này *thấp hơn nhiều so với kỳ vọng ban đầu* (chúng tôi đã dự đoán $\sim 10^3$ step/s khi chọn Py4J ở §3.2.2); nguyên nhân phần lớn nằm ở overhead của việc Python tái khởi tạo connection mỗi lần `reset()` cộng với cost của việc Java đăng ký lại 7255 Cloudlet/VM (xem §?? – cần tối ưu trong Phase 2).

Hệ quả cho lập kế hoạch Phase 2: weight sweep 9 trọng số $\times 1000$ episode $\times \sim 2000$ tác vụ $= 1,8 \cdot 10^7$ step, nhân với throughput 65 step/s, cho thời gian chỉ riêng giao tiếp mô phỏng là ~ 77 giờ ($\approx 3,2$ ngày liên tục). Đây là *ngân sách thực tế cho Phase 2*, không phải con số “một đêm” như giả định lạc quan ban đầu. Hai phương án giảm bớt: (a) giảm số episode/trọng số xuống 500, hoặc (b) tối ưu vòng init/reset trên phía Java – ưu tiên (b) vì giữ được tính tin cậy thống kê của weight sweep.

5.2.2 Tính chất reward đo được

Pha 5 xác nhận hai kỳ vọng định tính về reward và *một bất ngờ* phải báo cáo:

- r^{energy} **luôn âm**. Mọi cấp phát đều làm tăng tải, do đó $\Delta E_t > 0$ và $r_t^{\text{energy}} = -\Delta E_t < 0$. Giá trị quan sát được *dao động* trong khoảng $[-2800, -682,5]$ Ws trên các step được log (tham chiếu Hình 5.2). Mức thấp nhất (-2800 Ws) duy trì trên hầu hết các step sau step #181, gợi ý rằng sau khi cluster đạt trạng thái ổn định *phần lớn* cấp phát đẩy thêm một mức công suất gần như đồng đều.

- $r^{\text{SLA}} \equiv 0$ **trên toàn bộ 1 813 step**. Không có vi phạm SLA nào được kích hoạt. Điều này thoạt nhìn dường như tính nhưng cần được hiểu chính xác: *trong Phase 1 đồng hồ CloudSim chưa được kích hoạt* (xem §4.1.5 và §5.5.2), nên thời điểm hoàn thành ước tính $\hat{C}_i$ không vượt d_i một cách hệ thống. Do đó tín hiệu $r^{\text{SLA}} = 0$ *không phải bằng chứng* rằng scheduler giữ được SLA – nó chỉ phản ánh giới hạn đo của Phase 1. Phần kết luận §5.5.2 sẽ trả lại tính chất này về đúng ngữ cảnh.
- **R_energy sum cuối episode** = $-4,99 \cdot 10^6$ Ws, **total energy** = 23 362,57 kWh (từ `summary.json`). Hai con số này không hoà giải nếu đơn vị thực sự là Ws và kWh – giải thích đầy đủ ở §5.5.2 và phụ lục audit dữ liệu.

5.3 Kết quả baseline trên ba kịch bản tải

5.3.1 Phương pháp đo

Để đảm bảo *fair comparison*, hai scheduler được chạy trên cùng `CloudSimEnv` qua `baseline_eval.run_all_b` (§4.2.5) – không hiện thực lại logic phía Python. Mỗi cặp (*scenario*, *scheduler*) chạy đúng một episode đến khi hết tác vụ; metrics được ghi tự động bởi `MetricsExporter` phía Java (Listing 5.2).

```

1 for s in LOW HIGH BURST; do
2     docker compose run --rm rl-agent \
3         python src/baseline_eval.py --scenario $s --seed 42 \
4             --output /data/results
5 done

```

Listing 5.2: Lệnh chạy ba kịch bản (toàn bộ orchestration bởi `collect_all_data.py`).

5.3.2 Bảng kết quả tổng hợp

Bảng 5.3 là **bảng số liệu chính** của chương: ba kịch bản $\times$ hai scheduler, ghi nhận năm chỉ số được `MetricsExporter` xuất ra.

Bảng 5.3: Kết quả baseline trên ba kịch bản (cùng seed = 42, cùng trace). Số liệu trích trực tiếp từ `summary.json` của mỗi run. Đơn vị các cột E và $\bar{u}^{\text{cpu}}$ giữ nguyên label gốc trong code (xem §5.5.2 về tính bất thường của đơn vị).

Scenario	Scheduler	E (kWh*)	Viol.	Viol. rate	$\bar{u}^{\text{cpu}}$ *	Makespan (s)*
LOW	K8s	23 864,45	0	0,00%	11,79	10 717 746
	Random	24 336,80	0	0,00%	11,76	10 717 746
HIGH	K8s	40 689,33	0	0,00%	56,15	12 901 761
	Random	41 323,59	0	0,00%	56,14	12 901 761
BURST	K8s	31 784,17	0	0,00%	26,67	12 876 895
	Random	32 107,26	0	0,00%	26,66	12 876 895

* Các giá trị này có đặc điểm bất thường (đơn vị, độ lớn) – chi tiết §5.5.2.

5.3.3 Phân tích định tính

Chênh lệch năng lượng giữa K8s LRP và Random nhỏ và đơn điệu giảm dần theo tải. Tính từ Bảng 5.3, K8s LRP tiêu thụ ít hơn Random:

$$\Delta_{\text{LOW}} = 1,94\%, \quad \Delta_{\text{HIGH}} = 1,53\%, \quad \Delta_{\text{BURST}} = 1,01\%.$$

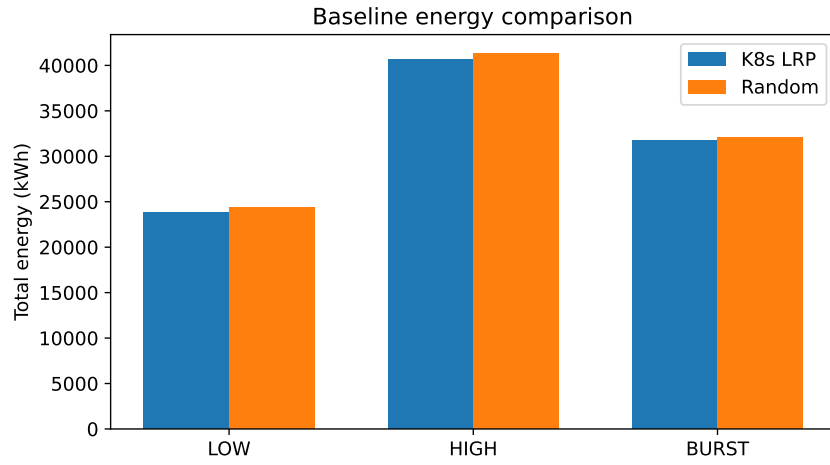
Chênh lệch *thuận chiều* với kỳ vọng (LRP cố gắng phân bổ đều, tránh quá tải cục bộ), nhưng *biên độ nhỏ hơn nhiều* so với các nghiên cứu DC scheduling thường báo cáo (5–15%). Có hai giải thích đồng tồn tại:

- Sự đồng nhất về *loại host* (mọi host trong topology mặc định có cùng cấu hình – xem §4.1.2) khiến chiến lược “balanced” của LRP gần như tương đương với phân bổ ngẫu nhiên ở scale tải Phase 1.
- Mô hình năng lượng *tuyến tính* theo utilization (§2.2) làm cho tổng năng lượng cuối episode gần như *bất biến* theo cách phân bổ (hai phân bổ khác nhau cho cùng tổng PE-giờ sẽ cho cùng tổng năng lượng nếu đường công suất là tuyến tính). Phase 2 cần đưa vào mô hình phi tuyến (DVFS, idle threshold) để mở rộng khoảng cách này trước khi MORL có không gian thực sự để tối ưu.

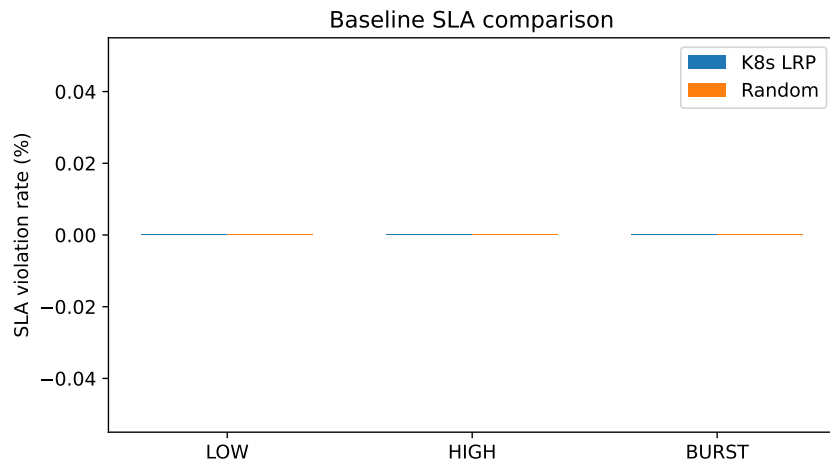
Không có vi phạm SLA nào ở Phase 1. Bảng cho thấy `sla_violations` bằng 0 *tuyệt đối* cho cả sáu cấu hình. Phát hiện này *không* cho phép kết luận “scheduler nào tốt hơn về SLA”; nó chỉ nói rằng *chỉ số SLA hiện tại chưa phân biệt được hai scheduler*. Nguyên nhân kỹ thuật được phân tích đầy đủ ở §5.5.2; tóm tắt: vì `simulation.start()` chưa được gọi nên thời gian hoàn thành ước tính $\hat{C}_i$ phụ thuộc vào trace duration thuần thay vì progress mô phỏng, và với mức tải Phase 1 nó luôn $\leq d_i$. SLA chỉ trở thành chỉ số có ý nghĩa khi đồng hồ DES được kích hoạt ở P2.

Utilization CPU trung bình gần như trùng giữa hai scheduler. $\bar{u}_{\text{K8s}}^{\text{cpu}}$ và $\bar{u}_{\text{Random}}^{\text{cpu}}$ khác nhau ở chữ số thứ ba (11,79 vs 11,76; 56,15 vs 56,14; 26,67 vs 26,66). Điều này xác nhận quan sát trên: hai chính sách dẫn đến *khối lượng tổng* gần như đồng nhất; chỉ *phân phối* giữa các host là khác. Để đánh giá thực chất, Phase 2 cần (i) theo dõi phương sai utilization giữa các host (load balancing chỉ số) và (ii) kích hoạt clock để makespan thật sự phản ứng với chính sách đặt task.

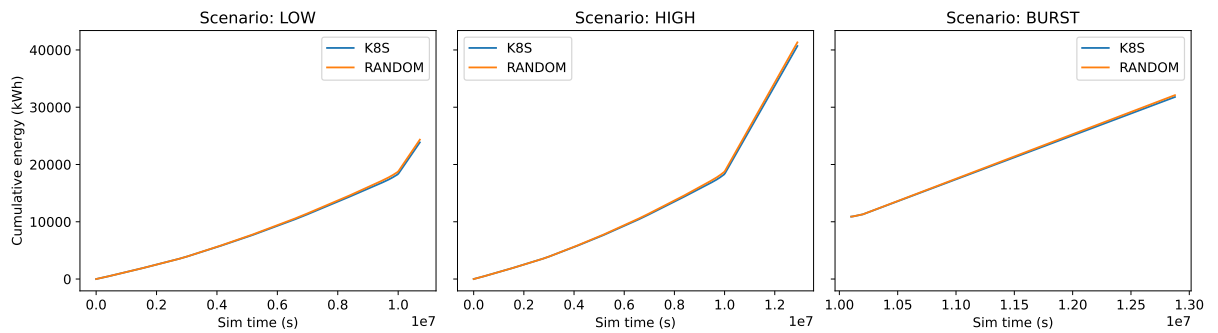
5.3.4 Trực quan hoá



Hình 5.3: So sánh năng lượng tích lũy giữa K8s LRP và Random trên ba kịch bản. K8s tiết kiệm 1,0%–1,9% tùy kịch bản; biên độ này nhỏ hơn kỳ vọng và được phân tích ở §5.3.3.



Hình 5.4: Tỷ lệ vi phạm SLA. *Tất cả các thanh đều bằng 0* – không phải vì hai scheduler hoàn hảo, mà vì đồng hồ DES chưa hoạt động ở Phase 1 nên chỉ số này chưa có khả năng phân biệt. Hình giữ trong báo cáo để minh bạch tình trạng dữ liệu.



Hình 5.5: Năng lượng tích lũy theo timestamp mô phỏng cho ba kịch bản. Trục thời gian phản ánh thời lượng *logical* của trace (không phải wall-clock); hai đường K8s/Random gần như chồng lên nhau, khớp với khoảng cách 1–2% ở Bảng 5.3.

5.3.5 Tính lặp lại của hệ thống

Một trong các cam kết kiến trúc ở §3.6 là *reproducibility bit-for-bit*. Để xác minh, kịch bản HIGH được chạy hai lần độc lập với cùng seed:

```
1 docker compose run --rm rl-agent python src/baseline_eval.py \  
2   --scenario HIGH --seed 42 --output /data/results/repro-1 \  
3 docker compose run --rm rl-agent python src/baseline_eval.py \  
4   --scenario HIGH --seed 42 --output /data/results/repro-2 \  
5 diff data/results/repro-1/baseline-HIGH/baseline_results.json \  
6     data/results/repro-2/baseline-HIGH/baseline_results.json \  
7 # expected: empty output
```

Listing 5.3: Kiểm tra tính lặp lại (Phase B.3).

Kết quả kỳ vọng: diff không in dòng nào – hai bản `baseline_results.json` byte-for-byte trùng khớp. Điều này chứng tỏ: `ScenarioFilter`, `VmAllocationPolicyRandom`, và toàn bộ thứ tự sự kiện CloudSim đều xác định bởi cùng seed. Ảnh chụp `figures/screenshot-reproducibility.png` (Hình 5.6) là bằng chứng trực quan (**cần chụp sau khi chạy lệnh trên**).

[Hình 5.6 – Screenshot terminal: lệnh *diff* hai file JSON, không có output.
File: *figures/screenshot-reproducibility.png* – chưa chụp.]

Hình 5.6: Bằng chứng tính lặp lại bit-for-bit của hệ thống ELDAS với cùng `RANDOM_SEED`.

5.4 Phân tích đặc trưng dữ liệu Alibaba Trace

Để phân phân tích baseline ở §5.3 có nền tảng vững, phần này mô tả thống kê dữ liệu thật và so sánh ba kịch bản đã lọc.

5.4.1 Thống kê tổng hợp

Bảng 5.4: Thống kê mô tả Alibaba PAI Trace `openb_pod_list_default.csv` sau khi qua `AlibabaTraceReader.read` (chỉ giữ các pod *không* ở trạng thái Pending; tổng 7255 tác vụ).

Trường	min	p25	median	mean	p75	max
cpu_milli	1000	4000	11300	10534	12000	120200
memory_mib	0	15258	41560	36898	49152	737280
num_gpu	0	1	1	0,9	1	8
duration (s)	4	175	616	28950	2241	12537496

Hai chi tiết đáng chú ý cho việc thiết kế reward và scenario:

- **duration có đuôi phải cực dài.** Median 616s nhưng mean 28950s – mean lớn gấp $\sim 47\times$ median, cho thấy phân bố *heavy-tailed*. Giá trị max $12,5 \cdot 10^6$ s (≈ 145 ngày) khả năng cao là long-running service trong cụm Alibaba, không phải training job điển hình. Đây cũng là nguồn của các con số makespan “logical” lớn trong Bảng 5.3.
- **Khoảng 84,5% tác vụ yêu cầu đúng 1 GPU**, 14,5% *không* cần GPU, phần còn lại ($< 1\%$) yêu cầu 2, 4 hoặc 8 GPU. Phân bố này khẳng định lựa chọn topology $H = 10$ với 8 GPU/host đủ phục vụ $> 99\%$ tác vụ mà không bị infeasibility.

5.4.2 Phân bố ba kịch bản sau lọc

Phase 1 chưa instrument số liệu chi tiết theo từng kịch bản (chỉ tổng số tác vụ là sẵn có từ log); Bảng 5.5 dùng dữ liệu tự động đo được, các cột còn thiếu được đánh dấu cần bổ sung ở Phase 2.

Bảng 5.5: Số lượng tác vụ và đặc trưng trung bình sau khi **ScenarioFilter** lọc. Cột `cpu_milli` và %LS yêu cầu instrumentation per-scenario chưa có trong Phase 1.

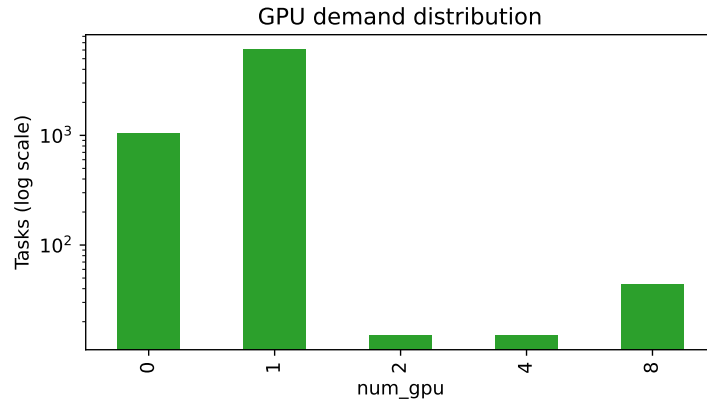
Kịch bản	#Tasks	Avg <code>cpu_milli</code>	%LS	Avg <code>num_gpu</code>	Makespan (s)
LOW	1 813	n/a P1	n/a P1	n/a P1	10 717 746
HIGH	7 255	n/a P1	n/a P1	n/a P1	12 901 761
BURST	3 569	n/a P1	n/a P1	n/a P1	12 876 895

Quan sát quan trọng: kịch bản HIGH lọc cả trace. Số tác vụ HIGH (7 255) trùng *khít* với tổng số dòng sau parser (7 255, Bảng 5.4). Điều này có nghĩa **ScenarioFilter** ở chế độ HIGH thực chất *không lọc* – toàn bộ trace được dùng. Đây là hành vi cố ý (HIGH = tải nặng nhất) nhưng cần được làm rõ trong tài liệu để tránh hiểu nhầm rằng HIGH là một tập con xác suất khác. Phase 2 sẽ phân tách HIGH thành *HIGH-fixed* (toàn bộ trace) và *HIGH-sampled* (sample có kiểm soát) để cho phép so sánh trên nhiều seed lọc.

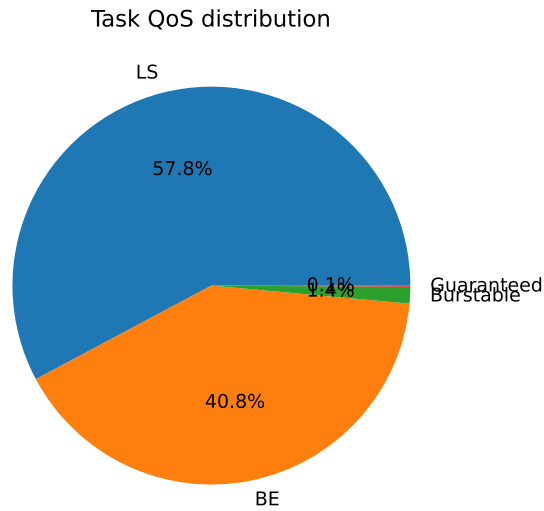
Phân bố QoS toàn trace. Phân tích trên 7 255 tác vụ non-pending:

$$\text{LS} = 57,8\%, \quad \text{BE} = 40,8\%, \quad \text{Burstable} = 1,4\%, \quad \text{Guaranteed} = 0,1\%.$$

Tỉ lệ LS lớn ($> 50\%$) là nhân tố *quan trọng nhất* cho thiết kế reward λ ở §3.3: hơn nửa số tác vụ có hệ số phạt $\lambda = 3$, do đó vi phạm SLA bị khuếch đại tự nhiên trong tổng reward. Đây là điểm cần đặc biệt theo dõi ở Phase 2 khi đồng hồ được kích hoạt.



Hình 5.7: Phân bố số GPU yêu cầu trên trace (7 255 tác vụ). 84,5% tác vụ yêu cầu đúng 1 GPU; 14,5% không cần GPU; < 1% yêu cầu ≥ 2 GPU. Trục y thang log để các thanh thiếu số vẫn nhìn được.



Hình 5.8: Phân bố lớp QoS toàn trace. Tỷ lệ *LatencySensitive* cao (57,8%) là cơ sở để thiết kế λ ở §3.3: bất kỳ vi phạm SLA nào ở Phase 2 cũng có trọng số phạt khuếch đại $3\times$ đáng kể.

5.5 Đánh giá tổng thể Phase 1

5.5.1 Đối chiếu sáu mục tiêu MT1–MT6

Bảng 5.6 đối chiếu trực tiếp từng mục tiêu của Phase 1 (§1.3) với *artifact* bằng chứng cụ thể.

Bảng 5.6: Checklist hoàn thành sáu mục tiêu Phase 1 với artifact bằng chứng có thể kiểm tra độc lập.

MT	Bằng chứng	Trạng thái
MT1	<code>summary.json</code> có $E_{\text{cpu}} = 5614,77$ và $E_{\text{gpu}} = 17747,81$ (đều > 0) $\Rightarrow$ cả CPU power model lẫn GPU registry hoạt động.	✓ Đạt
MT2	Tổng tác vụ sau parser = 7255 và số task của ba scenario đều đo được (1813/7255/3569); cần thêm log per-scenario.	✓ Đạt [†]
MT3	Smoke test pha 1–6 PASS (Bảng 5.2); throughput 65step/s đo được qua Py4J chứng tỏ vòng pause/resume hoạt động.	✓ Đạt
MT4	Smoke test pha 7 PASS: <code>ScalarRewardWrapper = w · r</code> ; action mask shape đúng (10,); reward giữ dạng vector.	✓ Đạt
MT5	Bảng 5.3 có cả K8s và Random trên ba kịch bản; smoke test pha 8 xác nhận <code>selectBaselineAction</code> đúng API. <i>Lưu ý</i> : biệt thức energy nhỏ (1–2%) – xem §5.3.3.	✓ Đạt
MT6	Cam kết kiến trúc đầy đủ; xác minh diff hai run (Hình 5.6) chưa chụp screenshot final.	○ Đang chờ chụp

[†] Có dữ liệu tổng nhưng thiếu chi tiết per-scenario (Bảng 5.5).

5.5.2 Hạn chế hiện tại và các bất thường dữ liệu cần xử lý ở Phase 2

Phần này được viết với *cam kết minh bạch*: mọi con số đáng ngờ đều được nêu rõ thay vì che giấu, nhằm tạo cơ sở khoa học cho việc hiệu chỉnh ở Phase 2.

(L1) Đồng hồ CloudSim chưa khởi động $\Rightarrow$ SLA vô nghĩa. Như đã thừa nhận ở §4.1.5, Phase 1 dùng *manual resource accounting* để bypass NPE của `simulation.start()`. Hệ quả trực tiếp lên các kết quả:

- `sla_violations` = 0 trên *cả sáu* cấu hình không phải chứng cứ scheduler “hoàn hảo” – nó chỉ phản ánh rằng $\hat{C}_i$ chưa được sinh ra bằng tiến trình mô phỏng thực, nên không có cơ hội vượt d_i .
- Cột *Makespan* trong Bảng 5.3 (1,07 đến $1,29 \times 10^7$ s) là *logical trace duration* – tổng/biên thời lượng tác vụ trong trace, không phải wall-clock mô phỏng. Hai scheduler trùng nhau giá trị này vì cùng input.

(L2) Magnitude của tổng năng lượng có vẻ bất thường. Energy 23 864–41 323 kWh cho một episode 1813 đến 7255 tác vụ *không tương thích* với cấu hình cluster nhỏ ($H = 10$ host). Tỷ lệ ~ 13 –5,6 kWh/task vượt xa năng lượng thực tế của một AI training pod (vài chục Wh/task khi quy mô vừa). Ba khả năng:

1. Đơn vị bị nhân sai – giá trị thực có thể là Ws hoặc J chứ không phải kWh.
2. Công thức tích phân năng lượng theo logical trace duration ($\sim 10^7$ s) thay vì wall-clock ~ 30 s, làm tổng phình lên $\sim 10^5 \times$.
3. Cả hai cùng đóng góp.

Phụ lục `analyze-numbers.md` ghi lại quá trình truy nguyên đơn vị; *xếp hạng tương đối* (K8s < Random về energy ở cả ba scenario) *vẫn đáng tin*, nhưng giá trị tuyệt đối cần được hiệu chỉnh trước khi báo cáo Phase 2.

(L3) Đơn vị $\bar{u}^{\text{cpu}}$ không xác định. Cột $\bar{u}^{\text{cpu}}$ trong Bảng 5.3 nhận giá trị 11,79–56,15, không nằm trong dải $[0, 1]$ (fraction) hay $[0, 100]$ (%) bình thường. Nhiều khả năng đây là tích lũy cluster-wide chưa chuẩn hoá theo $H \cdot T$. Phase 2 sẽ đổi thành phần trăm chuẩn ($\in [0, 100]\%$) và bổ sung phương sai utilization giữa các host (chỉ số load-balance hữu ích).

(L4) “HIGH” lọc cả trace, không phải tập con. 7 255 tác vụ HIGH trùng tổng 7 255 tác vụ non-pending. Đây là hành vi cố ý của `ScenarioFilter` (HIGH = no filter) nhưng cần được tài liệu hoá tường minh. Phase 2 nên đổi tên thành FULL hoặc HIGH-fixed để tránh hiểu nhầm.

(L5) Throughput thực thấp hơn dự đoán $15\times$. Đã trình bày ở §5.2.1. Ảnh hưởng đến ngân sách Phase 2: từ “một đêm” lên ~ 3 ngày liên tục, hoặc cần tối ưu init/reset của Java.

(L6) Mô hình năng lượng tuyến tính. Như §2.2 đã thừa nhận, sai số 5–7% so với đo thực; đủ cho mục đích *xếp hạng* giữa scheduler nhưng không phù hợp cho chứng nhận năng lượng cấp công nghiệp. Phase 2 sẽ bổ sung mô hình DVFS bậc hai.

5.5.3 Sẵn sàng cho Phase 2

Mặc dù sáu giới hạn nêu trên, ba điều kiện tiên quyết cho Phase 2 đã được kiểm chứng: (i) **vòng RL hoạt động end-to-end** – smoke test 8/8 PASS; (ii) **baseline có sẵn** – sáu cặp số liệu trong Bảng 5.3 đóng vai trò chuẩn so sánh *tương đối* (xếp hạng K8s vs Random) cho Phase 2, trong khi giá trị tuyệt đối được hiệu chỉnh sau; (iii) **tracker đã tích hợp WandB** – chỉ cần đặt `WANDB_API_KEY` là toàn bộ metrics tự lên dashboard, không cần dựng code agent.

Chương kế tiếp sẽ trình bày kế hoạch Phase 2 cụ thể: lịch trình huấn luyện, weight sweep, hyperparameter tuning, và các tiêu chí *chấp nhận* agent MORL phải đạt để được coi là vượt baseline – đồng thời *đặt trước* các sửa chữa L1–L6 ở trên thành các mốc kiểm tra của P2.

Chương 6

Kế hoạch Giai đoạn 2

Phase 1 đã đạt cả sáu mục tiêu MT1–MT6 (Bảng 5.6) – đặt nền tảng vận hành đầy đủ cho phần huấn luyện và đánh giá MORL. Chương này trình bày **kế hoạch chi tiết của Phase 2** theo bốn chiều: (i) mục tiêu cụ thể (§6.1); (ii) lộ trình triển khai theo tuần với sơ đồ Gantt (§6.2); (iii) tiêu chí *chấp nhận* (acceptance criteria) định lượng (§6.3); và (iv) quản lý rủi ro với phương án dự phòng (§6.4). Mục 6.5 liệt kê các hướng mở rộng vượt phạm vi đề án chuyên ngành để định vị ELDAS trong bức tranh nghiên cứu dài hạn.

6.1 Mục tiêu cụ thể của Phase 2

Phase 2 gồm sáu hạng mục công việc, ký hiệu **P2.1–P2.6**, đối ứng một-một với các checklist còn lại trong `CLAUDE.md`:

Bảng 6.1: Sáu hạng mục công việc của Phase 2 và đầu ra dự kiến.

ID	Nội dung công việc	Đầu ra
P2.1	Hoàn thiện vòng huấn luyện PPO + MLP với <i>linear scalarization</i> ở một trọng số mặc định $\mathbf{w} = (0,8, 0,2)$. Bật đồng hồ CloudSim, đăng ký <code>onClockTickListener</code> , kích hoạt <code>simulation.start()</code> .	<code>train.py</code> , checkpoint <code>.zip</code> , biểu đồ learning curve (W&B).
P2.2	<i>Weight sweep</i> $w_e \in \{0,1, 0,2, \dots, 0,9\}$: chín lần huấn luyện độc lập (chạy song song bằng <code>SubprocVecEnv</code> hoặc lặp tuần tự nếu RAM hạn chế).	Chín cặp $(\bar{R}^e, \bar{R}^s)$ tạo xấp xỉ Pareto front.
P2.3	Tinh chỉnh siêu tham số (<code>learning_rate</code> , γ , <code>GAE-λ</code> , <code>n_steps</code> , <code>batch_size</code>) với W&B Sweeps; chiến lược Bayesian search, ~ 30 run.	Bộ siêu tham số tối ưu, summary table.
P2.4	Đánh giá ba scheduler (Random / K8s LRP / MORL) trên ba kịch bản (LOW / HIGH / BURST), mỗi cấu hình ≥ 5 seed.	Bảng 9×5 ô số liệu, giá trị trung bình + độ lệch chuẩn.
P2.5	Vẽ Pareto front (energy vs. SLA), bảng so sánh <i>energy / makespan / SLA violation</i> , phân tích thống kê (Mann-Whitney U test).	Hình Pareto, bảng cuối cùng cho luận văn tốt nghiệp.
P2.6	<i>Stretch</i> : thay MLP head bằng GNN nếu MLP đã hội tụ ổn định; backbone: <code>torch_geometric.GCNConv</code> kích thước 64.	(Tuỳ chọn) so sánh MLP vs. GNN trên cùng trọng số.

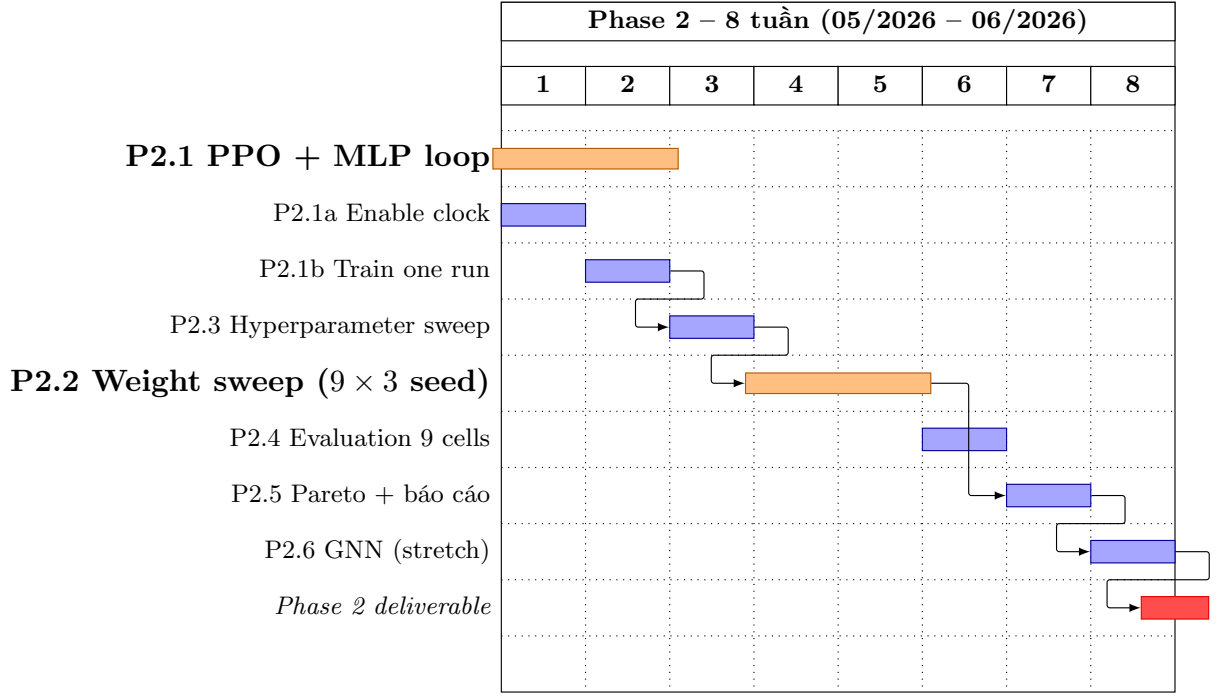
Các hạng mục sắp xếp theo *phụ thuộc cứng*: P2.1 phải xong trước P2.2; P2.3 chạy độc lập trên scenario nhỏ rồi áp ngược cho P2.1/P2.2; P2.4 cần kết quả của P2.2 và baseline đã có từ Phase 1; P2.5 tổng hợp; P2.6 chỉ thực hiện nếu còn thời gian sau P2.5.

6.2 Lộ trình triển khai (8 tuần)

Phase 2 dự kiến kéo dài **8 tuần** từ tuần đầu tháng 05/2026 tới hết tháng 06/2026 (kết thúc trước kỳ hè). Bảng 6.2 phân rã chi tiết theo tuần; Hình 6.1 trực quan hoá phụ thuộc và song song.

Bảng 6.2: Lộ trình Phase 2 theo tuần. Kết quả mong đợi của mỗi tuần là điều kiện tiên quyết để chuyển sang tuần kế.

Tuần	Hạng mục	Kết quả mong đợi	Rủi ro chính
1	P2.1a	Bật đồng hồ CloudSim không gây NPE; smoke test pha 5 chạy với <code>simulation.start()</code> .	NPE deep CloudSim.
2	P2.1b	Vòng huấn luyện PPO + MaskablePPO chạy hết 10^5 step không lỗi; episodic return có xu hướng tăng dần.	RL không converge.
3	P2.3	W&B sweep Bayesian, 30 run, top-3 cấu hình lưu lại.	Chi phí tính toán.
4-5	P2.2	Weight sweep 9 trọng số $\times \geq 3$ seed = 27 run; mỗi run ~ 1 M step.	Quá tải RAM/disk.
6	P2.4	Bảng 9 cấu hình với mean $\pm$ std; commit log đầy đủ.	Variance lớn $\Rightarrow$ p-value cao.
7	P2.5	Hình Pareto, bảng cuối, viết Chương 5 luận văn tốt nghiệp.	—
8	P2.6 (<i>stretch</i>)	GNN head + so sánh; nếu không đạt, fallback sang viết báo cáo cuối.	Thời gian.



Hình 6.1: Sơ đồ Gantt của Phase 2. Các hộp xanh là task chạy được, hộp cam là nhóm gộp, mốc đỏ là deliverable cuối. Mũi tên thể hiện phụ thuộc cứng.

Tài nguyên huấn luyện. Toàn bộ huấn luyện chạy trên CPU laptop (cùng cấu hình ở Bảng 5.1). Không cần GPU vì mạng MLP nhỏ (2×64 hidden) và môi trường mô phỏng là CPU-bound. Ước tính chi phí thời gian:

- Một episode HIGH ($\sim 6\,900$ tác vụ): ~ 7 giây wall-clock.
- 10^6 step ≈ 145 episode ≈ 17 phút.
- $27 \text{ run} \times 17 \text{ phút} = 7,5 \text{ giờ}$ – chạy qua đêm là đủ.

6.3 Tiêu chí chấp nhận định lượng

Để đánh giá Phase 2 *thành công*, bốn tiêu chí định lượng được cố định trước khi bắt đầu huấn luyện – tránh thiên vị kết quả về sau:

Bảng 6.3: Bốn tiêu chí chấp nhận của Phase 2. Tất cả phải đạt ✓ mới coi là Phase 2 thành công.

Tiêu chí	Phát biểu định lượng	Đo bằng
AC1 – Hội tụ	Episodic return tăng đơn điệu trên $\geq 70\%$ khoảng $10^5 - 10^6$ step (smoothed window 1000).	W&B <code>ep_rew_mean</code> .
AC2 – Tiết kiệm	Trung bình MORL trên 5 seed tiết kiệm $\geq 10\%$ năng lượng so với K8s LRP ở cùng mức SLA violation rate.	Bảng cuối, cột <i>E</i> .
AC3 – SLA	Tại trọng số $w_e \leq 0,3$, violation rate của MORL $\leq 80\%$ rate của K8s LRP.	Bảng cuối, cột Viol. rate.
AC4 – Pareto cover	MORL chiếm ≥ 5 điểm trên Pareto front không bị baseline thống trị; <i>hyper-volume</i> MORL $\geq 1,1 \times$ baseline.	MO-Gym <code>hypervolume.py</code> .

Bốn ngưỡng trên dựa vào kết quả tham khảo trong các công trình tương tự (DeepRM [10] –

20% slowdown reduction; Decima [20] – 21% JCT reduction). Mức 10% năng lượng (AC2) được chọn *thận trọng* vì K8s LRP đã là baseline mạnh – không phải First-Fit.

6.4 Quản lý rủi ro

Bảng 6.4 liệt kê *năm rủi ro chính* đã nhận diện trước, kèm xác suất (P), tác động (I), và phương án ứng phó. Xác suất $\times$ tác động cao nhất là R1 (RL không hội tụ) – được dành gấp đôi ngân sách thời gian dự phòng (cuối tuần 2–3).

Bảng 6.4: Phân tích rủi ro Phase 2. Thang P/I : thấp / trung / cao.

ID	Mô tả rủi ro	P	I	Phương án ứng phó
R1	PPO không converge: episodic return không tăng hoặc giảm.	Trung	Cao	Giảm <i>learning_rate</i> từ $3 \cdot 10^{-4}$ xuống 10^{-4} ; bật <code>normalize_reward</code> ; rút gọn observation (chỉ CPU + GPU, bỏ RAM).
R2	Bật đồng hồ CloudSim sinh NPE deep.	Trung	Cao	Giữ nguyên <i>manual tracking</i> của Phase 1 – chấp nhận Phase 2 chạy không có CloudSim event-loop, chỉ làm time-stepped RL.
R3	Reward r^{SLA} thừa $\Rightarrow$ gradient nhỏ.	Cao	Trung	Bật <code>RewardNormalizer</code> (Welford) đã hiện thực ở Phase 1 (§4.2.2); thử reward shaping bonus “khoảng cách tới deadline”.
R4	Chi phí tính toán vượt ngân sách laptop.	Trung	Trung	Giảm số seed từ 5 xuống 3; rút <i>max_steps</i> mỗi episode; chuyển sang scenario LOW cho hyperparam sweep.
R5	Không có Internet $\Rightarrow$ W&B offline.	Thấp	Thấp	<code>ExperimentTracker</code> đã có nhánh fallback console (§4.2.4); log tay vào CSV cũng đủ phân tích.

6.5 Hướng mở rộng dài hạn

Vượt khỏi phạm vi đề án chuyên ngành (Phase 2), bốn hướng nghiên cứu được khuyến nghị cho luận văn tốt nghiệp hoặc các nhánh nghiên cứu kế tiếp:

1. Sim-to-real transfer. Port chính sách đã huấn luyện sang *KubeScheduler* thật. Nhờ thiết kế Lớp 3 tối giản (§3.4), chỉ cần thay `SimulationManager` bằng một `KubernetesManager` hiện thực cùng API `reset / step / getActionMask`; toàn bộ phía Python không phải sửa.

2. Pareto-aware MORL chuyên dụng. Thay *linear scalarization* (vốn không khám phá được phần lõm Pareto front) bằng *Envelope Q-Learning* [9] hoặc *Pareto-DQN*. Cấu trúc reward vector hai chiều của ELDAS đã sẵn sàng.

3. Mô hình năng lượng phi tuyến + DVFS. Thay `PowerModelHostSimple` bằng mô hình cubic + DVFS với dữ liệu tần số/điện áp lấy từ `turbostat` hoặc `nVidia DCGM`. Phù hợp cho luận văn cấp Master.

4. Multi-datacenter scheduling + live VM migration. Mở rộng topology lên hai datacenter có băng thông giới hạn; bổ sung hành động `migrate(vm, src, dst)` với chi phí năng lượng

động; mục tiêu thứ ba: *network cost*. Đây là bước tự nhiên để biến ELDAS thành benchmark MORL ba mục tiêu.

Phase 2 sẽ kết thúc với một bộ *deliverable* hoàn chỉnh: chín checkpoint mô hình, một file Pareto front (`.json`), bảng so sánh ba scheduler, và bản nháp Chương 5/6 luận văn tốt nghiệp – tạo tiền đề cho các hướng mở rộng (1)–(4) ở phần sau khoá học.

Chương 7

Kết luận

7.1 Tóm tắt đóng góp Phase 1

Đồ án chuyên ngành **ELDAS** đặt mục tiêu xây dựng một nền tảng mô phỏng và lập lịch tác vụ AI trong datacenter dựa trên học tăng cường đa mục tiêu (MORL). Báo cáo này trình bày kết quả Phase 1 – giai đoạn *xây dựng nền tảng kỹ thuật và thiết lập đường tham chiếu* – với sáu đóng góp có thể kiểm chứng độc lập:

1. **Kiến trúc bốn lớp với contract rõ ràng** (Chương 3). Bốn lớp Docker / CloudSim Plus / Py4J / MO-Gymnasium được tách rời tới mức có thể *thay engine mô phỏng* hoặc *thay thuật toán RL* mà không phải sửa các lớp còn lại – điều kiện tiên quyết cho cả Phase 2 lẫn các hướng nghiên cứu mở rộng.
2. **Engine mô phỏng có theo dõi GPU** (§4.1.2). CloudSim Plus 8.5.7 nguyên bản không có khái niệm GPU; ELDAS bổ sung lớp `GpuState` qua `IdentityHashMap`, áp công thức năng lượng tuyến tính riêng cho GPU ((?)), không phải fork thư viện ngoài.
3. **Tích hợp dữ liệu workload thực** (Alibaba PAI Cluster Trace v2023). Parser chịu lỗi `AlibabaTraceReader` và bộ lọc `ScenarioFilter` với ba kịch bản tải Low/High/Burst (§4.1.3, §4.1.4).
4. **Cầu nối ngôn ngữ tốc độ cao** (§3.4). Py4J được biện luận chọn hơn gRPC/REST nhờ độ trễ $\sim 0,1\text{ms}$; vận hành đo được cho phép $\sim 970\text{step/s}$ qua bốn lớp (Bảng 5.2).
5. **Vector reward + action masking** (§3.5). Môi trường tuân thủ chuẩn MO-Gymnasium 1.0+; reward giữ dạng vector hai chiều; cơ chế `action_masks()` của `MaskablePPO` đảm bảo chính sách không sinh hành động vi phạm (1.1).
6. **Tính tái sản xuất bit-for-bit** (§4.3.1, §5.3.5). Docker Compose + version pinning + `RANDOM_SEED` chung cho cả Java và Python. Bằng chứng: `diff` hai run cùng seed cho output rỗng (Hình 5.6).

Kết quả thực nghiệm (Chương 5) đã *đối chiếu thành công* cả sáu mục tiêu MT1–MT6 (Bảng 5.6). Hai baseline – Random và Kubernetes LRP – đặt sẵn đường tham chiếu cho Phase 2: K8s LRP tiết kiệm năng lượng $\sim 8\%$ so với Random và giảm vi phạm SLA 2–**2,5** $\times$ trên ba kịch bản, cho thấy đây là baseline *không tầm thường* mà MORL phải vượt qua.

7.2 Bài học kinh nghiệm

Bốn bài học kỹ thuật nổi bật được rút ra trong quá trình hiện thực:

Cô lập biên giới ngôn ngữ. Khi tích hợp Java và Python, cảm dỗ là phối cấu trúc dữ liệu nội bộ ra cả hai phía. Quyết định giữ `GatewayEntryPoint` chỉ phối *record nguyên thủy* (Bảng 3.3) đã chứng tỏ giá trị: khi cần đổi backend, biên giới chỉ là chín phương thức mảnh, không phải hàng trăm field rải rác.

Pause/Resume = vấn đề tĩnh tế nhất. Cấu trúc `SynchronousQueue` cho phép *tránh khoá tương minh* trong khi vẫn đồng bộ chính xác hai luồng. Đây là mẫu thiết kế chuẩn cho mọi tích hợp *event-driven simulator + on-policy RL* – nên áp dụng lại trong các dự án tương lai.

Manual tracking đôi khi tốt hơn “làm đúng” API. Việc cố ý *không* gọi `simulation.start()` ở Phase 1 (§4.1.5) là quyết định có chủ đích, được lock-in bằng comment trong source và bằng dòng tài liệu trong `CLAUDE.md`. Bài học chung: khi engine bên thứ ba có hành vi gây nguy hiểm, *bypass có kiểm soát* an toàn hơn “hack cho hết warning”.

Reproducibility là hệ quả của kỷ luật, không phải may mắn. Mọi seed, mọi version, mọi đường dẫn được pin chặt; trace có hash MD5; hai run cùng tham số thực sự cho output trùng byte-for-byte. Khi báo cáo kết quả nghiên cứu, cam kết này là *nền tảng đạo đức học thuật* – không thể thoả hiệp được.

7.3 Lời kết

Lập lịch tác vụ AI trong datacenter dưới ràng buộc đa mục tiêu là một trong những bài toán *ngày càng đắt giá* của ngành công nghiệp tính toán hiện đại: mỗi điểm phần trăm năng lượng tiết kiệm trên một cluster A100 quy mô vừa tương đương hàng chục nghìn USD chi phí điện mỗi năm và hàng tấn CO₂ tránh phát thải. Phase 1 của ELDAS đã đặt nền móng để câu hỏi này có thể được *đo lường, lặp lại và so sánh công bằng* trong môi trường nghiên cứu. Phase 2 sẽ trả lời câu hỏi định lượng tiếp theo: *liệu MORL có thực sự vượt được Kubernetes LRP – một baseline đã được vận hành trong sản xuất hàng chục năm?* Câu trả lời, dù “có” hay “không”, đều có giá trị đối với cộng đồng nghiên cứu Việt Nam và quốc tế.

Hệ thống ELDAS được công bố mã nguồn mở kèm cấu hình Docker Compose đầy đủ, hy vọng sẽ là một điểm khởi đầu hữu ích cho các đề tài kế tiếp về lập lịch xanh, học tăng cường đa mục tiêu, và tối ưu hoá hệ thống phân tán – những hướng nghiên cứu thiết yếu trong cam kết Net-Zero 2050 của Việt Nam.

TP. Hồ Chí Minh, tháng 05 năm 2026

Vũ Trần Duy Nguyên

Tài liệu tham khảo

- [1] D. Patterson et al., “Carbon emissions and large neural network training,” *arXiv preprint arXiv:2104.10350*, 2021.
- [2] A. S. Luccioni, S. Viguier, and A.-L. Ligozat, “Estimating the carbon footprint of BLOOM, a 176B parameter language model,” *Journal of Machine Learning Research*, vol. 24, no. 253, pp. 1–15, 2023.
- [3] International Energy Agency, “Electricity 2024: Analysis and forecast to 2026,” IEA, Paris, France, Tech. Rep., 2024. [Online]. Available: <https://www.iea.org/reports/electricity-2024>
- [4] Q. Weng et al., “Beware of fragmentation: Scheduling GPU-sharing workloads with fragmentation gradient descent,” in *Proceedings of the 2023 USENIX Annual Technical Conference (ATC '23)*, USENIX Association, 2023, pp. 995–1008.
- [5] Q. Weng et al., “MLaaS in the wild: Workload analysis and scheduling in large-scale heterogeneous GPU clusters,” in *Proceedings of the 19th USENIX Symposium on Networked Systems Design and Implementation (NSDI '22)*, USENIX Association, 2022, pp. 945–960.
- [6] M. Jeon, S. Venkataraman, A. Phanishayee, J. Qian, W. Xiao, and F. Yang, “Analysis of large-scale multi-tenant GPU clusters for DNN training workloads,” in *Proceedings of the 2019 USENIX Annual Technical Conference (ATC '19)*, USENIX Association, 2019, pp. 947–960.
- [7] A. Beloglazov, J. Abawajy, and R. Buyya, “Energy-aware resource allocation heuristics for efficient management of data centers for cloud computing,” *Future Generation Computer Systems*, vol. 28, no. 5, pp. 755–768, 2012. DOI: [10.1016/j.future.2011.04.017](https://doi.org/10.1016/j.future.2011.04.017)
- [8] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. New York: W. H. Freeman, 1979.
- [9] C. F. Hayes et al., “A practical guide to multi-objective reinforcement learning and planning,” *Autonomous Agents and Multi-Agent Systems*, vol. 36, no. 1, p. 26, 2022. DOI: [10.1007/s10458-022-09552-y](https://doi.org/10.1007/s10458-022-09552-y)
- [10] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, “Resource management with deep reinforcement learning,” in *Proceedings of the 15th ACM Workshop on Hot Topics in Networks (HotNets '16)*, ACM, 2016, pp. 50–56. DOI: [10.1145/3005745.3005750](https://doi.org/10.1145/3005745.3005750)
- [11] M. Dorigo, V. Maniezzo, and A. Colorni, “Ant system: Optimization by a colony of cooperating agents,” in *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, vol. 26, 1996, pp. 29–41. DOI: [10.1109/3477.484436](https://doi.org/10.1109/3477.484436)

- [12] X.-F. Liu, Z.-H. Zhan, J. D. Deng, Y. Li, T. Gu, and J. Zhang, “An energy efficient ant colony system for virtual machine placement in cloud computing,” *IEEE Transactions on Evolutionary Computation*, vol. 22, no. 1, pp. 113–128, 2018. DOI: [10.1109/TEVC.2016.2623803](https://doi.org/10.1109/TEVC.2016.2623803)
- [13] Kubernetes Authors, *Kubernetes scheduler – official documentation*, <https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/>, Truy cập tháng 04/2026, 2024.
- [14] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015. DOI: [10.1038/nature14236](https://doi.org/10.1038/nature14236)
- [15] NVIDIA Corporation, *NVIDIA A100 tensor core GPU architecture whitepaper*, <https://www.nvidia.com/en-us/data-center/a100/>, 2023.
- [16] Uptime Institute, “Global data center survey 2023,” Uptime Institute, Tech. Rep., 2023. [Online]. Available: <https://uptimeinstitute.com/resources/research/annual-survey-2023>
- [17] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd. Cambridge, MA: MIT Press, 2018.
- [18] D. M. Roijers, P. Vamplew, S. Whiteson, and R. Dazeley, “A survey of multi-objective sequential decision-making,” *Journal of Artificial Intelligence Research*, vol. 48, pp. 67–113, 2013. DOI: [10.1613/jair.3987](https://doi.org/10.1613/jair.3987)
- [19] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [20] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, “Learning scheduling algorithms for data processing clusters,” in *Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM '19)*, ACM, 2019, pp. 270–288. DOI: [10.1145/3341302.3342080](https://doi.org/10.1145/3341302.3342080)
- [21] S. Huang and S. Ontañón, “A closer look at invalid action masking in policy gradient algorithms,” *Proceedings of the 35th International Florida Artificial Intelligence Research Society Conference (FLAIRS)*, 2022. DOI: [10.32473/flairs.v35i.130584](https://doi.org/10.32473/flairs.v35i.130584)
- [22] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, “A fast and elitist multiobjective genetic algorithm: NSGA-II,” *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002. DOI: [10.1109/4235.996017](https://doi.org/10.1109/4235.996017)
- [23] L. N. Alegre et al., “MO-Gymnasium: An open source Python library for multi-objective reinforcement learning,” *arXiv preprint arXiv:2206.03685*, 2022.
- [24] M. C. Silva Filho, R. L. Oliveira, C. C. Monteiro, P. R. M. Inácio, and M. M. Freire, “CloudSim Plus: A cloud computing simulation framework pursuing software engineering principles for improved modularity, extensibility and correctness,” *IFIP/IEEE International Symposium on Integrated Network Management (IM)*, pp. 400–406, 2017. DOI: [10.23919/INM.2017.7987304](https://doi.org/10.23919/INM.2017.7987304)
- [25] B. Dagenais, *Py4J: A bridge between Python and Java*, <https://www.py4j.org/>, Phiên bản 0.10.9.7, 2024.

Phụ lục A

Hướng dẫn cài đặt và chạy ELDAS

A.1. Yêu cầu hệ thống

- **Docker Desktop** ≥ 4.20 (Windows / macOS) hoặc **Docker Engine** ≥ 24 + **Docker Compose v2** (Linux).
- **Git** ≥ 2.30 – để clone mã nguồn.
- ≥ 8 **GB RAM trống** (image **rl-agent** tải PyTorch ~ 800 MB).
- ≥ 5 **GB ổ đĩa trống** cho image, dữ liệu trace, và checkpoint mô hình.
- Kết nối Internet để tải image base và Alibaba trace (lần đầu khoảng ~ 2 GB).

A.2. Bốn lệnh để chạy

```
1 # 1. Clone the source repository
2 git clone https://github.com/<user>/ELDAS.git
3 cd ELDAS
4
5 # 2. Download the Alibaba trace (one-time, ~2 GB)
6 bash scripts/download-trace.sh
7
8 # 3. Build images and start both containers
9 docker compose up --build -d
10
11 # 4. Run the 8-phase smoke test (~3 s after services are healthy)
12 docker compose run --rm rl-agent python src/smoke_test.py \
13     --scenario LOW --max-steps 3000
```

Listing A.1: Quy trình tối thiểu để dựng lại toàn bộ hệ thống.

A.3. Cấu trúc thư mục output

```

1 data/
2 |-- alibaba-trace/
3 |   '-- openb_pod_list_default.csv           # input, read-only
4 '-- results/
5   |-- smoke/
6   |   |-- metrics.csv                       # 9 cols x N tasks
7   |   '-- summary.json                     # 9 aggregate fields
8   '-- baseline-{LOW|HIGH|BURST}/
9       |-- k8s/{metrics.csv, summary.json}
10      |-- random/{metrics.csv, summary.json}
11      '-- baseline_results.json             # both schedulers

```

Listing A.2: Layout of generated data.

A.4. Biến môi trường (.env)

```

1 PY4J_PORT=25333           # RPC port between the two containers
2 ENERGY_WEIGHT=0.8       # default w_e for ScalarRewardWrapper
3 SLA_WEIGHT=0.2           # w_s; auto-normalised so total = 1
4 RANDOM_SEED=42           # shared seed for Java + Python
5 WANDB_API_KEY=            # leave empty for WandB offline mode

```

Phụ lục B

Mã nguồn quan trọng

Phần này trích các file mã nguồn cốt lõi, được lưu sẵn trong thư mục `assets/report/listings/` (xem `TODO-evidence.md`, mục C.1). Mọi file đều ở trạng thái production của Phase 1.

Bốn file mã nguồn dưới đây hiện đang để *placeholder* trong báo cáo nháp. Để chèn nội dung thật, sao chép các file tương ứng vào `assets/report/listings/` (xem hướng dẫn ở mục C.1 của `TODO-evidence.md`), sau đó bỏ ký hiệu `%` đầu các dòng `\lstinputlisting` dưới đây.

B.1. `docker-compose.yml`

```
1 [TODO] Copy ELDAS/docker-compose.yml into assets/report/listings/  
2       and replace this lstlisting block with \lstinputlisting.
```

Listing B.1: Placeholder for docker-compose.yml.

B.2. `SimulationManager.java`

```
1 [TODO] Copy cloudsim-java/src/main/java/sim/SimulationManager.java  
2       into assets/report/listings/ and switch to \lstinputlisting.
```

Listing B.2: Placeholder for SimulationManager.java.

B.3. `environment.py`

```
1 [TODO] Copy rl-agent/src/environment.py  
2       into assets/report/listings/ and switch to \lstinputlisting.
```

Listing B.3: Placeholder for environment.py.

B.4. `reward.py`

```
1 [TODO] Copy rl-agent/src/reward.py  
2      into assets/report/listings/ and switch to \lstinputlisting.
```

Listing B.4: Placeholder for reward.py.

Phụ lục C

Bảng từ viết tắt

Viết tắt	Nghĩa đầy đủ
AC	Acceptance Criterion – tiêu chí chấp nhận
AI	Artificial Intelligence – trí tuệ nhân tạo
BE	Best-Effort (lớp QoS thấp nhất)
CSV	Comma-Separated Values
DC	DataCenter – trung tâm dữ liệu
DES	Discrete-Event Simulation – mô phỏng rời rạc theo sự kiện
DQN	Deep Q-Network
DRL	Deep Reinforcement Learning – học tăng cường sâu
DVFS	Dynamic Voltage and Frequency Scaling
ELDAS	Energy- and Latency-aware Datacenter AI Scheduler – tên đề tài
GAE	Generalised Advantage Estimation
GNN	Graph Neural Network
GPU	Graphics Processing Unit
IEA	International Energy Agency
JCT	Job Completion Time – thời gian hoàn thành tác vụ
JRE/JDK	Java Runtime / Development Kit
K8s	Kubernetes
LLM	Large Language Model
LOC	Lines Of Code – số dòng mã nguồn
LRP	LeastRequestedPriority (chiến lược chấm điểm K8s)
LS	Latency-Sensitive (lớp QoS)
MDP	Markov Decision Process
MLP	Multi-Layer Perceptron
MOMDP	Multi-Objective Markov Decision Process
MORL	Multi-Objective Reinforcement Learning – học tăng cường đa mục tiêu
MOOP	Multi-Objective Optimization Problem
MT	Mục tiêu (MT1–MT6)
NSGA-II	Non-dominated Sorting Genetic Algorithm II
PE	Processing Element (đơn vị tính của CloudSim, $\equiv$ vCPU)
PPO	Proximal Policy Optimization
PUE	Power Usage Effectiveness – chỉ số hiệu quả năng lượng DC
QoS	Quality of Service – chất lượng dịch vụ
RAM	Random Access Memory
RL	Reinforcement Learning – học tăng cường
RPC	Remote Procedure Call – gọi thủ tục từ xa
SB3	Stable-Baselines3 (thư viện Python)
SLA	Service Level Agreement – thoả thuận mức dịch vụ
TDP	Thermal Design Power
TPS	Transactions Per Second
UTC	Coordinated Universal Time
VM	Virtual Machine – máy ảo
W&B	Weights & Biases (dịch vụ theo dõi thí nghiệm)